



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Assisted Python Programming for Finance

TGS Ref No: TGS-2025052659

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 31.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
30.0	2 May 2026	Previous release — IBF-accredited Learner Guide for AI Assisted Python Programming for Finance.	Dr. Alfred Ang
31.0	20 July 2026	Rebuilt from the single-source content module. Expanded to 62 hands-on finance labs across the nine topics; all labs migrated to the uv project manager; added a progressive Streamlit finance-dashboard capstone; full build-out of Topics 5-9; added a Financial Formula Reference and an expanded glossary.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction.....	5
Course Learning Outcomes.....	5
About IBF-STC Accreditation and Funding.....	5
Before You Start — Environment Setup.....	6
Topic 01 — Introduction to Python Programming (8%).....	7
Lab 1 — Set Up an AI-Assisted Python Environment with uv.....	8
Lab 2 — Hello World — Your First Finance Script.....	9
Lab 3 — CPF Contribution Calculator.....	10
Lab 4 — From Business Requirement to Programming Objective — SGX Price Fetcher.....	12
Topic 02 — Data Types and Operators (12%).....	13
Lab 5 — Number and String Literal Formatting for Financial Reporting.....	14
Lab 6 — Loan Amortisation Calculator.....	15
Lab 7 — Retirement Savings Calculator.....	17
Lab 8 — String Methods and Extracting the Company Name from an Email.....	18
Lab 9 — Lists — Top 10 SGX Companies by Market Capitalisation.....	20
Lab 10 — Tuples — Immutable Instrument Identifiers.....	22
Lab 11 — Dictionaries — Company Stock Values and Market Cap Lookup.....	23
Lab 12 — Sets and Operators — Common Holdings Across Portfolios.....	25
Topic 03 — Problem Solving with Control Structures (14%).....	27
Lab 13 — If-Else PE Ratio Valuation — BUY / HOLD / SELL.....	27
Lab 14 — Activity: Return on Equity (ROE) Screener with a BUY / SELL Threshold.....	29
Lab 15 — If-Elif Income Grouping and Credit Risk Banding.....	30
Lab 16 — Ternary Operator — One-Line Trading Decisions.....	32
Lab 17 — While Loop — Fibonacci Sequence and the Golden Ratio in Technical Analysis.....	34
Lab 18 — For Loop and Range — Top 10 SGX Return on Assets (ROA) Ranking.....	35
Lab 19 — Enumerate, Zip, Break and Continue — Combined ROE and ROA Scanner.....	37
Lab 20 — Comprehensions — Net Profit Margin for 10 Singapore Stocks in One Line.....	39
Topic 04 — Scripting with Function and Lambda (14%).....	41
Lab 21 — Functions — A Reusable Financial Formula Library.....	41
Lab 22 — Return Multiple Values — A Full Income Statement Analysis in One Call.....	43
Lab 23 — Activity: Net Profit Margin Function with Zero-Revenue Handling.....	45
Lab 24 — Multiple, Default and Named Arguments — A Flexible Valuation Toolkit.....	47
Lab 25 — Variable Arguments — *args and **kwargs for Portfolios of Any Size.....	49
Lab 26 — Activity: Max ROE Function — Finding the Best Performer in a Portfolio.....	51

Lab 27 — Lambda and Map — Gross Profit Margin and ROE Across a Portfolio.....	53
Lab 28 — Filter — Top Income Customers and High-ROE Stock Screening.....	55
Topic 05 — Error Handling Using Exception (10%).....	57
Lab 29 — Exceptions vs Syntax Errors in a Price Loader.....	57
Lab 30 — Try/Except on a Failing Market-Data Fetch.....	58
Lab 31 — ZeroDivisionError in an ROE and Ratio Calculator.....	60
Lab 32 — KeyError and ValueError in a Portfolio Input Handler.....	61
Lab 33 — Else, Finally and a Custom InsufficientDataError for a Risk Model.....	63
Lab 34 — Activity: Loan Risk Classifier with Guarded Data Loading.....	64
Topic 06 — Import and Process Finance Data (14%).....	66
Lab 35 — Pandas Series vs DataFrame with Price Data.....	66
Lab 36 — Import Finance Data from CSV.....	68
Lab 37 — Import Live Market Data with yfinance.....	69
Lab 38 — Inspect a Dataset with head, info and describe.....	71
Lab 39 — Filter and Slice by Date, Sector and Ticker.....	73
Lab 40 — Handle Missing Market Data and Compare the Choices.....	74
Lab 41 — Derived Columns — Daily Returns and Moving Averages.....	76
Lab 42 — Activity: Algorithmic Trading — Moving Average Crossover Backtest.....	78
Topic 07 — Aggregate and Visualize Finance Data (14%).....	80
Lab 43 — Generate Mock Credit-Loan Data with df_finance.....	80
Lab 44 — Join Finance Datasets with concat, append and merge.....	82
Lab 45 — Groupby: Annual Income by Gender and Region.....	83
Lab 46 — Loan Application Counts by Status and Type.....	84
Lab 47 — Multi-Metric Aggregation by Employment Sector with .agg().....	85
Lab 48 — Pivot Table of Debt-to-Income Ratio and Matplotlib Visualisation.....	87
Lab 49 — Streamlit Capstone Part 1 — Portfolio & Loan Analytics App Shell.....	88
Lab 50 — Streamlit Capstone Part 2 — Charts and Pivot Tab in the Dashboard.....	90
Topic 08 — Object Oriented Programming (8%).....	91
Lab 51 — Build a Stock Class — Attributes, Methods and __init__.....	91
Lab 52 — A Portfolio Class Holding Many Stock Objects.....	93
Lab 53 — Dunder Methods — __str__, __repr__, __len__ and __add__ to Combine Portfolios.....	94
Lab 54 — Inheritance — an Instrument Base Class with Equity and Bond Subclasses.....	96
Lab 55 — Polymorphism — One .value() Call Across a Mixed Instrument Book.....	97
Lab 56 — StockTechnicalAnalyzer — an OOP Class Computing SMA, RSI and BUY/SELL Signals.....	99
Topic 09 — Analyze Finance Data (6%).....	100
Lab 57 — Credit Risk Classification with .apply().....	101

Lab 58 — Lambda versus Named Functions in <code>.apply()</code>	102
Lab 59 — Building a clean → enrich → score Pipeline with <code>.pipe()</code>	104
Lab 60 — Descriptive Statistics, Correlation and Tracking Change Over Time.....	105
Lab 61 — Streamlit Capstone Part 3 — Technical Analysis and Risk Scoring Pages.....	107
Lab 62 — Streamlit Capstone Finale — Test, Polish and Deploy the Analytics App.....	109
Financial Formula Reference.....	110
Assessment Preparation.....	111
Glossary.....	111

Introduction

This Learner Guide accompanies the course AI Assisted Python Programming for Finance (TGS-2025052659), conducted by Tertiary Infotech Academy Pte Ltd. The course is accredited under the Skills Framework for Financial Services, IBF Standard FSE-DIT-3018-1.1, and is eligible for funding under the IBF Standards Training Scheme (IBF-STs). This guide provides step-by-step instructions for all 62 hands-on labs, organised by the 9 course topics.

Every lab is grounded in a real financial-services task — CPF and loan calculations, SGX market data, financial ratios such as ROE, ROA and net profit margin, credit-risk classification, algorithmic-trading backtests and portfolio analytics. You will use AI assistants (Google Colab's Gemini, GitHub Copilot in VS Code, or Cursor) to draft code quickly, and then review, correct and test every line before relying on it.

Use this guide alongside the course slides and the lab files in the labs/ folder. Each lab is also provided as a standalone Python file, and all labs are combined into a single Jupyter notebook you can open in Google Colab.

Course Learning Outcomes

- LO1: Translate business requirements in financial services into Python programming objectives, and set up an AI-assisted Python development environment.
- LO2: Apply Python data types, operators and control structures to solve financial calculation and classification problems.
- LO3: Construct reusable functions and lambda expressions to compute financial ratios and metrics for business use cases.
- LO4: Implement error handling to build robust financial programs that survive missing, zero or malformed market data.
- LO5: Import, clean, filter and process financial data using the pandas package.
- LO6: Aggregate and visualise financial data with groupby, pivot tables and charts, and publish the results as an interactive Streamlit application.
- LO7: Apply object-oriented programming to model financial instruments and analysers.
- LO8: Analyse financial data using apply and pipe pipelines, assess code to identify gaps, and test the results.

About IBF-STs Accreditation and Funding

This course has been accredited under the Skills Framework for Financial Services, IBF Standards: FSE-DIT-3018-1.1, and is eligible for funding under the IBF Standards Training Scheme (IBF-STs), subject to all eligibility criteria being met. Participants are advised to assess the suitability of the course and its relevance to his/her business activities or job roles. The IBF-STs is available to eligible entities and individuals based on the prevalent funding eligibility, quantum and caps. IBF-STs provides 50% - 70%

course fee subsidy support for direct training costs subject to a cap of S\$3,000 per candidate per course subject to all eligibility criteria being met.

Find out more at www.ibf.org.sg.

Before You Start — Environment Setup

What you need

- A computer with internet access, or a Google account for Google Colab (browser-based, nothing to install).
- Python 3.12 or later for local work, installed together with uv.
- An AI-assisted editor: Google Colab (built-in Gemini), Visual Studio Code with GitHub Copilot, Cursor, or AntiGravity.
- An internet connection for the labs that download live market data through the yfinance package.

Install uv — the project manager used in every lab

This course uses uv, a single fast tool that replaces pip, virtualenv and pyenv. uv creates an isolated environment per lab and records exact dependency versions, so your results are reproducible — an audit requirement in financial services. Install it once, then verify:

```
# macOS / Linux
```

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# Windows (PowerShell)
```

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1  
| iex"
```

```
# verify the installation
```

```
uv --version
```

The three uv commands you will use in every lab

```
uv init lab-01-hello-finance      # create a new project folder +  
pyproject.toml
```

```
cd lab-01-hello-finance
```

```
uv add pandas yfinance matplotlib # install and record dependencies
```

```
uv run python main.py            # run inside the project environment
```

Working in Google Colab instead

If you prefer Colab, open a new notebook and install packages in a cell with a leading exclamation mark (for example `!pip install yfinance`). The lab logic is identical; only the environment setup differs. The combined notebook for this course is provided in the Activities folder.

Conventions used in every lab

- Commands prefixed with `$` are typed into your terminal; the `$` itself is not part of the command.
- Each lab is a self-contained uv project in its own folder, named `lab-NN-<short-title>`.
- Where a lab depends on a file produced by an earlier lab, the step says so explicitly and copies it forward.
- AI-prompting steps show the exact prompt in quotation marks — type it, then review the generated code against the review step that follows.
- Financial figures use Singapore dollars unless a lab states otherwise; market data is live, so exact values will differ from the examples.

A note on AI-generated code

An AI assistant will produce plausible-looking code very quickly. In financial services, plausible is not the same as correct: a wrong sign, an off-by-one on a rolling window, or a look-ahead bias in a backtest produces numbers that look reasonable and are wrong. Every lab therefore pairs an AI-prompting step with a review step. Never submit or deploy generated code you have not read, run and tested.

Topic 01 — Introduction to Python Programming (8%)

Business requirements · Applications in financial services · AI-assisted Python IDE setup with uv

Key concepts

- Python is the leading language in financial services for algorithmic trading, financial analysis and modelling, portfolio management, fraud detection and risk management.
- Business requirements are translated into programming objectives before any code is written.
- AI-assisted development tools — Google Colab, VS Code, Cursor and AntiGravity — generate, explain and refactor Python code from natural-language prompts.
- uv is the modern Python project manager: `uv init` creates a project, `uv add` installs dependencies, and `uv run` executes scripts in a reproducible environment.
- Every prompt-generated snippet must be reviewed, tested and understood before it enters a production financial system.

Lab 1 — Set Up an AI-Assisted Python Environment with uv

Learning objective: LO1: Set up an AI-assisted Python development environment and translate a finance business requirement into a programming objective.

Goal: The learner installs uv, creates the first finance project, adds the market-data and analysis dependencies used throughout the course, and confirms that the AI assistant in Google Colab / VS Code / Cursor can generate and explain Python code. The learner then writes the environment check as a short finance script that prints the SGX tickers the course will analyse.

What you'll build

A working uv project `lab-01-hello-finance` with yfinance and pandas installed, and an environment-check script that prints the course SGX watchlist. (Tools: uv, Python 3.12, yfinance, pandas, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

1. Discuss the business requirement: a Singapore bank's analyst team needs a repeatable Python environment so that every desk runs the same library versions when valuing SGX equities. Write the requirement as one programming objective.
2. Install uv, the fast Python package and project manager used for every lab in this course.

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

3. Confirm uv is on the PATH and note the version for your lab record.

```
uv --version
```

4. Create the first project for this topic.

```
uv init lab-01-hello-finance
```

5. Move into the project and pin the Python version the course standardises on.

```
cd lab-01-hello-finance && uv python pin 3.12
```

6. Add the market-data and analysis dependencies. uv resolves and locks them into pyproject.toml and uv.lock.

```
uv add yfinance pandas
```

7. Inspect the locked dependency tree — reproducibility is an audit requirement in financial services.

```
uv tree
```

8. PROMPT THE AI ASSISTANT (Colab Gemini / Cursor / Copilot chat) with: 'Write a Python script named env_check.py that prints the Python version, the installed

yfinance and pandas versions, and then prints a numbered watchlist of these SGX tickers with their company names: D05.SI DBS, U11.SI UOB, O39.SI OCBC, Z74.SI Singtel, C38U.SI CapitaLand Integrated Commercial Trust. Use f-strings and no external API calls.' Save the generated code as `env_check.py`.

9. REVIEW the AI output before running it: check that it imports only yfinance and pandas, makes no network call, and that all five tickers carry the correct .SI suffix. Correct anything the assistant got wrong.
10. Run the script inside the uv-managed environment.

```
uv run python env_check.py
```

11. Record in your lab notes one thing the AI assistant got wrong or left out — this habit of verification is the core discipline of AI-assisted development in a regulated environment.

Test it

``uv run python env_check.py`` prints the Python version, the yfinance and pandas versions, and all five SGX tickers with their company names, with no traceback.

Note: The runnable source for this lab is `labs/lab-01-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 2 — Hello World — Your First Finance Script

Learning objective: LO1: Translate a simple business requirement in financial services into a Python programming objective and execute the resulting script.

Goal: The learner writes and runs the classic first program, then immediately upgrades it into a finance-relevant script: a trading-desk banner that prints the desk name, the reporting currency, the valuation date and a formatted portfolio value. The lab establishes the prompt-review-test cycle used for every later lab.

What you'll build

A script ``hello_finance.py`` that prints a formatted trading-desk banner with a currency-formatted portfolio value. (Tools: uv, Python 3.12, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

12. Create and enter the project for this lab.

```
uv init lab-02-hello-world && cd lab-02-hello-world
```

13. Create `hello_finance.py` containing the single classic line, then run it to prove the toolchain works end to end.

```
print('hello world')
```

14. Execute the script with uv.

```
uv run python hello_finance.py
```

15. Now state the business requirement: the SGD equities desk wants every report to open with a banner showing the desk name, base currency, valuation date and total portfolio value in SGD to two decimal places with thousands separators.

16. Add the variables by hand so you understand what the AI will later generate.

```
desk = 'SGD Equities Desk'; base_ccy = 'SGD'; portfolio_value = 12487650.75
```

17. Print the banner using an f-string with currency formatting.

```
print(f'{desk} | {base_ccy} | Portfolio Value: ${portfolio_value:,.2f}')
```

18. PROMPT THE AI ASSISTANT with: 'Refactor hello_finance.py so it prints a bordered banner 60 characters wide. The banner must show the desk name, base currency SGD, today's valuation date from datetime, and the portfolio value formatted with a dollar sign, thousands separators and two decimals. Use only the standard library.' Apply the generated code.

19. REVIEW the AI output: confirm it uses datetime.date.today() rather than a hard-coded date, that the currency format is ,.2f and not .2f, and that no third-party import was added.

20. Run the refactored script and check the alignment of the banner.

```
uv run python hello_finance.py
```

21. Change portfolio_value to 0.0 and to 1234567890.12 and re-run — confirm the formatting holds at both extremes.

```
uv run python hello_finance.py
```

Test it

The script prints a bordered banner showing the desk name, SGD, today's date and the portfolio value as \$12,487,650.75 — with the comma separators and exactly two decimal places.

Note: The runnable source for this lab is labs/lab-02-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 3 — CPF Contribution Calculator

Learning objective: LO1: Translate the CPF contribution business rules into a Python programming objective and implement them as a working script.

Goal: Mirroring the reference notebook activity, the learner builds a CPF calculator that applies the age-banded employee and employer contribution rates against the Ordinary Wage ceiling, then hardens it with input validation and a formatted contribution breakdown suitable for a payroll report.

What you'll build

A script `cpf\_calculator.py` that takes a monthly salary and age and prints the employee, employer and total CPF contribution with the wage ceiling applied. (Tools: uv, Python 3.12, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

22. Create the project.

```
uv init lab-03-cpf-calculator && cd lab-03-cpf-calculator
```

23. Write the business rules in plain English first: rates are banded by age (55 and below, 56–60, 61–65, 66–70, above 70); the Ordinary Wage ceiling caps the contributable salary at \$6,800.

24. Create `cpf\_calculator.py` and define the function signature and the age bands.

```
def calculate_cpf(salary, age):
```

25. Implement the age-banded rates with if/elif — employee and employer rate as a pair.

```
if age <= 55: emp_rate, emr_rate = 0.20, 0.17
```

26. Apply the Ordinary Wage ceiling before computing any contribution.

```
capped_salary = min(salary, 6800)
```

27. Return the three figures the payroll report needs.

```
return capped_salary * emp_rate, capped_salary * emr_rate, capped_salary *  
(emp_rate + emr_rate)
```

28. Collect the inputs and print the breakdown to two decimal places.

```
sal = float(input('Enter monthly salary: ')); age = int(input('Enter age: '))
```

29. Run it with a salary of 5000 and age 30, then with 9000 and age 30 to see the ceiling bite.

```
uv run python cpf_calculator.py
```

30. PROMPT THE AI ASSISTANT with: 'Harden `cpf\_calculator.py`. Reject a negative or non-numeric salary and an age below 16 or above 100 with a clear message instead of a traceback. Print the capped salary alongside the raw salary so the user can see when the \$6,800 Ordinary Wage ceiling was applied, and format every amount as

\$x,xxx.xx. Keep `calculate_cpf` a pure function that returns three values.' Apply the generated code.

31. REVIEW the AI output: verify the rate bands were not altered, that `min(salary, 6800)` is still applied before the rate, and that the totals still equal employee + employer.
32. Test the edge cases: salary 0, salary 20000, age 56, age 71, and a non-numeric salary such as 'abc'.

```
uv run python cpf_calculator.py
```

Test it

A salary of 9000 at age 30 returns an employee contribution of \$1,360.00, an employer contribution of \$1,156.00 and a total of \$2,516.00 — proving the \$6,800 ceiling was applied. Entering 'abc' as the salary prints a friendly message, not a traceback.

Note: The runnable source for this lab is `labs/lab-03-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 4 — From Business Requirement to Programming Objective — SGX Price Fetcher

Learning objective: LO1: Translate a financial services business requirement into a programming objective and set up an AI-assisted Python environment that fetches live SGX market data.

Goal: The learner practises the requirement-to-objective translation that opens every real project: a written business requirement from a wealth desk is decomposed into inputs, processing, outputs and acceptance criteria, then implemented as a first live market-data script using `yfinance` against the SGX tickers used throughout the course.

What you'll build

A requirements note plus `sgx_prices.py`, which fetches and prints the latest close for DBS, UOB, OCBC, Singtel and CapitaLand. (Tools: `uv`, Python 3.12, `yfinance`, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

33. Create the project and add the market-data dependency.

```
uv init lab-04-sgx-prices && cd lab-04-sgx-prices && uv add yfinance
```

34. Read the business requirement aloud: 'Each morning the wealth desk needs the previous close of our five core SGX holdings, in SGD, on one screen, so advisers can brief clients before the market opens.'
35. Decompose it in a plain-text file `requirements.md` under four headings — INPUTS (the five tickers), PROCESSING (fetch last close), OUTPUTS (ticker, name, close

price), ACCEPTANCE CRITERIA (all five print, two decimals, no crash if one ticker fails).

36. Define the watchlist in Python as the programming objective's input.

```
tickers = ['D05.SI', 'U11.SI', 'O39.SI', 'Z74.SI', 'C38U.SI']
```

37. Fetch one ticker manually first to see the shape of the data before automating.

```
uv run python -c "import yfinance as yf;
print(yf.Ticker('D05.SI').history(period='5d')['Close'])"
```

38. PROMPT THE AI ASSISTANT with: 'Write sgx_prices.py. For each ticker in D05.SI, U11.SI, O39.SI, Z74.SI, C38U.SI use yfinance to fetch the most recent closing price and the short name. Print one aligned row per ticker showing the ticker, the company name and the close formatted as SGD with two decimals. If a ticker returns no data, print NO DATA for that row and continue to the next ticker.' Save the result.

39. REVIEW the AI output against your acceptance criteria: does it handle the empty-data case, does it continue after a failure, and does it format to two decimals?

40. Run the script.

```
uv run python sgx_prices.py
```

41. Introduce a deliberately invalid ticker such as 'ZZZZ.SI' into the list and re-run to prove the acceptance criterion about failure is genuinely met.

```
uv run python sgx_prices.py
```

42. Discuss: which of your four headings did the AI get right without being told, and which did you have to specify? This is the difference between a prompt and a requirement.

Test it

`uv run python sgx\_prices.py` prints five aligned rows with company names and SGD closing prices to two decimal places, and prints NO DATA rather than crashing when the invalid ticker is included.

Note: The runnable source for this lab is labs/lab-04-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Topic 02 — Data Types and Operators (12%)

Numbers · Strings · List · Tuple · Dictionary · Set · Operators

Key concepts

- Python supports six major data types used constantly in finance: number, string, list, tuple, dictionary and set.
- String literal formatting (f-strings) renders currency, percentages and millions in board-ready precision.
- Lists are ordered and mutable — ideal for time series of prices; tuples are immutable — ideal for fixed instrument identifiers.
- Dictionaries map a ticker key to a value such as market capitalisation, and sets remove duplicate tickers and find common holdings across portfolios.
- Arithmetic, compound, comparison, membership and logical operators express financial rules directly in code.

Lab 5 — Number and String Literal Formatting for Financial Reporting

Learning objective: LO2: Apply Python number and string data types and literal formatting to present financial figures in board-ready precision.

Goal: The learner works with int and float financial values and uses f-string literal formatting to render revenue and market capitalisation in millions and billions, percentages to basis point precision, and currency with thousands separators — then confronts float rounding, the reason Decimal exists in settlement systems.

What you'll build

A script `format\_report.py` that renders a set of raw financial figures as a formatted revenue and market-cap summary in millions. (Tools: uv, Python 3.12, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

43. Create the project.

```
uv init lab-05-number-format && cd lab-05-number-format
```

44. Create format_report.py with two raw revenue figures using Python's numeric underscore separators for readability.

```
num1 = 12_345_678.90; num2 = 7_890_123.45; result = num1 + num2
```

45. Format the sum in millions to two decimals — the standard unit in an SGX results announcement.

```
print(f'{num1/1_000_000:.2f}M + {num2/1_000_000:.2f}M =  
{result/1_000_000:.2f}M')
```

46. Add a market-cap line in billions and a margin line as a percentage.

```
print(f'Market cap: {84_500_000_000/1_000_000_000:.2f}B | Net margin:  
{0.2837:.2%}')
```

47. Demonstrate the float trap that matters in settlement: run the classic check and note the result is not exactly 0.3.

```
uv run python -c "print(0.1 + 0.2, 0.1 + 0.2 == 0.3)"
```

48. Discuss why a bank posts ledger entries with Decimal or integer cents rather than float, and where the tolerance threshold sits in your own organisation.

49. PROMPT THE AI ASSISTANT with: 'Extend format_report.py with a function format_sgd(amount) that returns a string: values of 1 billion or more as \$x.xx^B, values of 1 million or more as \$x.xx^M, values of 1 thousand or more as \$x.x^K, and anything smaller as \$x,xxx.xx. Include a right-aligned printed table of DBS 84500000000, Singtel 52300000000, CapitaLand 13800000000 and a small holding of 4250.75.' Apply the code.

50. REVIEW the AI output: check the thresholds are tested largest-first, that negative amounts do not break the sign, and that the alignment uses an f-string width specifier.

51. Run the script and inspect the alignment of the table.

```
uv run python format_report.py
```

52. Test the boundaries: pass exactly 1_000_000, exactly 999_999.99, 0 and -2_500_000 to format_sgd and confirm each renders sensibly.

```
uv run python format_report.py
```

Test it

format_sgd returns '\$84.50^B' for DBS, '\$13.80^M'-style output for millions, and the printed table is right-aligned with every figure carrying its correct unit suffix.

Note: The runnable source for this lab is labs/lab-05-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 6 — Loan Amortisation Calculator

Learning objective: LO2: Apply numeric data types and arithmetic operators to solve a financial calculation problem — the monthly instalment on an amortising loan.

Goal: Mirroring the reference notebook activity, the learner implements the standard amortisation formula $P \cdot [r(1+r)^n] / [(1+r)^n - 1]$ for a Singapore housing loan, then extends it with total interest paid, the zero-interest edge case, and input validation so the tool is safe to hand to a relationship manager.

What you'll build

A script 'loan_calculator.py' that returns the monthly instalment, total repayment and total interest for a given principal, annual rate and tenor. (Tools: uv, Python 3.12, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

53. Create the project.

```
uv init lab-06-loan-calculator && cd lab-06-loan-calculator
```

54. Create `loan_calculator.py` and collect the three inputs a mortgage quote needs.

```
principal = float(input('Enter loan amount: ')); annual_rate =  
float(input('Enter annual interest rate (e.g., 5 for 5%): ')); years =  
int(input('Enter loan term in years: '))
```

55. Convert the annual rate to a monthly rate and the tenor to a number of payments — the two conversions candidates most often get wrong.

```
monthly_rate = annual_rate / 100 / 12; n = years * 12
```

56. Implement the amortisation formula using the exponent operator.

```
monthly_payment = principal * (monthly_rate * (1 + monthly_rate) ** n) /  
((1 + monthly_rate) ** n - 1)
```

57. Derive the total repayment and the total interest, then print all three formatted as SGD.

```
total_payment = monthly_payment * n; print(f'Monthly: $  
{monthly_payment:,.2f} | Total: ${total_payment:,.2f} | Interest: $  
{total_payment - principal:,.2f}')
```

58. Run it for a typical Singapore HDB loan: principal 500000, rate 2.6, term 25 years.

```
uv run python loan_calculator.py
```

59. Try a rate of 0 and observe the `ZeroDivisionError` — a real defect, because interest-free staff loans exist.

```
uv run python loan_calculator.py
```

60. PROMPT THE AI ASSISTANT with: 'Refactor `loan_calculator.py` into a function `monthly_instalment(principal, annual_rate_pct, years)` that returns monthly payment, total repayment and total interest. Handle a zero interest rate by dividing the principal evenly across the months instead of using the amortisation formula. Reject a principal of zero or less, a negative rate and a term under one year with a clear message. Format all output as SGD with thousands separators.' Apply the code.

61. REVIEW the AI output: confirm the zero-rate branch is chosen before the formula runs, that the rate is still divided by both 100 and 12, and that the three returned values are consistent.

62. Re-run the zero-rate case and a private-property case: 1200000 at 3.5% over 30 years.

```
uv run python loan_calculator.py
```

63. Compare your $500000 / 2.6\% / 25\text{-year}$ answer with a bank's published mortgage calculator and record any difference in your lab notes.

Test it

A loan of \$500,000 at 2.6% over 25 years gives a monthly instalment of approximately \$2,268.51 and total interest near \$180,554. A rate of 0% returns principal/months with no ZeroDivisionError.

Note: The runnable source for this lab is labs/lab-06-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 7 — Retirement Savings Calculator

Learning objective: LO2: Apply numeric data types, arithmetic operators and compound assignment to solve a retirement future-value calculation problem.

Goal: Mirroring the reference notebook activity, the learner computes the future value of a regular monthly savings plan using the annuity formula, then adds a CPF top-up scenario and a year-by-year projection table so a client can see the compounding effect rather than just a single number.

What you'll build

A script `retirement\_calculator.py` that projects retirement savings from a monthly contribution, expected return and years to retirement. (Tools: uv, Python 3.12, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

64. Create the project.

```
uv init lab-07-retirement && cd lab-07-retirement
```

65. Create retirement_calculator.py and collect the four planning inputs.

```
current_age = int(input('Enter your current age: ')); retirement_age =  
int(input('Enter your target retirement age: ')); monthly_savings =  
float(input('Enter amount you save monthly: ')); annual_return_rate =  
float(input('Enter expected annual return rate (e.g., 5 for 5%): '))
```

66. Derive the horizon in months and the monthly rate.

```
months = (retirement_age - current_age) * 12; monthly_rate =  
annual_return_rate / 100 / 12
```

67. Implement the future-value-of-an-annuity formula, guarding the zero-return case with a conditional.

```
total = monthly_savings * (((1 + monthly_rate) ** months - 1) /
monthly_rate) if monthly_rate > 0 else monthly_savings * months
```

68. Print the projection with thousands separators, and print the total contributed so the client can see how much is growth rather than saving.

```
print(f'Projected: ${total:,.2f} | Contributed: $
{monthly_savings*months:,.2f} | Growth: ${total -
monthly_savings*months:,.2f}')
```

69. Run a realistic Singapore case: age 30 to 65, \$1,000 a month, 5% expected return.

```
uv run python retirement_calculator.py
```

70. PROMPT THE AI ASSISTANT with: 'Extend retirement_calculator.py to also print a year-by-year table with columns Age, Contributed To Date, Balance and Growth, using a compound assignment (balance += ...) inside a loop over the months and printing one row at each year boundary. Add a CPF Special Account scenario at a fixed 4% return alongside the user rate, and print both projected balances side by side. Reject a retirement age not greater than the current age.' Apply the code.

71. REVIEW the AI output: check the loop applies the monthly return before adding the contribution consistently, that the final loop balance agrees with the closed-form formula to within a dollar, and that the validation actually stops execution.

72. Run and verify the closed-form and the iterative balance agree.

```
uv run python retirement_calculator.py
```

73. Test sensitivity: re-run at 3%, 5% and 7% and note in your lab record how much of the final balance is growth in each case.

```
uv run python retirement_calculator.py
```

74. Discuss: what does this model ignore — inflation, fees, sequence-of-returns risk — and how would you disclose that to a client?

Test it

Saving \$1,000 a month from age 30 to 65 at 5% projects approximately \$1,136,000, with roughly \$420,000 contributed and the remainder growth; the year-by-year table's final row matches the closed-form figure.

Note: The runnable source for this lab is labs/lab-07-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 8 — String Methods and Extracting the Company Name from an Email

Learning objective: LO2: Apply the string data type and its methods to parse, clean and classify financial client data.

Goal: The learner exercises the core string methods — len, upper, lower, strip, split, startswith, replace — on messy client data, then mirrors the reference notebook activity by extracting a corporate client's company name from an email address, hardening it against malformed input and free-email domains such as gmail.com.

What you'll build

A script `extract\_company.py` that parses an email address and returns the corporate client's company name, with validation. (Tools: uv, Python 3.12, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

75. Create the project.

```
uv init lab-08-string-methods && cd lab-08-string-methods
```

76. Create strings_demo.py using a messy value of the kind that arrives from a CRM export, and print its length, upper case and stripped form.

```
text = ' DBS Group Holdings Ltd. '; print(len(text), text.upper(),  
repr(text.strip()))
```

77. Split it into words and test a prefix, the way a screening rule would.

```
print(text.strip().split(' '), text.strip().startswith('DBS'))
```

78. Clean a ticker field with replace and strip so 'd05 .si ' becomes a valid uppercase ticker.

```
print(' d05 .si '.replace(' ', '').upper())
```

79. Now start extract_company.py and implement the reference activity: split the email on '@' and take the first label of the domain.

```
email = input('Enter your email address: '); parts = email.split('@')
```

80. Guard the split and extract the company label, printing it in upper case.

```
if len(parts) == 2: print(f"Extracted Company Name: {parts[1].split('.')[0].upper()}")
```

81. Run it against angch@tertiaryinfotech.com and against treasury@dbs.com.sg.

```
uv run python extract_company.py
```

82. PROMPT THE AI ASSISTANT with: 'Harden extract_company.py. Reject an input with no @ or more than one @, or an empty local part or domain, printing a clear validation message. Treat gmail.com, yahoo.com, hotmail.com and outlook.com as personal domains and print PERSONAL ACCOUNT - NOT A CORPORATE CLIENT instead of a company name. Handle multi-level domains such as dbs.com.sg by

- taking only the first label. Also add a function that processes a list of ten sample client emails and prints a table of email, company and account type.' Apply the code.
83. REVIEW the AI output: check the personal-domain list is compared in lower case, that the multi-@ case is genuinely rejected, and that the batch function does not stop at the first bad row.
84. Run the batch function and confirm the table.

```
uv run python extract_company.py
```

85. Test the edge cases: 'no-at-sign', 'a@@b.com', '@dbs.com', 'user@', and 'ANALYST@UOB.COM.SG'.

```
uv run python extract_company.py
```

Test it

treasury@dbs.com.sg returns 'DBS', analyst@uob.com.sg returns 'UOB', jane@gmail.com is flagged as a personal account, and every malformed input prints a validation message rather than a traceback.

Note: The runnable source for this lab is labs/lab-08-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 9 — Lists — Top 10 SGX Companies by Market Capitalisation

Learning objective: LO2: Apply the list data type, indexing, slicing and sorting to rank financial instruments by a business metric.

Goal: The learner exercises list creation, indexing, slicing, index() and pop() on a price series, then mirrors the reference notebook activity by fetching market capitalisation for a basket of SGX tickers with yfinance and producing a ranked top-10 list of Singapore's largest listed companies.

What you'll build

A script 'top10_sgx.py' that fetches market caps for a basket of SGX tickers and prints the top 10 ranked by market capitalisation. (Tools: uv, Python 3.12, yfinance, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

86. Create the project and add the market-data dependency with uv (not pip).

```
uv init lab-09-top10-sgx && cd lab-09-top10-sgx && uv add yfinance
```

87. Create lists_demo.py with a short closing-price series and practise indexing and slicing.

```
prices = [35.20, 35.80, 36.10, 35.95, 36.40]; print(prices[0], prices[-1],  
prices[0:3])
```

88. Find the position of a value and pop the latest observation — the two list operations used most in a rolling window.

```
print(prices.index(36.10)); print(prices.pop(), prices)
```

89. Create `top10_sgx.py` and define the SGX basket as a list of tickers.

```
sgx_tickers =  
['D05.SI', 'U11.SI', '039.SI', 'Z74.SI', 'J36.SI', 'BN4.SI', 'V03.SI', 'C38U.SI', '  
A17U.SI', 'C07.SI', 'C31.SI', 'U96.SI', 'F34.SI', 'S68.SI', 'BS6.SI']
```

90. Loop the basket, pull `marketCap` and `shortName` from `yfinance`, and append a dict per company to a results list, skipping any ticker that fails.

```
company_data.append({'name': name, 'ticker': ticker, 'market_cap':  
market_cap})
```

91. Sort descending by market cap and slice the top ten.

```
top_10 = sorted(company_data, key=lambda x: x['market_cap'], reverse=True)  
[:10]
```

92. Run it and note how long the fetch takes — API latency is a design constraint in any production screen.

```
uv run python top10_sgx.py
```

93. PROMPT THE AI ASSISTANT with: 'Improve `top10_sgx.py`. Skip any ticker whose `marketCap` is missing or zero instead of ranking it as zero. Print a numbered table with rank, company name, ticker and market cap formatted in SGD billions to two decimals, plus a final line giving the combined market cap of the top 10 and each company's percentage share of that total. Cache the fetched results to a local JSON file so a re-run does not hit the API again.' Apply the code.

94. REVIEW the AI output: confirm missing caps are filtered before the sort, that the percentage shares total 100%, and that the cache is read before the network call rather than after.

95. Run the improved script twice and confirm the second run is served from the cache.

```
uv run python top10_sgx.py
```

96. Add a deliberately invalid ticker `'ZZZZ.SI'` to the basket and re-run to prove the skip logic holds.

```
uv run python top10_sgx.py
```

Test it

The script prints a numbered top-10 table led by DBS (D05.SI) with market caps in SGD billions, the percentage shares sum to 100%, and the invalid ticker is silently skipped.

Note: The runnable source for this lab is labs/lab-09-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 10 — Tuples — Immutable Instrument Identifiers

Learning objective: LO2: Apply the tuple data type and unpacking to represent fixed financial identifiers that must not change during processing.

Goal: The learner creates tuples of SGX tickers and OHLC records, practises indexing, slicing, length and unpacking, and proves immutability by attempting a write. The lab then applies tuples where they genuinely belong in finance: as fixed instrument identifiers, as multiple return values from a ratio function, and as dictionary keys.

What you'll build

A script `tuples\_demo.py` demonstrating tuple unpacking of OHLC records and a function returning a tuple of financial ratios. (Tools: uv, Python 3.12, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

97. Create the project.

```
uv init lab-10-tuples && cd lab-10-tuples
```

98. Create tuples_demo.py with the core bank tickers as an immutable tuple, and index and slice it.

```
stock_tuple = ('D05.SI', 'U11.SI', '039.SI'); print(stock_tuple[0],  
stock_tuple[0:2], len(stock_tuple))
```

99. Unpack the tuple into three named variables in one statement.

```
ticker1, ticker2, ticker3 = stock_tuple; print(ticker1, ticker2, ticker3)
```

100. Prove immutability: uncomment the assignment and observe the TypeError, then discuss why an approved instrument list should be immutable.

```
stock_tuple[0] = 'S68.SI'
```

101. Build a list of OHLC tuples for one trading day and unpack each in a loop.

```
bars = [('D05.SI', 35.20, 36.10, 35.05, 35.90), ('U11.SI', 32.10, 32.80,  
31.95, 32.60)]
```

102. Iterate with tuple unpacking directly in the for statement.

```
for tkr, o, h, l, c in bars: print(f'{tkr}: range {h-l:.2f}, change {c-o:
+.2f}')
```

103. PROMPT THE AI ASSISTANT with: 'Add a function ratios(net_profit, revenue, equity, assets) to tuples_demo.py that returns a tuple of net profit margin, ROE and ROA as percentages. Call it for DBS with net profit 11290000000, revenue 20180000000, equity 62400000000 and assets 739000000000, unpack the returned tuple into three variables and print each to two decimals with a percent sign. Then build a dictionary keyed by the tuple (ticker, financial_year) holding those ratios for two years and print it.' Apply the code.
104. REVIEW the AI output: confirm the function returns a tuple rather than a list, that each ratio is multiplied by 100 exactly once, and that the tuple key is valid because it is immutable.
105. Run the script.

```
uv run python tuples_demo.py
```

106. Try to use a list as the dictionary key instead of the tuple and observe the TypeError — this is the practical reason tuples exist.

```
uv run python tuples_demo.py
```

Test it

The OHLC loop prints the day range and change for each bank, ratios() returns a three-element tuple giving a DBS net margin near 55.95%, ROE near 18.09% and ROA near 1.53%, and the tuple-keyed dictionary prints without error.

Note: The runnable source for this lab is labs/lab-10-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 11 — Dictionaries — Company Stock Values and Market Cap Lookup

Learning objective: LO2: Apply the dictionary data type to map financial instrument keys to values and build a fast lookup for portfolio reporting.

Goal: Mirroring the reference notebook activity, the learner builds a dictionary of stock attributes, practises access, update, membership testing, pop and iteration, then constructs a ticker-to-market-cap dictionary with a comprehension and uses .get() with a default to make the lookup safe for a missing ticker.

What you'll build

A script `stock\_dict.py` holding a ticker-to-market-cap dictionary with a safe lookup and a formatted portfolio valuation. (Tools: uv, Python 3.12, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

107. Create the project.

```
uv init lab-11-dictionaries && cd lab-11-dictionaries
```

108. Create `stock_dict.py` with a dictionary describing one holding, and read a value by key.

```
stock_info = {'ticker': 'D05.SI', 'name': 'DBS Group Holdings', 'price': 35.50}; print(stock_info['name'])
```

109. Add a new key and update an existing one — the two operations that make dictionaries the right structure for a mutable position record.

```
stock_info['currency'] = 'SGD'; stock_info['price'] = 36.20
```

110. Test membership before access, then pop a key and iterate the remaining items.

```
if 'ticker' in stock_info: print(stock_info['ticker'])
```

111. Iterate keys and values for a printed position record.

```
for key, value in stock_info.items(): print(f'{key}: {value}')
```

112. Build the ticker-to-market-cap dictionary with a dict comprehension over a list of company records.

```
company_market_values = {item['ticker']: item['market_cap'] for item in top_10_data}
```

113. Use `.get()` with a default so a delisted ticker returns a safe value instead of raising `KeyError`.

```
print(f"DBS market cap: ${company_market_values.get('D05.SI', 0):.0f}")
```

114. Compare `.get()` with direct bracket access on a missing ticker and observe the `KeyError`.

```
uv run python stock_dict.py
```

115. PROMPT THE AI ASSISTANT with: 'Extend `stock_dict.py` with a portfolio dictionary mapping ticker to number of shares for D05.SI 2000, U11.SI 1500, O39.SI 3000 and Z74.SI 10000, and a prices dictionary for the same tickers. Compute each position value and the total portfolio value, print a table of ticker, shares, price, value and weight as a percentage of the portfolio sorted by value descending, and use `.get()` with a default of 0 so a ticker missing from prices reports a value of zero with a warning line rather than crashing.' Apply the code.

116. REVIEW the AI output: confirm the weights sum to 100%, that `.get()` defaults are used on the prices lookup and not on the shares lookup, and that the sort is descending by value.

117. Delete Z74.SI from the prices dictionary and re-run to prove the warning path works.

```
uv run python stock_dict.py
```

Test it

The portfolio table prints four positions sorted by value with weights summing to 100%; removing Singtel from the prices dictionary produces a warning line and a zero-value row instead of a KeyError.

Note: The runnable source for this lab is labs/lab-11-*.py, and it is also included as a section of the combined notebook labs/Al Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 12 — Sets and Operators — Common Holdings Across Portfolios

Learning objective: LO2: Apply the set data type and arithmetic, compound, comparison, membership and logical operators to compare portfolios and encode financial rules.

Goal: Mirroring the reference notebook activities, the learner uses set intersection, union and difference to find overlapping and unique holdings across two SGX portfolios and to de-duplicate a ticker list, then works through the full operator families — arithmetic, compound assignment, comparison, membership and logical — applied to a portfolio concentration and eligibility rule.

What you'll build

A script `portfolio\_sets.py` that reports common, unique and combined holdings across two portfolios plus an operator-driven eligibility check. (Tools: uv, Python 3.12, VS Code / Cursor / Google Colab, Gemini or Copilot.)

Step-by-step

118. Create the project.

```
uv init lab-12-sets-operators && cd lab-12-sets-operators
```

119. Create portfolio_sets.py with two portfolios as sets of SGX tickers.

```
set_a = {'D05.SI', 'U11.SI', '039.SI', 'Z74.SI', 'V03.SI'}; set_b =  
{ 'Z74.SI', 'C38U.SI', 'D05.SI', 'A17U.SI', 'S68.SI' }
```

120. Find the overlap with intersection — these are the concentrated positions a risk officer cares about.

```
print(f'Common: {set_a.intersection(set_b)}')
```

121. Find the combined universe with union and the fund-specific holdings with difference.

```
print(set_a.union(set_b)); print(set_a.difference(set_b));  
print(set_b.difference(set_a))
```

122. Add a ticker, then de-duplicate a messy list of tickers by converting it through a set.

```
unique = list(set(['D05.SI', 'U11.SI', 'D05.SI', '039.SI', 'U11.SI']));  
print(unique)
```

123. Work the operators: use compound assignment to accumulate a running portfolio value, and membership to test whether a ticker is held.

```
total = 0.0; total += 2000 * 35.90; print('D05.SI' in set_a)
```

124. Combine comparison and logical operators into a real eligibility rule: a stock qualifies if ROE is above 10% AND the PE ratio is below 20, OR it is held in both portfolios.

```
eligible = (roe > 10 and pe < 20) or (ticker in set_a.intersection(set_b))
```

125. Run the script and check each operator's result against your expectation before trusting it.

```
uv run python portfolio_sets.py
```

126. PROMPT THE AI ASSISTANT with: 'Extend portfolio_sets.py so it prints a reconciliation report between the two portfolios: the count and list of common tickers, tickers only in Fund A, tickers only in Fund B, the total combined universe size, and the overlap percentage computed as the intersection size divided by the union size. Then add a screen that uses comparison, membership and logical operators to classify each ticker in the union as BUY, HOLD or REVIEW from a dictionary of ROE and PE values, treating a missing ROE or PE as REVIEW.' Apply the code.

127. REVIEW the AI output: confirm the overlap percentage uses the union and not the sum of the two sets, that the missing-data branch is checked before the comparison operators run, and that the classifications are mutually exclusive.

128. Run the reconciliation and the screen.

```
uv run python portfolio_sets.py
```

129. Test the edge cases: two identical portfolios (overlap should be 100%), two disjoint portfolios (0%), and a ticker whose PE is missing.

```
uv run python portfolio_sets.py
```

Test it

The reconciliation reports D05.SI and Z74.SI as the common tickers, an overlap of 25% across a combined universe of eight tickers, and every ticker in the union is classified as exactly one of BUY, HOLD or REVIEW with missing data landing in REVIEW.

Note: The runnable source for this lab is labs/lab-12-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Topic 03 — Problem Solving with Control Structures (14%)

If-Else · If-Elif · Ternary · While · For · Range · Enumerate · Zip · Break · Continue · Comprehensions

Key concepts

- Conditional logic encodes investment rules: a PE ratio or ROE threshold becomes a BUY / HOLD / SELL recommendation.
- While loops repeat until a condition is met — used for Fibonacci sequences underlying the golden ratio in technical analysis.
- For loops with range, enumerate and zip iterate portfolios and combine parallel series such as tickers with their ROE and ROA.
- Break and continue control early exit and skipping when scanning market data.
- List, set and dict comprehensions compute metrics such as net profit margin across a portfolio in a single readable line.

Lab 13 — If-Else PE Ratio Valuation — BUY / HOLD / SELL

Learning objective: LO2: Apply control structures to classify a stock's valuation and produce a BUY / HOLD / SELL recommendation from its PE ratio.

Goal: You encode the first investment rule of the course as Python conditional logic. A PE ratio below 15 is treated as Undervalued (Consider Buying), 15 to 25 as Fairly Valued (Hold), and above 25 as Overvalued (Consider Selling). You start from an if-else skeleton, extend it to a full if-elif-else ladder, then harden it so a negative or non-numeric PE ratio is rejected instead of silently producing a BUY signal on a loss-making company.

What you'll build

pe_valuation.py — a uv-managed script that reads a PE ratio and prints a formatted valuation and recommendation, with input validation. (Tools: uv, Python 3.12, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

130. Create and enter the uv project for this lab.

```
uv init lab-13-pe-valuation && cd lab-13-pe-valuation
```

131. Confirm uv pinned a Python 3.12 interpreter for the project.

```
uv run python --version
```

132. Create the script file that will hold your valuation rule.

```
touch pe_valuation.py
```

133. Write a simple two-branch if-else first: PE below 15 prints 'Undervalued', otherwise 'Overvalued'. Read the PE ratio with `float(input("Enter the stock's PE Ratio: "))`.

134. Run it and try PE = 12 then PE = 30 to see both branches fire.

```
uv run python pe_valuation.py
```

135. Extend the two-branch version into a three-branch if / elif / else ladder: PE < 15 gives Undervalued + 'Consider Buying'; 15 <= PE <= 25 gives Fairly Valued + 'Hold'; else Overvalued + 'Consider Selling'.

136. Format the output with f-strings so it reads as an analyst note: print the PE ratio to 2 decimals, then the valuation, then the recommendation on separate lines.

```
uv run python pe_valuation.py
```

137. AI-ASSIST: in Colab's Gemini panel (or Cursor / Copilot) prompt: "Rewrite this PE ratio valuation script so it validates the input — reject a negative PE ratio with a clear message because a negative PE means the company is loss-making, and reject non-numeric input using try/except ValueError. Keep the if-elif-else structure." Read every line of the generated code before you accept it.

138. Paste the AI-generated version into `pe_valuation.py`, then test the edge cases it claims to handle: -5, the string 'abc', and exactly 15 and exactly 25 (the boundary values).

```
uv run python pe_valuation.py
```

139. Discuss: the boundary PE = 25 falls in 'Hold' because the condition uses <=. Change it to < and re-run with 25 to see the recommendation flip — a one-character change that alters a trading signal.

140. Add a short comment block at the top of the script naming the source of the 15 and 25 thresholds, so a reviewer knows the rule is a documented policy and not a magic number.

Test it

Run `uv run python pe_valuation.py` five times with PE = 12, 15, 20, 25 and 30. You should see Undervalued/Buy, Fairly Valued/Hold, Fairly Valued/Hold, Fairly Valued/Hold and Overvalued/Sell. Entering -5 or 'abc' must print a validation message instead of a recommendation.

Note: The runnable source for this lab is `labs/lab-13-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 14 — Activity: Return on Equity (ROE) Screener with a BUY / SELL Threshold

Learning objective: LO2: Apply conditional logic to a real SGX fundamental metric, computing ROE from live financial statements and converting it into an investment decision.

Goal: You pull DBS Group Holdings (D05.SI) financials with yfinance, compute Return on Equity as Net Income divided by AVERAGE shareholders' equity, and apply a 15% threshold to generate a BUY or SELL signal. Using average equity (latest and prior year) rather than closing equity is the analyst convention, because net income is earned across the whole year. You also meet your first ZeroDivisionError risk: a company with zero or missing equity.

What you'll build

roe_screener.py — computes ROE for D05.SI from live yfinance statements and prints a threshold-based BUY/SELL recommendation. (Tools: uv, yfinance, pandas, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

141. Create the uv project and add the market-data dependencies.

```
uv init lab-14-roe-screener && cd lab-14-roe-screener && uv add yfinance pandas
```

142. Check that uv recorded yfinance and pandas in pyproject.toml.

```
uv run python -c "import yfinance, pandas; print(yfinance.__version__, pandas.__version__)"
```

143. Create roe_screener.py and fetch DBS: build a yf.Ticker('D05.SI') object, then read stock.financials into income_stmt and stock.balance_sheet into balance_sheet.

144. Inspect the raw statements once so you know the row labels you are about to index — print income_stmt.index and balance_sheet.index.

```
uv run python roe_screener.py
```

145. Extract Net Income with income_stmt.loc['Net Income'].iloc[0] (iloc[0] is the most recent year), and extract Stockholders Equity for both .iloc[0] and .iloc[1].

146. Compute avg_equity = (equity_latest + equity_prev) / 2, then roe = (net_income / avg_equity) * 100.

147. Print a formatted report: company name and ticker, Net Income with thousands separators (\${:,0f}), Average Shareholders Equity, and ROE to 2 decimals with a percent sign.

```
uv run python roe_screener.py
```

148. Add the decision rule with if-else: ROE above 15% prints 'STRONG — Consider BUY', 10% to 15% prints 'MODERATE — HOLD', below 10% prints 'WEAK — Consider SELL'.

```
uv run python roe_screener.py
```

149. AI-ASSIST: prompt your AI tool with: "This script divides net income by average shareholders equity. Add guards so it fails safely: raise or report a clear message if average equity is zero (ZeroDivisionError), if the 'Net Income' or 'Stockholders Equity' row is missing from the statement (KeyError), or if only one year of balance sheet data exists (IndexError). Do not swallow the error silently." Review the generated guards.

150. Apply the guarded version and deliberately break it: change the ticker to a nonsense symbol such as 'ZZZZ.SI' and confirm the script reports a clear message rather than crashing with a traceback.

```
uv run python roe_screener.py
```

151. Wrap the whole calculation in a loop over three banks — D05.SI, U11.SI, O39.SI — so one run screens all three and prints one recommendation line each.

```
uv run python roe_screener.py
```

152. Discuss: why does using average equity instead of year-end equity matter most for a company that raised a lot of capital mid-year?

Test it

Run `uv run python roe_screener.py`. You should see a Net Income figure, an Average Shareholders Equity figure and an ROE percentage for each of D05.SI, U11.SI and O39.SI, each followed by a BUY / HOLD / SELL line. Substituting an invalid ticker must produce a readable message, never an unhandled traceback.

Note: The runnable source for this lab is `labs/lab-14-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 15 — If-Elif Income Grouping and Credit Risk Banding

Learning objective: LO2: Apply a multi-branch if-elif-else ladder to classify bank customers into income bands and credit-risk categories.

Goal: Retail banking runs on segmentation. You build a classifier that maps annual income into Low (below \$30,000), Middle (up to \$80,000), High (up to \$150,000) and Super High Income bands, then a second classifier that maps a 300-850 credit score into Excellent / Good / Fair / Poor with the bank's stated lending posture for each. You will see why the ORDER of elif branches is load-bearing: reverse two of them and customers land in the wrong band.

What you'll build

income_grouping.py — classifies annual income into four bands and a credit score into four risk categories, with range validation. (Tools: uv, Python 3.12, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

153. Create the uv project for the segmentation lab.

```
uv init lab-15-income-grouping && cd lab-15-income-grouping
```

154. Create income_grouping.py and read the customer's annual income with `float(input('Enter annual income: '))`.

155. Write the if-elif ladder: below 30000 is 'Low Income'; up to 80000 is 'Middle Income'; up to 150000 is 'High Income'; else 'Super High Income'.

156. Print the result with formatting: income as `${:,.2f}` and the classification on its own line.

```
uv run python income_grouping.py
```

157. Test the band boundaries deliberately: 29999.99, 30000, 80000, 150000 and 150000.01. Note which side of each boundary each value lands on.

```
uv run python income_grouping.py
```

158. BREAK IT ON PURPOSE: move the 'income <= 150000' branch above the 'income <= 80000' branch, re-run with 50000, and observe it now misclassifies as High Income. Restore the correct order and write one sentence explaining why elif order matters.

159. Add the second classifier in the same script: read a credit score with `int(input(...))` and band it — 800+ Excellent ('High probability of loan approval at best rates'), 700+ Good ('Likely approved with standard rates'), 600+ Fair ('May be approved with higher interest rates'), else Poor ('High risk; loan approval unlikely').

```
uv run python income_grouping.py
```

160. AI-ASSIST: prompt your AI tool with: "Add input validation to this banking classifier — reject a negative annual income, and reject any credit score outside the valid 300 to 850 range with a specific message naming the valid range. Use try/except to handle non-numeric input. Then refactor both classifiers into a single reusable function each." Review the generated functions before running them.

161. Run the AI-refactored script and test the invalid paths: income of -1000, credit score of 250, credit score of 900, and the text 'seven hundred'.

```
uv run python income_grouping.py
```

162. Add a combined rule at the end: a customer in the High or Super High income band AND with a credit score of 700 or above is flagged 'PRIORITY WEALTH SEGMENT'. Use the logical `and` operator inside a single if.

```
uv run python income_grouping.py
```

163. Discuss: the bands here are hard-coded. What is the operational risk when the bank's segmentation policy changes and this rule is duplicated across five scripts?

Test it

Run `uv run python income\_grouping.py` and enter income 25000 / 55000 / 120000 / 250000 — you should get Low, Middle, High and Super High Income respectively. A credit score of 810 must return Excellent; 250 and 900 must both be rejected with a range message. Income 160000 with score 750 must print PRIORITY WEALTH SEGMENT.

Note: The runnable source for this lab is labs/lab-15-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 16 — Ternary Operator — One-Line Trading Decisions

Learning objective: LO2: Apply the ternary conditional expression to write compact, readable one-line financial decision rules.

Goal: The ternary operator collapses a four-line if-else into one expression: `value_if_true` if condition else `value_if_false`. You use it for a price-versus-target buy/wait decision, a margin-call check, a dividend-yield flag and a safe division that returns 0 instead of raising `ZeroDivisionError`. You then meet its limit — a nested ternary that is technically correct but unreadable — and learn when to go back to if-elif.

What you'll build

`ternary_decisions.py` — a set of one-line ternary rules covering buy/wait, margin call, yield flagging and zero-safe division. (Tools: uv, Python 3.12, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

164. Create the uv project.

```
uv init lab-16-ternary-decisions && cd lab-16-ternary-decisions
```

165. Create `ternary_decisions.py`. Set `current_price = 145.50` and `target_price = 150.00`, then write `decision = 'Buy' if current_price < target_price else 'Wait'`.

166. Print the current price, target price and decision, all formatted to 2 decimals.

```
uv run python ternary_decisions.py
```

167. Write the same rule the long way as a four-line if-else, print both results, and confirm they agree. Keep both in the file as a side-by-side comparison.

```
uv run python ternary_decisions.py
```

168. Add a margin-call ternary: given `account_equity = 18000` and `maintenance_margin = 20000`, set `status = 'MARGIN CALL'` if `account_equity < maintenance_margin` else `'OK'`.

169. Add a zero-safe ternary — the pattern you will reuse all course: `margin = (net_income / revenue) * 100` if `revenue != 0` else `0`. Test it with `revenue = 0` and confirm no `ZeroDivisionError`.

```
uv run python ternary_decisions.py
```

170. Use a ternary inside an f-string to flag high-yield stocks: for a dividend yield of 5.8%, print a line ending in `'HIGH YIELD'` if the yield exceeds 4 else `'STANDARD'`.

```
uv run python ternary_decisions.py
```

171. AI-ASSIST: prompt your AI tool with: "Convert this PE-ratio if-elif-else ladder with three branches (Undervalued / Fairly Valued / Overvalued) into a nested ternary expression, then tell me honestly whether the nested version is more readable and when I should NOT use a ternary." Read the answer critically.

172. Paste the nested ternary in, run it against PE values of 12, 20 and 30, and confirm it matches Lab 13's output. Then write a one-line comment recording your own judgement on readability.

```
uv run python ternary_decisions.py
```

173. Apply the ternary across a small portfolio: loop over the list `[('D05.SI', 35.50), ('U11.SI', 28.40), ('O39.SI', 13.10)]` with a target of 30.00 and print a Buy/Wait decision per ticker in an aligned table.

```
uv run python ternary_decisions.py
```

174. Discuss: a ternary always evaluates to a value, so it can sit inside a list comprehension or a function argument — where an if statement cannot. Name one place in your own code where that matters.

Test it

Run `uv run python ternary_decisions.py`. The buy/wait decision must be `'Buy'` (`145.50 < 150.00`), the margin status must be `'MARGIN CALL'`, the zero-revenue margin must print `0` with no traceback, and the portfolio table must show one Buy/Wait decision per ticker.

Note: The runnable source for this lab is `labs/lab-16-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 17 — While Loop — Fibonacci Sequence and the Golden Ratio in Technical Analysis

Learning objective: LO2: Apply a while loop to generate a sequence that repeats until a condition is met, and use it to derive the Fibonacci retracement levels used in technical analysis.

Goal: Fibonacci retracement is one of the most widely used tools in technical analysis, and it rests on the golden ratio 1.618034 that successive Fibonacci numbers converge to. You generate the sequence with a while loop and multiple assignment ($a, b = b, a + b$), watch the ratio b/a converge, then apply the derived levels — 23.6%, 38.2%, 50%, 61.8% and 78.6% — to a real price swing to produce actual support and resistance prices. You also add the guard every while loop needs: a maximum iteration count so a bad condition cannot loop forever.

What you'll build

fibonacci_ta.py — generates the Fibonacci sequence with a while loop, shows golden-ratio convergence, and prints Fibonacci retracement price levels for a stock's swing high and low. (Tools: uv, Python 3.12, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

175. Create the uv project.

```
uv init lab-17-fibonacci-ta && cd lab-17-fibonacci-ta
```

176. Warm up with a savings while loop: goal 5000, monthly deposit 450, balance starting at 0. Loop while balance < goal, adding the deposit and counting months, printing each month's balance. Confirm it terminates.

```
uv run python fibonacci_ta.py
```

177. Now build the Fibonacci generator: read `n_terms` with `int(input('How many terms? '))`, set `a, b = 0, 1` and `count = 0`, and loop while `count < n_terms` appending `a` to a list.

178. Inside the loop, use the multiple assignment `a, b = b, a + b` to advance the sequence, and increment count. Print the finished sequence for 15 terms.

```
uv run python fibonacci_ta.py
```

179. Add ratio tracking: inside the loop, when `a` is not zero, append `b / a` to a ratios list. Print the last five ratios to 6 decimal places alongside the target golden ratio ~1.618034 and watch them converge.

```
uv run python fibonacci_ta.py
```

180. Explain in a comment why the ``if a != 0`` guard is required — the very first term would otherwise raise `ZeroDivisionError`.

181. Apply it to markets: set `swing_high = 42.80` and `swing_low = 31.20` for an SGX bank. Compute the price range, then loop over the retracement levels [0.236, 0.382, 0.5, 0.618, 0.786] and print `swing_high - (range * level)` for each, formatted as `${:.2f}`.

```
uv run python fibonacci_ta.py
```

182. Note in a comment that 0.618 is $1/1.618034$ and 0.382 is 0.618 squared — the retracement grid is derived from the ratio your while loop just converged to.

183. AI-ASSIST: prompt your AI tool with: "Add a safety guard to this while loop so it can never run forever — cap it at a maximum of 500 iterations and print a warning if the cap is hit. Also validate that `n_terms` is a positive integer, and explain what happens today if the user enters 0 or a negative number." Review and apply the guard.

184. Test the guard: enter 0 terms, then -5 terms, then 500 terms, and confirm the script handles all three without hanging or crashing.

```
uv run python fibonacci_ta.py
```

185. Add a decision line: given a current price of 35.90, use an if-elif ladder to report which two retracement levels the price currently sits between.

```
uv run python fibonacci_ta.py
```

186. Discuss: a while loop is the right choice here because you do not know in advance how many terms are needed to converge. Name one financial task where a for loop would be the better choice instead.

Test it

Run `uv run python fibonacci_ta.py` and enter 15 terms. The sequence must begin 0, 1, 1, 2, 3, 5, 8, 13 and the last printed ratios must be within 0.0001 of 1.618034. The retracement block must print five price levels between \$31.20 and \$42.80, with the 61.8% level at approximately \$35.63. Entering 0 or -5 must be rejected without hanging.

Note: The runnable source for this lab is `labs/lab-17-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 18 — For Loop and Range — Top 10 SGX Return on Assets (ROA) Ranking

Learning objective: LO2: Apply for loops and the range function to iterate a portfolio of tickers, compute Return on Assets for each, and rank the results.

Goal: You iterate a list of ten major SGX tickers — DBS, UOB, OCBC, SingTel, Jardine, Keppel, Venture, CapitaLand Integrated, Ascendas REIT and City Developments — fetching Net Income and Total Assets for each and computing $ROA = \text{Net Income} / \text{Total Assets} * 100$. You then sort the results and print a ranked league table. Along the way

you use range for a compound-growth projection and learn why a per-ticker try/except keeps a ten-stock scan alive when one ticker's data is missing.

What you'll build

roa_ranking.py — fetches fundamentals for 10 SGX tickers in a for loop and prints a sorted Top 10 ROA league table. (Tools: uv, yfinance, pandas, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

187. Create the uv project and add market-data dependencies.

```
uv init lab-18-roa-ranking && cd lab-18-roa-ranking && uv add yfinance pandas
```

188. Warm up with range: in roa_ranking.py project a \$1,000 principal at 5% for 10 years using `for year in range(1, years + 1)` and the formula `principal * (1 + rate) ** year`, printing each year-end balance.

```
uv run python roa_ranking.py
```

189. Define the ticker list: `sgx_tickers = ['D05.SI', 'U11.SI', 'O39.SI', 'Z74.SI', 'J36.SI', 'BN4.SI', 'V03.SI', 'C38U.SI', 'A17U.SI', 'C07.SI']`.

190. Write the main for loop: for each ticker build a `yf.Ticker` object, read `stock.financials` and `stock.balance_sheet`, extract Net Income and Total Assets with `.loc[...].iloc[0]`, compute ROA, and append a dict `{'name', 'ticker', 'roa'}` to a results list.

191. Wrap the body of the loop in try/except so a single failing ticker prints a skip message and the loop continues to the next one instead of aborting the whole scan.

```
uv run python roa_ranking.py
```

192. Pull a readable company name with `stock.info.get('shortName', ticker)` — the `.get` default means a missing name falls back to the ticker rather than raising `KeyError`.

193. Sort the results descending by ROA using `sorted(results, key=lambda x: x['roa'], reverse=True)`.

194. Print a ranked table with aligned columns: use `enumerate(..., 1)` for the rank number and f-string width specifiers such as `{name:<28} {ticker:<10} {roa:>8.2f}`, under a header row and a separator line.

```
uv run python roa_ranking.py
```

195. AI-ASSIST: prompt your AI tool with: "This loop makes one network call per ticker and re-fetches on every run. Add a simple check that skips a ticker whose Total Assets is zero or missing to avoid `ZeroDivisionError`, count how many tickers succeeded versus were skipped, and print that summary at the end. Do not add any external caching library." Review the changes before applying.

196. Apply the AI version and deliberately insert an invalid ticker such as 'XXXX.SI' in the middle of the list. Confirm the scan completes all ten, reports the skip, and the summary counts are correct.

```
uv run python roa_ranking.py
```

197. Add a range-based slice of the league table: use `for i in range(3)` to print only the top three performers under a 'BEST IN CLASS' heading.

```
uv run python roa_ranking.py
```

198. Discuss: ROA and ROE tell different stories for a bank, which is highly leveraged. Which of the two is more comparable across a bank and a REIT, and why?

Test it

Run `uv run python roa\_ranking.py`. You should see the 10-year compound projection, then a ranked ROA table with up to 10 rows sorted highest ROA first, a BEST IN CLASS top-three block, and a final summary line counting successes and skips. Inserting 'XXXX.SI' must not stop the scan.

Note: The runnable source for this lab is labs/lab-18-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 19 — Enumerate, Zip, Break and Continue — Combined ROE and ROA Scanner

Learning objective: LO2: Apply enumerate and zip to combine parallel financial series, and break and continue to control early exit and skipping while scanning market data.

Goal: Real screening code walks several parallel lists at once — tickers, ROE values and ROA values — and must decide row by row whether to include, skip or stop. You use zip to fuse the three series into one table, enumerate to rank it, continue to skip tickers with missing data or negative returns, and break to stop a budget-constrained buying loop the moment the cash limit would be breached. You also see the classic zip trap: zip stops silently at the SHORTEST list, so a mismatched length loses data without any error.

What you'll build

roe_roa_scanner.py — a combined ROE + ROA screening table built with zip and enumerate, with continue-based skipping and a break-controlled budget allocator. (Tools: uv, yfinance, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

199. Create the uv project and add yfinance.

```
uv init lab-19-roe-roa-scanner && cd lab-19-roe-roa-scanner && uv add yfinance
```

200. In `roe_roa_scanner.py`, loop over `sgx_tickers = ['D05.SI', 'U11.SI', 'O39.SI', 'Z74.SI', 'V03.SI']`, read `stock.info`, and append to three parallel lists: `tickers`, `roes` (`info.get('returnOnEquity', 0) * 100`) and `roas` (`info.get('returnOnAssets', 0) * 100`).

201. Fuse the three lists with `data = zip(tickers, roes, roas)` and print a header row plus one aligned line per stock using `{t:<10} {roe:<10.2f} {roa:<10.2f}`.

```
uv run python roe_roa_scanner.py
```

202. SEE THE ZIP TRAP: temporarily append one extra ticker to `'tickers'` only, re-run, and confirm the extra row disappears with no error. Restore the lists and add a comment recording the lesson.

```
uv run python roe_roa_scanner.py
```

203. Add `enumerate` to rank the table: `'for rank, (t, roe, roa) in enumerate(zip(tickers, roes, roas), 1)'` and print the rank in the first column. Note that `start=1` gives human-readable ranks.

```
uv run python roe_roa_scanner.py
```

204. Use `continue` to skip rows with no usable data: inside the loop, if `roe == 0` or `roa == 0`, print a `'no data — skipped'` line and `continue` to the next ticker.

```
uv run python roe_roa_scanner.py
```

205. Add a quality flag column computed with a ternary: `'QUALITY'` when `ROE > 10` and `ROA > 1`, otherwise `'REVIEW'`.

```
uv run python roe_roa_scanner.py
```

206. Build the break demo — a budget allocator: with `budget = 100000` and a list of intended trade sizes `[20000, 35000, 30000, 25000, 40000]`, accumulate them in a `for` loop and break the moment the next trade would exceed the budget, printing which trade was refused.

```
uv run python roe_roa_scanner.py
```

207. Add a second `continue` demo on a transaction feed: given `[120.50, -50.00, 300.25, -20.00, 150.00, 0, 45.75]`, skip every non-positive amount (refunds and zeros) and total only genuine expenses.

```
uv run python roe_roa_scanner.py
```

208. AI-ASSIST: prompt your AI tool with: "Rewrite this scanner so the three parallel lists become a single list of dictionaries, and explain why that is safer than zipping three separate lists that must stay the same length. Keep the `enumerate` ranking, the

continue-based skipping and the identical printed output." Compare the two designs side by side.

209. Apply the AI refactor into a second file, run both, and confirm the printed tables are byte-for-byte identical.

```
uv run python roe_roa_scanner_v2.py
```

210. Discuss: break exits the loop entirely while continue only skips the current iteration. In a compliance scan of 5,000 transactions, which one would you use to skip malformed records, and which to abort on a hard stop rule?

Test it

Run `uv run python roe_roa_scanner.py`. The scanner must print a ranked table with Rank, Ticker, ROE %, ROA % and a QUALITY/REVIEW flag for each stock that returned data, skip lines for any that did not, a budget allocator that stops before breaching \$100,000, and a total-valid-expenses figure of \$616.50 from the transaction feed.

Note: The runnable source for this lab is `labs/lab-19-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 20 — Comprehensions — Net Profit Margin for 10 Singapore Stocks in One Line

Learning objective: LO2: Apply list, set and dict comprehensions to compute and filter financial metrics across a portfolio in a single readable expression.

Goal: Comprehensions are the idiom that makes Python financial code short and reviewable. You build up from simple currency conversion and GST examples to the topic's headline activity: net profit margin ($\text{Net Income} / \text{Total Revenue} * 100$) computed across ten SGX tickers with a single list comprehension, plus a set comprehension that de-duplicates tickers from a transaction history and a dict comprehension that builds a ticker-to-margin lookup and filters it to the high-margin names. You finish by comparing the comprehension against the equivalent for loop.

What you'll build

`margin_comprehensions.py` — computes net profit margin for 10 SGX stocks with a list comprehension, plus set and dict comprehension utilities and a filtered high-margin lookup. (Tools: uv, yfinance, pandas, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

211. Create the uv project and add the dependencies.

```
uv init lab-20-margin-comprehensions && cd lab-20-margin-comprehensions &&
uv add yfinance pandas
```

212. Warm up with three list comprehensions in `margin_comprehensions.py`: convert `prices_sgd = [35.50, 28.40, 13.10, 5.20]` to USD at 0.74; apply 9% GST to subtotals `[100, 250, 45]` with `round(amt * 1.09, 2)`; and select tickers priced above \$20 using `[t for t, p in zip(tickers, prices) if p > 20]`.

```
uv run python margin_comprehensions.py
```

213. Add a set comprehension over a transaction history list of dicts to extract the unique tickers: `{t['ticker'] for t in transactions}`. Print the set and its length to prove duplicates were removed.

```
uv run python margin_comprehensions.py
```

214. Add two dict comprehensions: build a market-cap lookup from parallel lists with `{t: c for t, c in zip(tickers, market_caps)}`, then filter an existing price dict to only stocks above \$20 with `{t: p for t, p in prices.items() if p > 20}`.

```
uv run python margin_comprehensions.py
```

215. Now the main activity. Define a helper function `get_margin(ticker)` that fetches `stock.financials`, reads Net Income and Total Revenue with `.loc[...].iloc[0]`, and returns `{'ticker': ticker, 'margin': (net_income / revenue) * 100}`, returning a margin of `None` inside an `except` block.

216. Define the ten tickers `['D05.SI', 'U11.SI', 'O39.SI', 'Z74.SI', 'J36.SI', 'BN4.SI', 'V03.SI', 'C38U.SI', 'A17U.SI', 'C07.SI']` and compute every margin in ONE line: `results = [get_margin(t) for t in sgx_tickers]`.

217. Print the results as an aligned table with a header, handling the `None` case in the f-string so a failed fetch shows 'N/A' rather than crashing on format.

```
uv run python margin_comprehensions.py
```

218. Add a filtering comprehension on top of the results: `high_margin = [r for r in results if r['margin'] is not None and r['margin'] > 20]` and print how many of the ten cleared the 20% bar.

```
uv run python margin_comprehensions.py
```

219. Build the dict lookup with a dict comprehension: `margin_lookup = {r['ticker']: round(r['margin'], 2) for r in results if r['margin'] is not None}`, then query a single ticker from it.

```
uv run python margin_comprehensions.py
```

220. AI-ASSIST: prompt your AI tool with: "Rewrite this single-line list comprehension as an explicit for loop with the same behaviour, then tell me which version you would approve in a code review for a financial reporting system and why. Also point out any

case where the comprehension silently hides an error that the for loop would surface." Read the reasoning carefully.

221. Add the AI's for-loop version alongside the comprehension, run both, and assert they produce identical results with a simple equality check printed as PASS or FAIL.

```
uv run python margin_comprehensions.py
```

222. Discuss: the bare `except:` inside `get_margin` catches everything, including a typo in a row label. Replace it with `except (KeyError, IndexError, ZeroDivisionError)` and re-run — does anything now fail loudly that was silently returning `None` before?

```
uv run python margin_comprehensions.py
```

Test it

Run `uv run python margin_comprehensions.py`. It must print the USD prices, GST totals and >\$20 ticker list; a unique-ticker set of size 3; a market-cap dict and a filtered high-value price dict; a 10-row net profit margin table where any failed fetch shows N/A; a count of stocks above a 20% margin; and a final PASS line confirming the comprehension and the for-loop versions agree.

Note: The runnable source for this lab is `labs/lab-20*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Topic 04 — Scripting with Function and Lambda (14%)

Functions · Return values · Argument styles · Lambda · Map · Filter

Key concepts

- Functions are reusable blocks that turn a business requirement into a named, testable unit.
- A function can return multiple values, letting one call deliver a full set of financial ratios.
- Multiple, default, named and variable arguments make a pricing or ratio function flexible without duplication.
- Lambda creates small anonymous functions for one-off financial formulas.
- Map applies a lambda across a sequence of stocks; filter selects the subset that meets a business threshold such as high-income customers.

Lab 21 — Functions — A Reusable Financial Formula Library

Learning objective: LO3: Construct reusable functions that turn a business requirement into a named, testable unit computing standard financial metrics.

Goal: You build your first reusable module: three named functions covering Compound Annual Growth Rate, simple interest and break-even units. Each gets a docstring stating the formula, because in a regulated environment the formula must be auditable from the code. You then see the difference between a function that returns a value and one that only prints, and why the returning version is the only one you can test.

What you'll build

finance_formulas.py — a module of documented, reusable financial functions (CAGR, simple interest, break-even units) with a demonstration block. (Tools: uv, Python 3.12, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

223. Create the uv project for your formula library.

```
uv init lab-21-finance-formulas && cd lab-21-finance-formulas
```

224. In finance_formulas.py define calculate_cagr(beginning_value, ending_value, years) returning $((\text{ending_value} / \text{beginning_value})^{1 / \text{years}}) - 1$, with a one-line docstring naming the metric.

225. Define calculate_simple_interest(principal, rate, time) returning $\text{principal} * (\text{rate} / 100) * \text{time}$ — note the rate is passed as a whole number of percent, and say so in the docstring.

226. Define break_even_units(fixed_costs, price_per_unit, variable_cost_per_unit) returning $\text{fixed_costs} / (\text{price_per_unit} - \text{variable_cost_per_unit})$.

227. Call all three with the worked examples — CAGR from 10000 to 15000 over 5 years, simple interest on 5000 at 4.5% for 2 years, break-even on 20000 fixed costs at a \$50 price and \$30 variable cost — and print each result with the right format specifier (`{:.2%}` for CAGR, `${:.2f}` for interest, whole units for break-even).

```
uv run python finance_formulas.py
```

228. CONTRAST: write a fourth function that prints the CAGR instead of returning it. Try to use its output in a further calculation and observe that you get None. Write a comment recording why 'return' beats 'print' in a library.

```
uv run python finance_formulas.py
```

229. AI-ASSIST: prompt your AI tool with: "Add input validation to these three financial functions. calculate_cagr must reject a zero or negative beginning value and zero years; break_even_units must reject a contribution margin of zero or below because that means the product never breaks even (ZeroDivisionError). Raise ValueError with a message naming the offending argument." Review the exceptions it chose.

230. Apply the validated version and test each failure path: CAGR with beginning_value = 0, CAGR with years = 0, and break-even where the variable cost exceeds the price.

```
uv run python finance_formulas.py
```

231. Add a `if __name__ == '__main__':` guard around the demonstration block so the module can be imported by later labs without running the demo.

```
uv run python finance_formulas.py
```

232. Prove the guard works by importing the module and calling one function from a one-liner.

```
uv run python -c "from finance_formulas import calculate_cagr;
print(f'{calculate_cagr(10000, 15000, 5):.2%}')
```

233. Discuss: what does the CAGR figure hide about the year-by-year path an investment took, and why does that matter to a client conversation?

Test it

Run `uv run python finance_formulas.py`. CAGR must print 8.45%, simple interest \$450.00, and break-even 1000 units. The import one-liner must print 8.45% with no demonstration output, proving the `__main__` guard works. All three invalid-input tests must raise a `ValueError` naming the bad argument.

Note: The runnable source for this lab is `labs/lab-21-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 22 — Return Multiple Values — A Full Income Statement Analysis in One Call

Learning objective: LO3: Construct a function that returns multiple values so a single call delivers a complete set of financial ratios, and unpack them cleanly.

Goal: A single call should be able to hand back a whole analysis. You build `analyze_income_statement(revenue, cogs, operating_expenses)` which returns gross profit, net income and profit margin as a tuple, then unpack all three in one assignment. You compare tuple return against dictionary return, learn why the tuple's ORDER is a silent trap when a caller unpacks it wrongly, and finish with a named-tuple version that removes the trap.

What you'll build

`income_analysis.py` — an income statement analyser returning gross profit, net income and margin, in tuple, dict and namedtuple variants. (Tools: uv, Python 3.12, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

234. Create the uv project.

```
uv init lab-22-income-analysis && cd lab-22-income-analysis
```

235. In `income_analysis.py` define `analyze_income_statement(revenue, cogs, operating_expenses)`. Compute `gross_profit = revenue - cogs`, `net_income = gross_profit - operating_expenses`, and `profit_margin = (net_income / revenue) * 100` if `revenue != 0` else 0.

236. Return all three as a tuple: ``return gross_profit, net_income, profit_margin``. Note in the docstring that this is a tuple return even though there are no parentheses.

237. Call it with revenue 500000, COGS 200000 and operating expenses 150000, unpacking with ``gp, ni, pm = analyze_income_statement(...)``.

238. Print a formatted mini income statement: Revenue, Gross Profit and Net Income with thousands separators, and Profit Margin to 2 decimals with a percent sign.

```
uv run python income_analysis.py
```

239. SEE THE TRAP: deliberately unpack in the wrong order as ``pm, ni, gp = ...`` and re-run. Nothing errors, but the report is nonsense. Restore the correct order and comment on the risk.

```
uv run python income_analysis.py
```

240. Confirm the zero-revenue guard: call the function with `revenue = 0` and check the margin comes back as 0 rather than raising `ZeroDivisionError`.

```
uv run python income_analysis.py
```

241. AI-ASSIST: prompt your AI tool with: "Rewrite this function to return a dictionary with keys `gross_profit`, `net_income` and `profit_margin` instead of a positional tuple, and then a third version using `collections.namedtuple`. Explain which of the three is safest for a financial reporting system where callers are written by different teams." Read the trade-off argument.

242. Add both AI variants to the file as `analyze_income_statement_dict` and `analyze_income_statement_nt`, and call all three on the same inputs to confirm identical numbers.

```
uv run python income_analysis.py
```

243. Extend the analysis: add operating margin $((\text{gross_profit} - \text{operating_expenses}) / \text{revenue} * 100)$ and a cost ratio $(\text{cogs} / \text{revenue} * 100)$ to the dict version, so one call now returns five metrics.

```
uv run python income_analysis.py
```

244. Run the analyser over three mock companies held in a list of tuples, printing one comparison row per company in an aligned table.

```
uv run python income_analysis.py
```

245. Discuss: the tuple version is faster to write and the dict version is safer to consume. At what team size does that trade-off flip?

Test it

Run `uv run python income_analysis.py`. For revenue 500000 / COGS 200000 / OpEx 150000 it must report Gross Profit \$300,000, Net Income \$150,000 and Profit Margin 30.00%. The dict and namedtuple versions must return the same three numbers, revenue = 0 must return a margin of 0 without a traceback, and the three-company comparison table must print three aligned rows.

Note: The runnable source for this lab is `labs/lab-22-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 23 — Activity: Net Profit Margin Function with Zero-Revenue Handling

Learning objective: LO3: Construct a single-responsibility function computing net profit margin, and defend it against the zero and missing data cases found in real filings.

Goal: This is the topic's core activity, built to production standard.

`calculate_net_profit_margin(net_income, revenue)` returns $(\text{net_income} / \text{revenue}) * 100$, guarded so a revenue of zero returns 0 rather than raising `ZeroDivisionError`. You then apply it to live SGX filings, handle the negative-margin case (a loss-making company is not a bug), and write a small set of assertions that prove the function behaves — your first taste of testing.

What you'll build

`net_profit_margin.py` — a validated net profit margin function with assertion-based tests, applied to live SGX financial statements. (Tools: `uv`, `yfinance`, `pandas`, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

246. Create the `uv` project and add market-data dependencies.

```
uv init lab-23-net-profit-margin && cd lab-23-net-profit-margin && uv add yfinance pandas
```

247. In `net_profit_margin.py` define `calculate_net_profit_margin(net_income, revenue)` with a docstring stating the formula $(\text{Net Income} / \text{Total Revenue}) * 100$. Return 0 if revenue is 0, otherwise the computed margin.

248. Test it with the sample figures from the reference: net income 1,000,000 on revenue 5,000,000. Print the result to 2 decimals with a percent sign.

```
uv run python net_profit_margin.py
```

249. Test the loss case: net income of -250,000 on revenue 5,000,000 should return -5.00%. Confirm the function does NOT suppress the negative — a loss must stay visible.

```
uv run python net_profit_margin.py
```

250. Test the guard: call it with revenue = 0 and confirm it returns 0 with no traceback. Add a comment on why returning 0 here is a design choice a reviewer might challenge.

```
uv run python net_profit_margin.py
```

251. Wire it to live data: fetch D05.SI with yfinance, read Net Income and Total Revenue from stock.financials with .loc[...].iloc[0], and pass both into your function.

```
uv run python net_profit_margin.py
```

252. Loop the live version over five SGX tickers ['D05.SI', 'U11.SI', 'O39.SI', 'Z74.SI', 'V03.SI'], printing a formatted margin table with a header row and a separator.

```
uv run python net_profit_margin.py
```

253. AI-ASSIST: prompt your AI tool with: "Write five assert statements that test this net profit margin function: the normal case, a zero-revenue case, a negative net income case, a 100% margin case, and a case where revenue is negative. Then tell me whether returning 0 for zero revenue is better than raising an exception in a financial reporting context, and argue both sides." Review the tests and the argument.

254. Add the generated assertions to the bottom of the file under a run_tests() function and execute them — all five must pass silently.

```
uv run python net_profit_margin.py
```

255. Add a classification wrapper on top: a margin above 20% is 'EXCELLENT', 10 to 20% is 'HEALTHY', 0 to 10% is 'THIN', below 0 is 'LOSS-MAKING'. Apply it to each row of the live table.

```
uv run python net_profit_margin.py
```

256. Discuss: net profit margin is not comparable across sectors — a bank and a supermarket sit on completely different bases. What would you add to the output to stop a reader making that mistake?

Test it

Run `uv run python net_profit_margin.py`. The sample case must report 20.00%, the loss case -5.00%, and the zero-revenue case 0 with no traceback. All five assertions must pass without output. The live table must show five SGX tickers each with a margin and one of the four classification labels.

Note: The runnable source for this lab is labs/lab-23-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 24 — Multiple, Default and Named Arguments — A Flexible Valuation Toolkit

Learning objective: LO3: Construct functions using multiple, default and named arguments so one function serves several business cases without duplication.

Goal: You learn the three argument styles that stop a codebase filling up with near-duplicate functions. Multiple arguments give a fixed three-stock portfolio total; default arguments let `calculate_net_salary` assume a 15% tax rate while still accepting a custom one; named arguments make `calculate_dividend_yield(stock_price=120.00, dividend_per_share=3.60)` unambiguous regardless of order. You also meet the mutable-default-argument trap, one of Python's most notorious bugs.

What you'll build

`valuation_toolkit.py` — portfolio total, net salary, investment projection, dividend yield and ROI functions demonstrating multiple, default and named arguments. (Tools: uv, Python 3.12, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

257. Create the uv project.

```
uv init lab-24-valuation-toolkit && cd lab-24-valuation-toolkit
```

258. MULTIPLE ARGUMENTS: define `calculate_portfolio_total(stock1, stock2, stock3)` returning their sum, and call it with 1500.50, 2300.75 and 850.00. Then call it with only two values and read the `TypeError` carefully.

```
uv run python valuation_toolkit.py
```

259. DEFAULT ARGUMENTS: define `calculate_net_salary(gross_salary, tax_rate=0.15)` returning `gross_salary` minus the tax. Call it once on 5000 using the default and once with `tax_rate=0.20`, printing both.

```
uv run python valuation_toolkit.py
```

260. Add `estimate_investment_value(principal, years, annual_return=0.07)` returning `principal * (1 + annual_return) ** years`. Call it as `(1000, 5)` with the default and as `(1000, 5, 0.10)` with a custom return.

```
uv run python valuation_toolkit.py
```

261. NAMED ARGUMENTS: define `calculate_dividend_yield(dividend_per_share, stock_price)` returning `(dps / price) * 100`. Call it twice with the arguments in DIFFERENT orders using keywords, and confirm both give the right answer.

```
uv run python valuation_toolkit.py
```

262. Now call the same function positionally with the arguments swapped — (50.00, 2.50) — and observe it returns a nonsense 2000% yield with no error. Write a comment: named arguments are a correctness control, not a style preference.

```
uv run python valuation_toolkit.py
```

263. Add `calculate_roi(initial_investment, final_value)` returning `((final - initial) / initial) * 100` and call it with named arguments for clarity.

```
uv run python valuation_toolkit.py
```

264. AI-ASSIST: prompt your AI tool with: "Show me the mutable default argument trap using a function like `add_holding(ticker, portfolio=[])` that appends to a portfolio list. Demonstrate the bug by calling it three times, explain exactly why the list persists between calls, and give me the correct fix using `portfolio=None`." Run the buggy version first so you SEE the bug.

265. Add both the buggy and the fixed `add_holding` to your file, call each three times, and print the resulting portfolios side by side so the difference is undeniable.

```
uv run python valuation_toolkit.py
```

266. Combine the styles: extend `calculate_net_salary` with a second default argument `cpf_rate=0.20` and call it with only the keyword you want to override, leaving the other at its default.

```
uv run python valuation_toolkit.py
```

267. Enforce clarity: place a bare ``*`` in a signature (`def calculate_dividend_yield(*, dividend_per_share, stock_price)`) to make both arguments keyword-only, then confirm the positional call now raises a `TypeError` instead of silently returning 2000%.

```
uv run python valuation_toolkit.py
```

268. Discuss: a default value baked into a function is a policy decision. Who owns the 15% tax rate default, and what happens when the policy changes?

Test it

Run `uv run python valuation_toolkit.py`. Portfolio total must be \$4,651.25; net salary \$4,250.00 at the default rate and \$4,000.00 at 20%; investment value \$1,402.55 at the default 7%; both dividend yield calls 5.00% and 3.00%. The buggy `add_holding` must

show a growing shared list while the fixed one shows a fresh list each call, and the keyword-only version must raise `TypeError` on a positional call.

Note: The runnable source for this lab is `labs/lab-24-*.py`, and it is also included as a section of the combined notebook `labs/Al Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 25 — Variable Arguments — `*args` and `**kwargs` for Portfolios of Any Size

Learning objective: LO3: Construct functions accepting a variable number of arguments so one function handles a portfolio of any size or a metric set of any shape.

Goal: A portfolio does not have a fixed number of holdings, so a fixed-arity function is the wrong shape. You rebuild `calculate_portfolio_total` with `*stock_values` so it sums three holdings or fifty, add a category-plus-amounts expense reporter, then move to `**kwargs` to accept an arbitrary named metric set. You finish with unpacking — passing an existing list into an `*args` function with the star operator, and a dict into a `**kwargs` function with double-star.

What you'll build

`variable_args.py` — variable-argument portfolio, expense and metric functions using `*args` and `**kwargs`, plus argument unpacking from existing collections. (Tools: `uv`, Python 3.12, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

269. Create the `uv` project.

```
uv init lab-25-variable-args && cd lab-25-variable-args
```

270. In `variable_args.py` redefine `calculate_portfolio_total(*stock_values)` returning `sum(stock_values)`. Call it with three values (1500.50, 2300.75, 850.00) and then five values (500, 1200, 450, 3000, 150) — same function, both work.

```
uv run python variable_args.py
```

271. Print `len(stock_values)` inside the function to prove `*args` arrives as a tuple, and confirm the type with a print of `type(stock_values)`.

```
uv run python variable_args.py
```

272. Handle the empty case: call `calculate_portfolio_total()` with no arguments. `sum()` returns 0 — decide whether that is the right answer for an empty portfolio and note your reasoning in a comment.

```
uv run python variable_args.py
```

273. Define `list_expenses(category, *amounts)` which prints the category name, the number of items, and the total formatted to 2 decimals. Call it for 'Trading Fees' with four amounts and for 'Custody' with one.

```
uv run python variable_args.py
```

274. UNPACKING: given an existing list `holdings = [1500.50, 2300.75, 850.00, 1200.00]`, call `calculate_portfolio_total(*holdings)` with the star operator. Then call it without the star and observe you get a `TypeError` or a single-tuple result.

```
uv run python variable_args.py
```

275. Move to `**kwargs`: define `report_metrics(ticker, **metrics)` that prints the ticker then one aligned line per keyword metric. Call it as `report_metrics('D05.SI', roe=15.8, roa=1.2, npm=42.5, pe=11.4)`.

```
uv run python variable_args.py
```

276. Unpack a dict into it: build `metrics_dict = {'roe': 12.1, 'roa': 0.9, 'pe': 9.8}` and call `report_metrics('O39.SI', **metrics_dict)`.

```
uv run python variable_args.py
```

277. AI-ASSIST: prompt your AI tool with: "Write a function `build_trade_order(ticker, quantity, *, order_type='LIMIT', **extras)` that accepts required positional arguments, a keyword-only default, and arbitrary extra parameters. Show me the exact order the four argument categories must appear in a Python signature and what error I get if I put them in the wrong order." Test the wrong order yourself to see the `SyntaxError`.

278. Add the generated `build_trade_order` to your file, call it with and without extras, and print the resulting order dict.

```
uv run python variable_args.py
```

279. Combine everything: write `summarise_portfolio(name, *values, currency='SGD', **tags)` printing the portfolio name, holding count, total in the given currency, and any tags supplied.

```
uv run python variable_args.py
```

280. Discuss: `*args` and `**kwargs` make a function flexible but also make its contract invisible to a caller reading the signature. Where would you refuse to use them in a trading system?

Test it

Run `uv run python variable_args.py`. The three-stock total must be \$4,651.25 and the five-stock total \$5,300.00 from the same function. The unpacked call with `*holdings` must total \$5,851.25. `report_metrics` must print four aligned metric lines for D05.SI and three

for O39.SI, and summarise_portfolio must print the name, count, currency-formatted total and every tag supplied.

Note: The runnable source for this lab is labs/lab-25-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 26 — Activity: Max ROE Function — Finding the Best Performer in a Portfolio

Learning objective: LO3: Construct a function that scans a list of financial records and returns the maximum value by a chosen metric, and compare it against Python's built-in max.

Goal: You fetch Return on Equity for ten SGX companies, then write find_max_roe(data_list) that walks the list of dicts and returns the single best entry — implementing the max algorithm by hand before replacing it with max(data_list, key=lambda x: x['roe']). Handling the empty list (return None, not an IndexError) is the point of the lab: a screener that finds nothing must say so calmly. You then generalise it to find_max_by(data_list, metric) so one function serves ROE, ROA and margin.

What you'll build

max_roe.py — fetches ROE for 10 SGX tickers and reports the top performer via a hand-written max function, a built-in max version and a generalised find_max_by. (Tools: uv, yfinance, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

281. Create the uv project and add yfinance.

```
uv init lab-26-max-roe && cd lab-26-max-roe && uv add yfinance
```

282. In max_roe.py loop over the ten tickers ['D05.SI', 'U11.SI', 'O39.SI', 'Z74.SI', 'J36.SI', 'BN4.SI', 'V03.SI', 'C38U.SI', 'A17U.SI', 'C07.SI'], read stock.info.get('returnOnEquity'), and append {'ticker': ..., 'roe': roe * 100} only when the value is not None.

283. Wrap each fetch in try/except so one failing ticker does not abort the scan, and print how many of the ten returned usable ROE data.

```
uv run python max_roe.py
```

284. Print the full ROE sequence as an aligned table before any ranking, so you can verify the winner by eye.

```
uv run python max_roe.py
```

285. Write `find_max_roe(data_list)` by hand: return `None` immediately if the list is empty, otherwise seed `max_entry` with `data_list[0]` and loop comparing `entry['roe']` against `max_entry['roe']`, reassigning on a higher value.

286. Call it and print the winner: `'TOP PERFORMER: {ticker} with ROE {roe:.2f}%'`.

```
uv run python max_roe.py
```

287. Test the empty case: call `find_max_roe([])` and confirm it returns `None` rather than raising `IndexError`. Print a 'no qualifying stocks found' message when it does.

```
uv run python max_roe.py
```

288. Replace the hand-written loop with the built-in: `best = max(data_list, key=lambda x: x['roe'])` and confirm it names the same winner. Note that `max` on an empty list raises `ValueError` unless you pass `default=None`.

```
uv run python max_roe.py
```

289. AI-ASSIST: prompt your AI tool with: "Generalise `find_max_roe` into `find_max_by(data_list, metric)` that takes the metric name as a string so the same function works for `roe`, `roa` or `margin`. Add a matching `find_min_by`, handle the empty list and the case where the metric key is missing from some records, and explain the trade-off versus just using `max` with a key `lambda`." Review the missing-key handling especially.

290. Add the generalised functions, extend your fetch loop to also collect `returnOnAssets`, and call `find_max_by` twice — once for `'roe'` and once for `'roa'` — printing both winners.

```
uv run python max_roe.py
```

291. Add a top-three report using `sorted(data_list, key=lambda x: x['roe'], reverse=True)[:3]` and print it as a ranked podium with `enumerate`.

```
uv run python max_roe.py
```

292. Discuss: the highest ROE in the list may be driven by high leverage rather than operating strength. What second metric would you require before acting on this screener's winner?

Test it

Run `uv run python max_roe.py`. It must print the count of tickers with usable data, the full ROE table, a single TOP PERFORMER line, and confirmation that the hand-written `find_max_roe` and the built-in `max` agree on the same ticker. `find_max_roe([])` must return `None` with a 'no qualifying stocks' message, and `find_max_by` must report a winner for both `'roe'` and `'roa'`.

Note: The runnable source for this lab is `labs/lab-26-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 27 — Lambda and Map — Gross Profit Margin and ROE Across a Portfolio

Learning objective: LO3: Construct lambda expressions for one-off financial formulas and apply them across a sequence of stocks with map.

Goal: Lambda gives you a formula without the ceremony of def, and map applies it across a whole portfolio in one expression. You write lambdas for simple interest, currency conversion and percent change, then the topic's activity: a gross profit margin lambda $((\text{Revenue} - \text{COGS}) / \text{Revenue} * 100)$ with a zero-revenue ternary guard. You then use map with a lambda to compute ROE for a list of SGX companies from hard-coded net income and equity, and compare map against the equivalent list comprehension.

What you'll build

lambda_map.py — gross profit margin and ROE computed with lambda expressions and applied across a portfolio using map, benchmarked against a list comprehension. (Tools: uv, Python 3.12, Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

293. Create the uv project.

```
uv init lab-27-lambda-map && cd lab-27-lambda-map
```

294. In lambda_map.py write three lambdas: `calculate_interest = lambda p, r, t: p * (r / 100) * t`; `sgd_to_usd = lambda amount: amount * 0.74`; `percent_change = lambda new, old: ((new - old) / old) * 100`. Call each and print the results.

```
uv run python lambda_map.py
```

295. ACTIVITY — GROSS PROFIT MARGIN: write `calculate_gross_margin = lambda revenue, cogs: ((revenue - cogs) / revenue) * 100 if revenue != 0 else 0`. Call it with revenue 500000 and COGS 300000 and print revenue, COGS and the margin.

```
uv run python lambda_map.py
```

296. Confirm the guard by calling `calculate_gross_margin(0, 100)` and checking it returns 0 rather than raising `ZeroDivisionError`.

```
uv run python lambda_map.py
```

297. Introduce map with three quick examples: apply 9% GST to transactions [100.0, 250.50, 45.0, 1200.0] with `round(x * 1.09, 2)`; extract tickers from a list of portfolio dicts; and normalise prices [35.50, 36.20, 34.80, 37.00] against the first price.

```
uv run python lambda_map.py
```

298. Note that map returns a lazy iterator — print the map object itself before wrapping it in list() so you see the difference, and comment on why the list() call is required.

```
uv run python lambda_map.py
```

299. ACTIVITY — ROE VIA MAP: define financial_data as a list of dicts for five SGX companies with net_income and equity in SGD billions (D05.SI 10.3/57.1, U11.SI 5.7/45.2, O39.SI 7.0/54.8, Z74.SI 1.9/26.5, V03.SI 1.1/12.4).

300. Write roe_calculator as a lambda returning {'ticker': x['ticker'], 'roe': (x['net_income'] / x['equity']) * 100 if x['equity'] != 0 else 0}, then apply it with roe_results = list(map(roe_calculator, financial_data)).

301. Print the results as an aligned table with a header and separator, each ROE to 2 decimals.

```
uv run python lambda_map.py
```

302. AI-ASSIST: prompt your AI tool with: "Rewrite this map plus lambda as a list comprehension and as a named def function, then tell me which of the three you would use in a financial codebase that other analysts must maintain, and explain when a lambda is genuinely the better choice." Weigh the answer against your own reading of the code.

303. Add all three versions to the file, run them on the same data, and print a PASS line confirming they produce identical output.

```
uv run python lambda_map.py
```

304. Apply the gross margin lambda across a portfolio with map: given a list of (revenue, cogs) tuples for four companies, use map with a lambda taking the tuple and print each company's gross margin.

```
uv run python lambda_map.py
```

Test it

Run `uv run python lambda\_map.py`. Simple interest must be \$100.00, \$100 SGD must convert to \$74.00, percent change must be 20.00%, and the gross profit margin on 500000/300000 must be 40.00%. The map-based ROE table must show five tickers with D05.SI at approximately 18.04% and V03.SI at approximately 8.87%, and the final PASS line must confirm the map, comprehension and def versions agree.

Note: The runnable source for this lab is labs/lab-27-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 28 — Filter — Top Income Customers and High-ROE Stock Screening

Learning objective: LO3: Construct filter expressions with lambda to select the subset of a dataset that meets a business threshold, and chain filtering with sorting to produce a ranked shortlist.

Goal: Filter is the screening primitive: it keeps only the records where a lambda returns True. You generate 20 mock banking customers with random incomes between \$30,000 and \$200,000, filter for the top earners above a \$150,000 threshold, and produce a sorted priority-contact list. You then apply the same pattern to stock screening (ROE above 10%), large-transaction monitoring and SGX-only ticker selection, and finish by chaining filter with map to build a complete screen-then-compute pipeline — the closing exercise of Topic 4.

What you'll build

customer_filter.py — a filter-based screening tool producing a ranked top-income customer list, a high-ROE stock screen and a chained filter-then-map pipeline. (Tools: uv, Python 3.12 (random module), Google Colab / Cursor, Gemini or Copilot.)

Step-by-step

305. Create the uv project.

```
uv init lab-28-customer-filter && cd lab-28-customer-filter
```

306. In customer_filter.py import random and generate 20 mock customers with a list comprehension: `[{'id': i, 'name': f'Customer {i}', 'income': random.randint(30000, 200000)} for i in range(1, 21)]`.

307. Set `random.seed(42)` before generating so your run is reproducible — essential when a colleague must verify your screening result. Print the first three customers to confirm.

```
uv run python customer_filter.py
```

308. **ACTIVITY — TOP INCOME CUSTOMERS:** set `threshold = 150000` and apply `top_earners = list(filter(lambda c: c['income'] > threshold, customers))`.

309. Print the total customer count, the number of top earners, and then the top earners sorted descending by income using `sorted(top_earners, key=lambda x: x['income'], reverse=True)`, each formatted as `'Customer N: $NNN,NNN'`.

```
uv run python customer_filter.py
```

310. Test the empty result: raise the threshold to `500000`, re-run, and confirm the script prints a clear `'no customers meet this threshold'` message rather than an empty block. Restore `150000`.

```
uv run python customer_filter.py
```

311. Add three more filter examples: high-ROE stocks (list of dicts, keep roe > 10), large transactions from [120.50, 800.00, 45.00, 1500.25, 300.00] keeping x > 500, and SGX-only tickers from ['AAPL', 'D05.SI', 'TSLA', 'U11.SI', 'GOOG'] using t.endswith('.SI').

```
uv run python customer_filter.py
```

312. CHAIN FILTER AND MAP: filter the customers to those above the threshold, then map a lambda that adds a 'tier' key ('PLATINUM' above 180000, else 'GOLD') and an estimated annual fee at 0.5% of income. Print the enriched list.

```
uv run python customer_filter.py
```

313. AI-ASSIST: prompt your AI tool with: "Refactor this filter-plus-lambda screening code to use a named predicate function instead of an inline lambda, add a second filter criterion so only customers who are BOTH above the income threshold AND have an id below 15 are selected, and explain when filter is clearer than a list comprehension with an if." Review the combined predicate for correct and/or logic.

314. Apply the AI version, run it, and manually verify two or three selected records against the raw customer list to confirm both criteria really were applied.

```
uv run python customer_filter.py
```

315. Produce the final deliverable: a formatted 'PRIORITY WEALTH CONTACT LIST' with a header, a rank column from enumerate, name, income and tier, plus a footer line reporting the total addressable income of the shortlist.

```
uv run python customer_filter.py
```

316. Discuss: this screen uses income alone. Name two additional fields a bank would need before this list could be used for an actual wealth-management outreach, and one governance concern about acting on it.

Test it

Run `uv run python customer_filter.py`. With `random.seed(42)` the run must be reproducible across executions. It must print 20 total customers, the count above \$150,000, and a descending-sorted list of those top earners. Raising the threshold to 500000 must produce a 'no customers meet this threshold' message. The final PRIORITY WEALTH CONTACT LIST must show a rank, name, formatted income and PLATINUM/GOLD tier per row, plus a total addressable income footer.

Note: The runnable source for this lab is `labs/lab-28-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Topic 05 — Error Handling Using Exception (10%)

Exceptions versus syntax errors · Try and Except · Else clause · Finally

Key concepts

- An exception is an event during execution that disrupts normal flow — distinct from a syntax error, which prevents the program from running at all.
- Try/except blocks keep a financial pipeline alive when a ticker is delisted, an API fails or a denominator is zero.
- Common errors in finance code: IndexError, KeyError, TypeError, ValueError and ZeroDivisionError from a zero equity or revenue base.
- The else clause runs when no exception occurred; finally always runs and is used to release connections and write audit logs.
- Robust error handling is a regulatory expectation — a silent failure in a risk model is worse than a loud one.

Lab 29 — Exceptions vs Syntax Errors in a Price Loader

Learning objective: LO4: Distinguish an exception from a syntax error and read a Python traceback for a financial script.

Goal: The learner builds a small closing-price loader that contains, in turn, a deliberate syntax error and a deliberate runtime exception. By running both versions under uv the learner sees that a syntax error stops the interpreter before a single line executes, while an exception occurs mid-flight and leaves a traceback pointing at the failing line. The learner then pastes the traceback into an AI assistant and asks it to explain and fix the fault, comparing the AI's diagnosis with their own reading of the traceback.

What you'll build

A uv project `lab-29-exception-basics` containing price_loader.py, a captured traceback file traceback.txt, and a short written comparison of syntax errors versus exceptions. (Tools: uv, Python 3.12, VS Code / Cursor, AI coding assistant.)

Step-by-step

317. Create and enter the uv project for this lab.

```
uv init lab-29-exception-basics && cd lab-29-exception-basics
```

318. Create price_loader.py holding a dict of closing prices for AAPL, MSFT and DBS, and a function get_price(ticker) that returns prices[ticker].

```
touch price_loader.py
```

319. Introduce a deliberate SYNTAX error — remove the colon from the `def get\_price(ticker)` line — then run the script and record what Python reports.

```
uv run python price_loader.py
```

320. Discuss: the interpreter printed `SyntaxError: expected ':'` and NO other output ran. A syntax error is found at parse time, so the program never starts. Nothing can catch it.
321. Restore the colon, then trigger a `RUNTIME` exception by calling `get_price('LEHMAN')` for a ticker that is not in the dict. Run again and capture the full traceback to a file.

```
uv run python price_loader.py 2> traceback.txt; cat traceback.txt
```

322. Discuss: this time the earlier print statements DID run before the `KeyError` appeared. An exception happens during execution, is raised at a specific line, and can be caught.
323. AI STEP — paste the captured traceback into your AI assistant with this prompt: "Here is a Python traceback from my stock price loader: <paste traceback.txt>. Explain in plain English what `KeyError` means here, which line raised it, and show me the smallest correct fix using `try/except` that returns `None` for an unknown ticker."
324. Review the AI's answer critically: check that it catches `KeyError` specifically and NOT a bare `except:`, then apply the fix to `price_loader.py`.
325. Re-run the fixed script and confirm the unknown ticker is now reported gracefully while the known tickers still print their prices.

```
uv run python price_loader.py
```

326. Write a three-line comment block at the top of `price_loader.py` summarising the difference between a syntax error and an exception in your own words.

Test it

Running `uv run python price_loader.py` exits with status 0, prints prices for AAPL, MSFT and DBS, and prints a friendly 'ticker LEHMAN not found' message instead of crashing. `traceback.txt` contains the original `KeyError` trace for reference.

Note: The runnable source for this lab is `labs/lab-29-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 30 — Try/Except on a Failing Market-Data Fetch

Learning objective: LO4: Wrap a live market-data call in `try/except` so a delisted or invalid ticker does not halt the pipeline.

Goal: The learner writes a market-data fetcher that downloads recent closing prices for a watchlist using `yfinance`. One entry in the watchlist is a delisted or misspelled ticker that returns an empty frame. The learner catches the failure per ticker so the remaining instruments are still processed, and separates a network/API exception from an empty-result condition — a distinction that matters when a batch job runs unattended overnight.

What you'll build

A uv project `lab-30-market-fetch` with `fetch_prices.py` that returns a summary dict of successful tickers and a separate list of failed tickers with the reason for each failure. (Tools: uv, yfinance, pandas, AI coding assistant.)

Step-by-step

327. Create the uv project and add the market-data dependencies.

```
uv init lab-30-market-fetch && cd lab-30-market-fetch && uv add yfinance pandas
```

328. Write `fetch_prices.py` with `WATCHLIST = ['AAPL', 'MSFT', 'ENRNQ', 'D05.SI']` — `ENRNQ` is a delisted ticker that will return no data.

```
touch fetch_prices.py
```

329. Write a naive loop with no error handling that calls `yf.download(t, period='5d')` and prints the last close for each ticker. Run it and observe the failure.

```
uv run python fetch_prices.py
```

330. Discuss: the delisted ticker produced an empty `DataFrame`, so ``df['Close'].iloc[-1]`` raised `IndexError` and the loop died BEFORE reaching `D05.SI`. One bad symbol silently cost you the rest of the watchlist.

331. Refactor the loop so each ticker is fetched inside its own `try/except`. Catch `IndexError` and `KeyError` for empty data, and catch `Exception` as a last resort for network/API faults, appending (ticker, reason) to a failures list.

332. Run the fixed script and confirm every good ticker is priced and the bad one is reported.

```
uv run python fetch_prices.py
```

333. AI STEP — prompt your AI assistant: "Review this Python function that fetches prices with yfinance inside a try/except. Is catching bare `Exception` acceptable here for an overnight batch job? Suggest more specific exception types and show how to log the failing ticker with the traceback so an analyst can debug it the next morning."

334. Apply the AI's logging suggestion, but verify yourself that a genuine bug (for example a typo in a column name) is still visible rather than swallowed by the broad handler.

335. Add a summary print at the end: how many tickers succeeded, how many failed, and the reason for each failure.

```
uv run python fetch_prices.py
```

336. Discuss: why the pipeline must record which tickers were skipped — a portfolio valuation computed from three of four holdings is wrong, not merely incomplete.

Test it

The script completes without crashing, prints a latest close for AAPL, MSFT and D05.SI, and ends with a failure report naming ENRNQ and its reason. Exit status is 0.

Note: The runnable source for this lab is labs/lab-30-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 31 — ZeroDivisionError in an ROE and Ratio Calculator

Learning objective: LO4: Handle ZeroDivisionError and TypeError in financial ratio calculations where the denominator can legitimately be zero.

Goal: Return on Equity divides net income by shareholders' equity — and a distressed company can report zero or negative equity, which makes the ratio undefined rather than merely large. The learner builds a ratio calculator covering ROE, net profit margin and debt-to-equity, then hardens it against a zero denominator and against a None value arriving from an incomplete fundamentals feed. The lab makes the key point that returning 0.0 for an undefined ratio is a data-quality lie; returning None and flagging it is correct.

What you'll build

A uv project `lab-31-roe-guard` with ratios.py exposing safe_roe(), safe_margin() and safe_debt_to_equity(), all returning None with a printed warning when the ratio is undefined. (Tools: uv, Python 3.12, AI coding assistant.)

Step-by-step

337. Create the uv project for the ratio calculator.

```
uv init lab-31-roe-guard && cd lab-31-roe-guard
```

338. Write ratios.py with an unguarded roe(net_income, equity) that simply returns net_income / equity, plus a COMPANIES list containing one firm with equity = 0 and one with equity = None.

```
touch ratios.py
```

339. Run the script over the company list and observe it crash on the zero-equity firm.

```
uv run python ratios.py
```

340. Discuss: ZeroDivisionError is not a bug in your code — it is the data telling you the ratio has no meaning. A company with zero book equity has an undefined ROE.

341. Rewrite roe() as safe_roe() with try/except catching ZeroDivisionError (return None, warn 'undefined: zero equity') and TypeError (return None, warn 'missing equity value').

342. Add safe_margin(net_income, revenue) and safe_debt_to_equity(debt, equity) using the same guarded pattern, and run over all companies.

```
uv run python ratios.py
```

343. AI STEP — prompt your AI assistant: "I have three financial ratio functions that each repeat the same try/except for ZeroDivisionError and TypeError. Refactor them into one reusable `safe_divide(numerator, denominator, label)` helper that returns None and logs a warning, and rewrite the three ratios to use it. Keep the warning text specific to each ratio."
344. Test the AI's `safe_divide` against a negative-equity firm (equity = -500000). Decide as a class whether a negative denominator should return a number or a flag, and justify the choice in a comment.
345. Print a final table of every company with its three ratios, showing 'n/a' where the result is None.

```
uv run python ratios.py
```

346. Discuss: why silently substituting 0.0 for an undefined ROE would corrupt a downstream screening model that ranks companies by ROE.

Test it

``uv run python ratios.py`` prints a ratio table for every company, shows 'n/a' for the zero-equity and None-equity firms with a warning line for each, and never raises.

Note: The runnable source for this lab is `labs/lab-31-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 32 — KeyError and ValueError in a Portfolio Input Handler

Learning objective: LO4: Catch `KeyError` for a missing portfolio holding and `ValueError` for malformed user input, using multiple except branches.

Goal: The learner builds an interactive portfolio lookup and loan-amount entry tool. Querying a ticker that is not in the holdings dictionary raises `KeyError`; typing '1,000,000' or 'fifty thousand' into a loan-amount prompt raises `ValueError` from `int()`. The learner writes a single try block with multiple except branches, then adds an input-validation loop that re-prompts until the value is usable — the standard pattern for any customer-facing finance form.

What you'll build

A uv project ``lab-32-portfolio-input`` with `portfolio_tool.py` providing a resilient ticker lookup and a validated loan-amount prompt that never crashes on bad input. (Tools: uv, Python 3.12, AI coding assistant.)

Step-by-step

347. Create the uv project.

```
uv init lab-32-portfolio-input && cd lab-32-portfolio-input
```

348. Write `portfolio_tool.py` with `PORTFOLIO = {'AAPL': 120, 'MSFT': 80, 'D05.SI': 500}` and an unguarded lookup that prints `PORTFOLIO[ticker]` for a ticker typed by the user.

```
touch portfolio_tool.py
```

349. Run it and enter 'TSLA', a ticker you do not hold, to trigger `KeyError`.

```
uv run python portfolio_tool.py
```

350. Add a second unguarded prompt: `amount = int(input('Loan amount: '))`. Run it and enter '1,000,000' to trigger `ValueError`.

```
uv run python portfolio_tool.py
```

351. Discuss: `KeyError` says the key is absent from the mapping; `ValueError` says the type is right but the content is not convertible. They need different messages to the user.

352. Wrap the lookup in `try/except KeyError`, printing 'You do not hold TSLA — holdings are: AAPL, MSFT, D05.SI' and continuing. Verify with a held and an unheld ticker.

```
uv run python portfolio_tool.py
```

353. Wrap the loan prompt in a `while True` loop with `try/except ValueError` that re-prompts until a valid positive integer is entered, and also rejects zero or negative amounts.

```
uv run python portfolio_tool.py
```

354. AI STEP — prompt your AI assistant: "Here is my Python input handler for a loan application form. Rewrite it so it accepts amounts typed with commas or a currency symbol like '\$250,000', still raises `ValueError` for genuinely invalid text, and caps the number of retries at three before exiting with a clear message."

355. Test the AI-generated parser against the awkward cases yourself: '250000', '\$250,000', '-5000', '0', 'abc' and an empty string. Fix anything the AI got wrong.

356. Discuss: why ``.get(ticker, default)`` is often better than `try/except KeyError` for a dictionary, and when the explicit exception is still the clearer choice.

Test it

The tool survives every input in the test set: unknown tickers produce a helpful holdings list, malformed amounts re-prompt, and a valid amount is echoed back formatted as currency. The script never terminates with a traceback.

Note: The runnable source for this lab is `labs/lab-32-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 33 — Else, Finally and a Custom `InsufficientDataError` for a Risk Model

Learning objective: LO4: Use the else and finally clauses and raise a custom exception to enforce a data-sufficiency rule in a risk model.

Goal: A volatility model computed from four price observations is not a risk estimate — it is noise. The learner defines a custom `InsufficientDataError`, raises it when a price history is shorter than the model's minimum window, and structures the calculation with the full try / except / else / finally form: else runs the downstream risk write-up only when no exception occurred, and finally always closes the simulated API connection and appends a line to an audit log — the behaviour a regulator expects to see evidenced.

What you'll build

A uv project `lab-33-risk-guard` with `risk_model.py` defining `InsufficientDataError`, a guarded `compute_volatility()`, and `audit.log` recording every run whether it succeeded or failed. (Tools: uv, pandas, numpy, AI coding assistant.)

Step-by-step

357. Create the project and add the numeric dependencies.

```
uv init lab-33-risk-guard && cd lab-33-risk-guard && uv add pandas numpy
```

358. In `risk_model.py` define `class `InsufficientDataError(Exception)`` with a docstring stating it is raised when a price history is too short for a reliable risk estimate.

```
touch risk_model.py
```

359. Write `compute_volatility(prices, min_obs=30)` that raises `InsufficientDataError` with a message naming the actual and required observation counts when `len(prices) < min_obs`, and otherwise returns the annualised standard deviation of daily returns.

360. Write a fake connection object with `open()` and `close()` methods that print their action, to stand in for a market-data API session.

361. Assemble the full structure: try (open connection, compute volatility) / except `InsufficientDataError` (print the reason, mark the run FAILED) / else (print the volatility and the risk band, mark the run OK) / finally (close the connection and append a timestamped line to `audit.log`).

362. Run the model against a 250-day series and confirm the else branch executes.

```
uv run python risk_model.py
```

363. Run it again against a 12-day series and confirm the custom exception is caught, the else branch is skipped, and the connection is STILL closed.

```
uv run python risk_model.py --short
```

364. Inspect the audit log and confirm both runs are recorded with their outcome.

```
cat audit.log
```

365. Discuss: finally ran in both the success and failure paths. Without it, a failed model run would leak the API session and leave no audit trail — the failure that is invisible is the one that gets you sanctioned.
366. AI STEP — prompt your AI assistant: "Review this Python risk model. I use a custom `InsufficientDataError` plus `try/except/else/finally`. Suggest two more domain-specific exceptions a volatility model should raise (for example on stale or non-numeric prices), show the class definitions, and explain when a custom exception is better than reusing `ValueError`."
367. Implement one of the AI's suggested exceptions yourself, add a test case that triggers it, and confirm `audit.log` still records the run.

```
uv run python risk_model.py --stale
```

Test it

All three runs (normal, short history, stale prices) finish without an uncaught traceback, 'connection closed' prints every time, and `audit.log` contains one timestamped OK/FAILED line per run.

Note: The runnable source for this lab is `labs/lab-33-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 34 — Activity: Loan Risk Classifier with Guarded Data Loading

Learning objective: LO4: Build an XGBoost multi-class credit-risk classifier whose data loading, feature selection and scoring are wrapped in exception handling that degrades gracefully.

Goal: The capstone activity for Topic 5, from the reference slide. The learner trains an XGBoost multi-class credit-risk classifier on `creditloandata.csv`. The loader is wrapped in `try/except`: if the CSV is absent or unreadable, the `except` branch generates a synthetic dataset of 120 mock records with income, debt-to-income and credit-score features and assigns risk labels 0/1/2 from a scoring function, so the model always trains. A second guard handles missing feature columns by dropping them and warning, rather than crashing. The model is evaluated with accuracy, a classification report, a Seaborn confusion-matrix heatmap and a multiclass ROC curve.

What you'll build

A uv project ``lab-34-loan-risk-classifier`` with `loan_risk.py`, a synthetic-data fallback, a printed classification report, `confusion_matrix.png` and `roc_curve.png`. (Tools: uv, pandas, numpy, scikit-learn, xgboost, matplotlib, seaborn, AI coding assistant.)

Step-by-step

368. Create the project and add the modelling stack.

```
uv init lab-34-loan-risk-classifier && cd lab-34-loan-risk-classifier && uv  
add pandas numpy scikit-learn xgboost matplotlib seaborn
```

369. AI STEP — prompt your AI assistant with the full requirement: "Write a Python script that trains an XGBoost multi-class classifier for credit risk. Wrap the data loading in try/except: try to read 'creditloandata.csv' with pandas; if FileNotFoundError or pandas.errors.EmptyDataError is raised, generate a synthetic dataset of 120 records with columns income, debt_to_income and credit_score, and assign risk labels 0, 1, 2 from a custom scoring function. Split 75/25 stratified, train XGBClassifier, then report accuracy, a classification_report, a Seaborn confusion-matrix heatmap and a multiclass ROC curve. Print clearly which data source was used."

370. Save the generated code as loan_risk.py and READ IT before running — check the fallback actually produces three populated classes and that the except branch is not catching bare Exception.

```
touch loan_risk.py
```

371. Run the script with NO creditloandata.csv present and confirm the synthetic fallback path is taken and the model still trains.

```
uv run python loan_risk.py
```

372. Create a small real creditloandata.csv with the three feature columns and a risk column, re-run, and confirm the try branch is taken instead.

```
uv run python loan_risk.py
```

373. Add a THIRD guard: wrap feature selection in try/except KeyError so that if credit_score is missing from the CSV the script warns, drops that feature and trains on the remaining two rather than crashing.

374. Test the missing-feature path by deleting the credit_score column from the CSV and re-running.

```
uv run python loan_risk.py
```

375. Discuss: the degraded two-feature model still produced a number. Is a silently degraded credit model acceptable? Agree as a class where the boundary sits between graceful degradation and a failure that must stop the run.

376. Break the script deliberately — feed it a CSV whose risk column contains the string 'high' instead of an integer — capture the resulting traceback, and paste it into your AI assistant with: "Explain this traceback from my XGBoost credit risk script and give me the exception handler that converts the label column safely, raising a clear InvalidLabelError if the values cannot be mapped to 0, 1 or 2."

```
uv run python loan_risk.py 2> traceback.txt
```

377. Apply and test the AI's handler, then confirm the confusion-matrix heatmap and ROC curve are written to disk on a successful run.

```
uv run python loan_risk.py && ls -la *.png
```

378. Write a five-line 'error handling contract' comment at the top of loan_risk.py listing every exception the script handles and the business decision behind each response.

Test it

The script runs to completion in all four scenarios — CSV present, CSV absent, feature column missing, label column malformed — never exits with an uncaught traceback, prints accuracy above 0.60 on the synthetic data, and writes confusion_matrix.png and roc_curve.png.

Note: The runnable source for this lab is labs/lab-34-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Topic 06 — Import and Process Finance Data (14%)

Pandas package · DataFrame and Series · Import · Filter and slice · Clean missing data

Key concepts

- Pandas is the workhorse of financial data analysis; Series is a single labelled column and DataFrame is a labelled table.
- Financial data is imported from CSV, Excel and market APIs such as yfinance.
- Filtering and slicing select the rows and columns relevant to an analysis — a date range, a sector, or a set of tickers.
- Missing market data is detected and cleaned by dropping or imputing, and the choice must be documented because it changes the result.
- Moving averages and returns are derived columns that feed trading signals.

Lab 35 — Pandas Series vs DataFrame with Price Data

Learning objective: LO5: Construct and distinguish a pandas Series and a DataFrame, and index both with financial labels.

Goal: The learner builds a Series of closing prices indexed by date, then a DataFrame holding open, high, low, close and volume for several tickers. Working with both side by side makes the distinction concrete: a Series is one labelled column — a single instrument's price history — while a DataFrame is a labelled table where a column selection returns a Series again. The learner also compares `.loc` label-based access against `.iloc` position-based access, the source of most beginner indexing bugs in finance code.

What you'll build

A uv project `lab-35-series-dataframe` with `series_vs_frame.py` demonstrating Series construction, DataFrame construction, column and row selection, and a printed comparison of `.loc` versus `.iloc`. (Tools: uv, pandas, AI coding assistant.)

Step-by-step

379. Create the project and add pandas.

```
uv init lab-35-series-dataframe && cd lab-35-series-dataframe && uv add pandas
```

380. In `series_vs_frame.py` build a Series of 10 daily closing prices for D05.SI indexed by a `pd.date_range` of business days, and print it with its `.index`, `.dtype` and `.name`.

```
touch series_vs_frame.py
```

381. Print `series.mean()`, `series.max()` and `series.pct_change()` and note that a Series carries its index through every operation.

```
uv run python series_vs_frame.py
```

382. Build a DataFrame with columns Open, High, Low, Close and Volume over the same dates, and print `df.shape`, `df.columns` and `df.dtypes`.

383. Select a single column with `df['Close']` and confirm with `type()` that it is a Series, not a DataFrame; then select `df[['Close', 'Volume']]` and confirm that a list of columns returns a DataFrame.

```
uv run python series_vs_frame.py
```

384. Discuss: the single-bracket versus double-bracket distinction. Passing a Series where a DataFrame is expected is a very common error in financial pipelines.

385. Compare label and position indexing: `df.loc['2024-03-05']` against `df.iloc[2]`, and `df.loc['2024-03-04':'2024-03-08']` against `df.iloc[1:4]`. Note that `.loc` slices are INCLUSIVE of the endpoint while `.iloc` slices are not.

```
uv run python series_vs_frame.py
```

386. AI STEP — prompt your AI assistant: "Explain the difference between a pandas Series and a DataFrame using a stock price example, and give me three short code snippets showing when `.loc` silently returns different rows than `.iloc` on a date-indexed price table. Include the off-by-one trap on slice endpoints."

387. Run the AI's snippets yourself and verify each claim against your own DataFrame — do not accept the endpoint behaviour on trust.

```
uv run python series_vs_frame.py
```

388. Add a multi-ticker DataFrame with a two-level column index (ticker, field) and select just the Close columns for all tickers.

```
uv run python series_vs_frame.py
```

Test it

The script prints the Series with its DatetimeIndex, confirms `type(df['Close'])` is Series and `type(df[['Close']])` is DataFrame, and shows `.loc` and `.iloc` returning the expected rows with the inclusive/exclusive endpoint difference clearly labelled.

Note: The runnable source for this lab is `labs/lab-35-*.py`, and it is also included as a section of the combined notebook `labs/Al Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 36 — Import Finance Data from CSV

Learning objective: LO5: Import a financial CSV into pandas with correct date parsing, index and dtypes.

Goal: The learner imports a historical prices CSV and confronts the three defects that make most real finance CSVs unusable on the first read: dates loaded as strings, numbers loaded as objects because of thousands separators or currency symbols, and an unnamed index column. The learner fixes each with `read_csv` parameters — `parse_dates`, `index_col`, `thousands`, `dtype` and `na_values` — rather than by post-processing, and confirms the fix with `.dtypes`.

What you'll build

A uv project `'lab-36-csv-import'` containing `prices.csv`, `load_csv.py` with a correctly parameterised `read_csv` call, and a printed dtype report proving Close is float64 and the index is a DatetimeIndex. (Tools: uv, pandas, AI coding assistant.)

Step-by-step

389. Create the project and add pandas.

```
uv init lab-36-csv-import && cd lab-36-csv-import && uv add pandas
```

390. Create `prices.csv` with columns Date, Ticker, Open, Close, Volume — deliberately writing volumes with thousands commas like `'1,204,300'`, a few cells as `'N/A'`, and dates in DD/MM/YYYY form.

```
touch prices.csv
```

391. Load it with a naive `pd.read_csv('prices.csv')` and print `df.dtypes` and `df.head()`.

```
uv run python load_csv.py
```

392. Discuss: Date came in as object, Volume came in as object because of the commas, and `'N/A'` was not recognised as missing. Every downstream calculation on these columns would either fail or be silently wrong.

393. Re-import with `parse_dates=['Date'], dayfirst=True, index_col='Date', thousands=','` and `na_values=['N/A', '-', '-']`, then reprint `df.dtypes`.

```
uv run python load_csv.py
```

394. Verify the index with `df.index.dtype` and confirm `df.index.is_monotonic_increasing`; sort the index if it is not.

```
uv run python load_csv.py
```

395. AI STEP — prompt your AI assistant: "Here are the first five rows of my stock price CSV <paste rows>. Write the single `pd.read_csv` call that parses the DD/MM/YYYY dates into a `DatetimeIndex`, strips thousands separators from Volume, treats 'N/A' and '-' as missing, and forces Close to `float64`. Explain each parameter you used."

396. Compare the AI's `read_csv` call with your own. Check specifically that it set `dayfirst` correctly — a wrong `dayfirst` silently swaps day and month for the first twelve days of each month and is nearly invisible.

397. Prove the parse is right by printing the rows for a date after the 12th of a month and checking it against the raw CSV text.

```
uv run python load_csv.py
```

398. Save the cleaned frame back out and confirm the round trip is stable.

```
uv run python load_csv.py && head -3 prices_clean.csv
```

Test it

`df.dtypes` shows Close and Open as `float64`, Volume as `int64` or `float64`, and `df.index` is a `DatetimeIndex` sorted ascending. The 'N/A' cells appear as `NaN`, and a spot-checked date after the 12th matches the raw CSV.

Note: The runnable source for this lab is `labs/lab-36-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 37 — Import Live Market Data with yfinance

Learning objective: LO5: Download multi-ticker market data with `yfinance` and reshape the result into an analysis-ready `DataFrame`.

Goal: The learner downloads three years of daily data for a small multi-market portfolio with `yfinance` and immediately meets its awkward output shape: a `MultiIndex` column frame when several tickers are requested, and a flat frame when only one is. The learner writes a loader that normalises both cases into a tidy long-format `DataFrame` with Date, Ticker and Close, caches it to CSV so the rest of the course is not dependent on the network, and wraps the download in the `try/except` pattern learned in Topic 5.

What you'll build

A uv project `lab-37-yfinance-import` with `load_market.py` producing a tidy long-format DataFrame and a cached `market_data.csv` covering three years and at least four tickers. (Tools: uv, yfinance, pandas, AI coding assistant.)

Step-by-step

399. Create the project and add the data dependencies.

```
uv init lab-37-yfinance-import && cd lab-37-yfinance-import && uv add yfinance pandas
```

400. Download three years of daily data for a single ticker and inspect the shape of what comes back.

```
uv run python -c "import yfinance as yf; d=yf.download('AAPL', period='3y'); print(d.columns); print(d.head())"
```

401. Now download four tickers at once — AAPL, MSFT, D05.SI, O39.SI — and print the columns again to see the MultiIndex.

```
uv run python load_market.py
```

402. Discuss: with one ticker the columns are flat; with several they are a (field, ticker) MultiIndex. Code written against one shape breaks against the other — a very common production bug.

403. In `load_market.py` write `fetch(tickers, period)` that always returns a tidy long frame with columns Date, Ticker, Open, High, Low, Close, Volume, using `.stack()` to flatten the MultiIndex case and adding a Ticker column in the single case.

404. Wrap the download in try/except (Topic 5 pattern) so a failed or empty ticker is reported and skipped rather than aborting the whole fetch.

```
uv run python load_market.py
```

405. Cache the tidy frame to `market_data.csv` and add a guard so the script reads the cache when it exists and only hits the network otherwise.

```
uv run python load_market.py && wc -l market_data.csv
```

406. AI STEP — prompt your AI assistant: "yf.download returns a MultiIndex column DataFrame for multiple tickers and a flat one for a single ticker. Write a function that normalises both into a long-format DataFrame with columns Date, Ticker, Open, High, Low, Close, Volume. Handle `auto_adjust` and explain what Adj Close means for a dividend-paying stock."

407. Test the AI's normaliser with a one-ticker call AND a four-ticker call and confirm both produce identical column layouts.

```
uv run python load_market.py
```

408. Discuss the AI's explanation of adjusted close: why backtesting on unadjusted close overstates losses around ex-dividend dates.
409. Print a summary — the date range, the row count per ticker, and the number of trading days captured.

```
uv run python load_market.py
```

Test it

market_data.csv exists, covers roughly three years, contains all four tickers in long format with one row per ticker per trading day, and the loader returns the same column layout whether called with one ticker or four.

Note: The runnable source for this lab is labs/lab-37-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 38 — Inspect a Dataset with head, info and describe

Learning objective: LO5: Profile a financial dataset with head, tail, info, describe and value_counts to identify data-quality problems before analysis.

Goal: Before any calculation, an analyst profiles the data. The learner runs the standard inspection sequence over the cached market data and reads each output as a finance question: does info() show fewer non-null Closes than rows, meaning gaps? Does describe() show a minimum price of zero or a negative volume, meaning corrupt records? Does value_counts() over Ticker show one instrument with far fewer rows, meaning a late listing or a suspension? The learner writes a reusable profiling function that answers all of them.

What you'll build

A uv project `lab-38-inspect-data` with profile.py printing a full data-quality report and a written list of every anomaly found in the dataset. (Tools: uv, pandas, AI coding assistant.)

Step-by-step

410. Create the project, add pandas, and copy in market_data.csv from lab 37.

```
uv init lab-38-inspect-data && cd lab-38-inspect-data && uv add pandas && cp ../lab-37-yfinance-import/market_data.csv .
```

411. Load the CSV with proper date parsing and print df.head(10) and df.tail(10) to see the start and end of the series.

```
uv run python profile.py
```

412. Print `df.info()` and read it as a finance question: compare the non-null count of Close against the total row count to find missing prices.

```
uv run python profile.py
```

413. Print `df.describe()` and inspect the min, max and quartiles of Close and Volume for impossible values — a zero price, a negative volume, or a max thousands of times the median.

```
uv run python profile.py
```

414. Print `df['Ticker'].value_counts()` and check every ticker has a similar row count; investigate any that does not.

```
uv run python profile.py
```

415. Discuss: the three anomalies you found and what each would do to a portfolio return calculation if left uncorrected.

416. Write `profile_dataset(df)` that packages the whole sequence — shape, dtypes, null counts per column, describe, per-ticker row counts and date coverage — into one printed report.

```
uv run python profile.py
```

417. AI STEP — prompt your AI assistant: "Write a pandas data-quality report function for a long-format stock price DataFrame with columns Date, Ticker, Close, Volume. It must report: missing values per column, duplicate (Date, Ticker) rows, non-positive prices, gaps of more than 5 calendar days in each ticker's date coverage, and any date that is not a business day. Print it as a readable table."

418. Run the AI's report on your data and verify each finding by hand against the raw CSV before trusting it.

```
uv run python profile.py
```

419. Record the confirmed anomalies as a comment block at the top of `profile.py` — this is the data-quality note that must accompany any published analysis.

Test it

`profile.py` prints shape, dtypes, per-column null counts, describe output, per-ticker row counts and date coverage, plus a list of duplicate or non-positive-price rows. The learner can name at least two concrete data-quality issues found in their own dataset.

Note: The runnable source for this lab is `labs/lab-38-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 39 — Filter and Slice by Date, Sector and Ticker

Learning objective: LO5: Select subsets of financial data with boolean masks, date-range slicing and multi-condition filters.

Goal: The learner attaches a sector mapping to the price data and then answers a series of real screening questions with boolean masks: all bank stocks in Q1, every day AAPL closed above its own 200-day average, the intersection of a date window and a ticker list. The lab covers the operator-precedence trap unique to pandas — `&` and `|` with parentheses, never `and`/`or` — plus `.isin()`, `.between()`, `.query()` and `.loc` date slicing on a `DatetimeIndex`.

What you'll build

A uv project `lab-39-filter-slice` with `screen.py` answering five stated screening questions, each as a documented boolean mask. (Tools: uv, pandas, AI coding assistant.)

Step-by-step

420. Create the project, add pandas and copy in the market data.

```
uv init lab-39-filter-slice && cd lab-39-filter-slice && uv add pandas &&
cp ../lab-37-yfinance-import/market_data.csv .
```

421. Add a `SECTORS` dict mapping each ticker to a sector — AAPL and MSFT to Technology, D05.SI and O39.SI to Financials — and map it into a new `Sector` column.

```
touch screen.py
```

422. Question 1: slice the last full calendar year using `.loc` with a date range on the `DatetimeIndex`, and print the row count.

```
uv run python screen.py
```

423. Question 2: select only the Financials rows with a boolean mask `df['Sector'] == 'Financials'`, and print the tickers it contains to verify.

```
uv run python screen.py
```

424. Question 3: combine two conditions — Financials AND Close above 30 — using `&` with parentheses around each condition. Then deliberately try it with `and` and read the resulting ValueError.`

```
uv run python screen.py
```

425. Discuss: pandas cannot evaluate the truth of a whole Series, which is why `and` fails and &` is required. Forgetting the parentheses gives a wrong answer silently because of operator precedence.`

426. Question 4: use `.isin(['AAPL', 'D05.SI'])` to select a ticker subset, and `.between()` on Close to select a price band, and combine both.

```
uv run python screen.py
```

427. Question 5: rewrite the most complex mask using `.query()` and compare readability with the bracket form; time both on the full dataset.

```
uv run python screen.py
```

428. AI STEP — prompt your AI assistant: "Using a pandas DataFrame with columns Date (DatetimeIndex), Ticker, Sector, Close and Volume, write filters for: all Financials rows in 2024 where Close rose more than 2 percent from the prior day, and the 10 highest-volume days for any Technology stock. Use boolean masks with correct parentheses and explain why ``and`` would fail."

429. Run the AI's filters and verify the row counts by cross-checking a couple of rows manually — an incorrect mask usually returns a plausible but wrong number of rows.

```
uv run python screen.py
```

430. Save each screened subset to its own CSV for use in the next lab.

```
uv run python screen.py && ls *.csv
```

Test it

All five screening questions print a result with its row count, the ``and`` version raises the expected `ValueError` with an explanatory printed note, and the `.query()` result is identical to the bracket-mask result.

Note: The runnable source for this lab is `labs/lab-39-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 40 — Handle Missing Market Data and Compare the Choices

Learning objective: LO5: Detect missing market data with `isna`, and compare `dropna`, `fillna` and forward-fill, quantifying how each choice changes the analytical result.

Goal: The central judgement lab of Topic 6. Market data has gaps: public holidays that differ between SGX and NASDAQ, trading suspensions, and vendor errors. The learner injects and detects gaps, then applies three treatments — `dropna`, `fillna` with the mean, and forward-fill — to the SAME series, computes total return and volatility under each, and puts the three answers side by side. The point is that the numbers genuinely differ, so the cleaning choice is an analytical decision that must be documented, not a formatting step.

What you'll build

A uv project `lab-40-missing-data` with `clean_compare.py` printing a comparison table of total return, mean daily return and annualised volatility under each of the four treatments, plus a written recommendation. (Tools: uv, pandas, numpy, AI coding assistant.)

Step-by-step

431. Create the project and add the dependencies.

```
uv init lab-40-missing-data && cd lab-40-missing-data && uv add pandas  
numpy && cp ../lab-37-yfinance-import/market_data.csv .
```

432. Reindex a single ticker's Close series onto a full business-day calendar so every non-trading gap becomes an explicit NaN, then count them with `.isna().sum()`.

```
uv run python clean_compare.py
```

433. Print the specific dates that are missing and identify which are public holidays versus genuine data gaps.

```
uv run python clean_compare.py
```

434. Treatment A — `dropna()`: drop the missing rows, then compute total return, mean daily return and annualised volatility.

```
uv run python clean_compare.py
```

435. Treatment B — `fillna(series.mean())`: fill every gap with the series mean and recompute the same three metrics.

```
uv run python clean_compare.py
```

436. Treatment C — `ffill()`: forward-fill the last traded price across the gap, which is the standard convention for a non-trading day, and recompute.

```
uv run python clean_compare.py
```

437. Treatment D — `interpolate(method='time')`: linearly interpolate across the gap and recompute.

```
uv run python clean_compare.py
```

438. Print all four sets of metrics in one comparison table and identify the largest discrepancy between treatments.

```
uv run python clean_compare.py
```

439. DISCUSS — the key judgement of this topic: filling with the series mean injects a huge artificial price jump and inflates volatility; `dropna` understates the number of periods; forward-fill correctly implies zero return on a non-trading day but suppresses

volatility if used across a long suspension. Which treatment would you defend to an auditor, and for which kind of gap?

440. AI STEP — prompt your AI assistant: "I have a daily stock Close series with NaNs from public holidays and one three-week trading suspension. Compare dropna, fillna(mean), ffill and time interpolation for computing daily returns and annualised volatility. For each, state the assumption it makes about what happened during the gap and when it biases the volatility estimate up or down."
441. Check the AI's reasoning against your own measured numbers from the comparison table — does its claimed direction of bias match what you actually observed?

```
uv run python clean_compare.py
```

442. Write your recommendation as a comment block: which treatment for a holiday, which for a suspension, and which you would never use.

Test it

The comparison table prints four rows of metrics that visibly differ, the missing dates are listed by date, and the file ends with a written, justified recommendation naming a treatment per gap type.

Note: The runnable source for this lab is labs/lab-40-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 41 — Derived Columns — Daily Returns and Moving Averages

Learning objective: LO5: Create derived analytical columns — daily and cumulative returns, rolling moving averages and rolling volatility — with pct_change, shift and rolling.

Goal: The learner turns a price table into an analytics table. Daily simple returns come from pct_change(); log returns from np.log on a shift() ratio; cumulative growth from cumprod. Rolling windows produce the 20-day and 50-day simple moving averages and a 20-day rolling volatility. The lab emphasises correct grouping — every derived column must be computed per ticker with groupby, or the first return of each ticker will be contaminated by the last price of the previous one, a mistake that is easy to make and hard to see.

What you'll build

A uv project `lab-41-derived-columns` with features.py producing enriched_prices.csv containing daily_return, log_return, cum_return, sma_20, sma_50 and vol_20 per ticker. (Tools: uv, pandas, numpy, AI coding assistant.)

Step-by-step

443. Create the project and add the dependencies.

```
uv init lab-41-derived-columns && cd lab-41-derived-columns && uv add  
pandas numpy && cp ../lab-37-yfinance-import/market_data.csv .
```

444. Load the long-format data, sort by Ticker then Date, and add daily_return using df.groupby('Ticker')['Close'].pct_change().

```
uv run python features.py
```

445. Deliberately compute the same column WITHOUT groupby and compare the first row of each ticker between the two versions.

```
uv run python features.py
```

446. Discuss: without groupby the first row of MSFT was computed against the last price of AAPL, producing a nonsense return. Nothing raised an error — this is a silent correctness bug that survives all the way to a published number.

447. Add log_return using np.log(close / close.shift(1)) within the groupby, and explain when log returns are preferred (time-additivity for multi-period aggregation).

```
uv run python features.py
```

448. Add cum_return as (1 + daily_return).cumprod() - 1 per ticker, and print the three-year cumulative return for each instrument.

```
uv run python features.py
```

449. Add sma_20 and sma_50 with groupby + rolling(window).mean(), and confirm the first 19 and 49 values respectively are NaN by design.

```
uv run python features.py
```

450. Add vol_20 as the 20-day rolling standard deviation of daily_return, annualised by multiplying by the square root of 252.

```
uv run python features.py
```

451. AI STEP — prompt your AI assistant: "Given a long-format pandas DataFrame with columns Date, Ticker and Close, write vectorised code that adds daily simple returns, log returns, cumulative return, a 20-day and 50-day SMA, and 20-day annualised rolling volatility — all computed per ticker with groupby so no value leaks across tickers. Explain why min_periods matters on the rolling calls."

452. Verify the AI's output on the boundary rows: check the first row of each ticker is NaN for daily_return, and that no SMA value appears before its window is full.

```
uv run python features.py
```

453. Write the enriched frame to enriched_prices.csv and print a per-ticker summary of cumulative return and mean annualised volatility.

```
uv run python features.py && head -3 enriched_prices.csv
```

Test it

enriched_prices.csv contains all six derived columns, the first row of every ticker has NaN daily_return, sma_50 is NaN for exactly the first 49 rows of each ticker, and the per-ticker summary prints a plausible cumulative return and volatility.

Note: The runnable source for this lab is labs/lab-41-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 42 — Activity: Algorithmic Trading — Moving Average Crossover Backtest

Learning objective: LO5: Backtest a moving-average crossover strategy on three years of market data, computing performance KPIs and visualising the signals.

Goal: The capstone activity for Topic 6, from the reference slide. The learner prompts for a ticker and for short and long moving-average windows, downloads three years of data with yfinance, generates buy and sell signals where the short SMA crosses the long SMA, and backtests the resulting position against buy-and-hold. Performance is reported as total return, annualised return, annualised volatility, Sharpe ratio at a zero risk-free rate, and maximum drawdown. A matplotlib chart plots the price with both moving averages, green '^' markers at buy signals and red 'v' markers at sell signals. The whole run is wrapped in the Topic 5 error handling and uses the Topic 6 cleaning decisions.

What you'll build

A uv project `lab-42-ma-crossover-backtest` with backtest.py, a printed KPI table comparing the strategy with buy-and-hold, and backtest_chart.png showing price, both SMAs and the marked signals. (Tools: uv, yfinance, pandas, numpy, matplotlib, AI coding assistant.)

Step-by-step

454. Create the project and add the full backtesting stack.

```
uv init lab-42-ma-crossover-backtest && cd lab-42-ma-crossover-backtest &&
uv add yfinance pandas numpy matplotlib
```

455. AI STEP — prompt your AI assistant with the full specification: "Write a Python script using yfinance, pandas, numpy and matplotlib that backtests a Moving Average Crossover strategy on 3 years of daily data. Prompt the user for a stock ticker and integer short and long SMA windows. Generate a buy signal when the short SMA crosses above the long SMA and a sell signal when it crosses below. Compute Total Return, Annualised Return, Annualised Volatility, Sharpe Ratio at a 0 percent risk-free rate, and Maximum Drawdown, for both the strategy and buy-and-hold. Plot the close price with both moving average lines, green '^' markers at buy

signals and red 'v' markers at sell signals, and save it as `backtest_chart.png`. Handle an invalid ticker with `try/except`."

456. Save the result as `backtest.py` and READ IT BEFORE RUNNING. Check three specific things: that the signal is shifted by one day before being applied to returns, that the SMAs use only past data, and that the drawdown is computed from the cumulative equity curve.

`touch backtest.py`

457. Run the backtest on AAPL with windows 20 and 50.

`uv run python backtest.py`

458. Discuss the look-ahead bias check: if the position is not shifted by one day, the strategy trades on a signal derived from the same day's close — an impossible trade that flatters every KPI. Confirm your script shifts.

459. Fix any look-ahead bias you found, re-run, and compare the KPIs before and after — note how much the Sharpe ratio fell.

`uv run python backtest.py`

460. Run the same backtest on a Singapore bank, D05.SI, with the same windows and compare the results across the two markets.

`uv run python backtest.py`

461. Sweep the window pair: run 10/30, 20/50 and 50/200 on the same ticker and tabulate total return and Sharpe for each.

`uv run python backtest.py`

462. Discuss overfitting: the best window pair in this sample is not the best pair out of sample. Why does picking the winner from a sweep overstate expected live performance?

463. Inspect `backtest_chart.png` and verify the markers sit exactly at the crossover points and that buys and sells alternate.

`open backtest_chart.png`

464. Add the Topic 6 cleaning decision explicitly: forward-fill any missing closes before computing the SMAs, and print a note stating the treatment used.

`uv run python backtest.py`

465. Write a short conclusion in the script header: did the crossover strategy beat buy-and-hold on your ticker after the look-ahead fix, and what would you need to add before trusting the result (transaction costs, slippage, out-of-sample testing)?

Test it

The script accepts a ticker and two windows, prints a KPI table with Total Return, Annualised Return, Annualised Volatility, Sharpe Ratio and Maximum Drawdown for both the strategy and buy-and-hold, writes `backtest_chart.png` with both SMA lines and alternating green '^' and red 'v' markers at the crossovers, and exits cleanly on an invalid ticker.

Note: The runnable source for this lab is `labs/lab-42-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Topic 07 — Aggregate and Visualize Finance Data (14%)

Concat, append and merge · Groupby · Pivot table · Assess code for gaps · Test and visualise · Streamlit

Key concepts

- Finance datasets are joined with `concat`, `append` and `merge` to combine prices, fundamentals and customer records.
- `Groupby` aggregates loan and portfolio data by sector, region, gender or status.
- Pivot tables cross-tabulate metrics such as average debt-to-income ratio by segment.
- Charts turn an aggregation into a decision — trends, distributions and comparisons.
- `Streamlit` publishes the analysis as an interactive web application that a business user can operate without touching Python.

Lab 43 — Generate Mock Credit-Loan Data with `df_finance`

Learning objective: LO6: Aggregate and visualise financial data — build the credit-loan dataset that every Topic 7 lab aggregates.

Goal: The learner creates a `uv` project and generates a reproducible mock credit-loan portfolio of 500 applicants as a pandas DataFrame named `df_finance`, with the columns used throughout Topics 7 and 9: `Applicant_ID`, `Gender`, `Region`, `Employment_Sector`, `Annual_Income`, `Credit_Score`, `Years_at_Job`, `Loan_Type`, `Loan_Status`, `Loan_Amount_Requested`, `Debt_to_Income_Ratio`, `Missed_Payments_Last_2Y`, `Education` and `Marital_Status`. An AI prompt drafts the generator; the learner reviews the ranges for financial realism before accepting the code.

What you'll build

A `uv` project containing `generate_data.py` and a saved `df_finance.csv` of 500 mock loan applicants. (Tools: `uv`, Python 3.12, `pandas`, `NumPy`, VS Code / Cursor AI assistant.)

Step-by-step

466. Create and enter the `uv` project for this lab.

```
uv init lab-43-mock-loan-data && cd lab-43-mock-loan-data
```

467. Add the data libraries to the project.

```
uv add pandas numpy
```

468. Prompt your AI assistant with this exact text: "Write a Python function `generate_finance_data(n=500, seed=42)` that returns a pandas DataFrame `df_finance` of mock credit-loan applicants with columns `Applicant_ID`, `Gender` (Male/Female), `Region` (North/South/East/West/Central), `Employment_Sector` (Banking/Technology/Healthcare/Manufacturing/Retail/Government), `Annual_Income` (lognormal, 30000-250000 SGD), `Credit_Score` (300-850), `Years_at_Job` (0-35), `Loan_Type` (Personal/Mortgage/Auto/Education), `Loan_Status` (Approved/Rejected/Pending), `Loan_Amount_Requested`, `Debt_to_Income_Ratio` (0.05-0.75), `Missed_Payments_Last_2Y` (0-6), `Education` and `Marital_Status`. Use `numpy` with a fixed seed so results are reproducible."

469. Save the generated code as `generate_data.py` and read every line — confirm the income and credit-score ranges are financially plausible and that the seed is fixed.

470. Add a line at the end that writes the frame to CSV:

```
df_finance.to_csv('df_finance.csv', index=False).
```

471. Run the generator.

```
uv run python generate_data.py
```

472. Inspect the shape, dtypes and first rows in a REPL to confirm the schema.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
print(d.shape); print(d.dtypes); print(d.head())"
```

473. Check the categorical spread — every `Region`, `Loan_Status` and `Employment_Sector` must be represented.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
[print(c, d[c].value_counts().to_dict()) for c in  
['Region', 'Loan_Status', 'Employment_Sector']]"
```

474. Discuss: why does a fixed random seed matter when a model output will be reviewed by a risk committee or an auditor?

Test it

`df_finance.csv` exists with 500 rows and 14 columns; re-running `generate_data.py` produces a byte-identical file, proving reproducibility.

Note: The runnable source for this lab is `labs/lab-43-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 44 — Join Finance Datasets with concat, append and merge

Learning objective: LO6: Combine prices, fundamentals and customer records into one analysis-ready frame using concat and merge.

Goal: The learner combines the loan book with two extra sources: a second branch's loan file (stacked vertically with `pd.concat`) and a regional risk-rating reference table (joined horizontally with `pd.merge` on Region). The lab contrasts inner, left and outer joins and shows what a mismatched key silently does to a loan book — the classic cause of disappearing applicants in a reconciliation.

What you'll build

A `join_data.py` script producing `df_all` — the combined multi-branch loan book enriched with regional risk ratings. (Tools: uv, pandas, VS Code / Cursor AI assistant.)

Step-by-step

475. Create the project and add pandas.

```
uv init lab-44-join-finance-data && cd lab-44-join-finance-data && uv add pandas numpy
```

476. Copy `df_finance.csv` from lab 43 into this project.

```
cp ../lab-43-mock-loan-data/df_finance.csv .
```

477. Create a second branch file by regenerating with a different seed and an offset `Applicant_ID`, saving it as `df_branch2.csv`.

478. Stack the two branches vertically and reset the index.

```
uv run python -c "import pandas as pd; a=pd.read_csv('df_finance.csv'); b=pd.read_csv('df_branch2.csv'); df=pd.concat([a,b], ignore_index=True); print(len(a), len(b), len(df))"
```

479. Prompt your AI assistant: "Given a pandas DataFrame `df` of loan applicants with a `Region` column, build a small reference DataFrame `df_risk` with one row per Region and columns `Regional_Risk_Rating` (A/B/C) and `Base_Interest_Rate`. Then merge it into `df` on `Region` using a left join, and print how many rows have a null `Regional_Risk_Rating` after the merge."

480. Review the generated merge — confirm it uses `how='left'` so no applicant is dropped, and that the null check is present.

481. Deliberately break one `Region` value (e.g. 'central' lower-case) and re-run the merge to see the applicant lose its rating.

482. Compare the row counts of an inner, left and outer merge on the broken key.

```
uv run python join_data.py
```

483. Discuss: in a loan reconciliation, which join type is safest and why must the post-merge row count always be asserted?

484. Save the final combined frame.

```
uv run python -c "import pandas as pd; print('df_all saved')"
```

Test it

df_all has exactly len(branch1) + len(branch2) rows after the left merge — no applicant is silently lost — and zero null Regional_Risk_Rating values once the key is fixed.

Note: The runnable source for this lab is labs/lab-44-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 45 — Groupby: Annual Income by Gender and Region

Learning objective: LO6: Aggregate financial data with groupby across one and multiple grouping keys.

Goal: The learner uses df.groupby() to compute average and median Annual_Income by Gender, then by Region, then by both keys together to produce a MultiIndex result. The lab covers unstack() to turn the MultiIndex into a readable Gender x Region matrix, and interprets the income gaps as a fair-lending analyst would.

What you'll build

A groupby_income.py script printing income statistics by Gender, by Region and by the Gender x Region cross-section. (Tools: uv, pandas, VS Code / Cursor AI assistant.)

Step-by-step

485. Create the project and add pandas.

```
uv init lab-45-groupby-income && cd lab-45-groupby-income && uv add pandas
```

486. Copy in df_finance.csv from lab 43.

```
cp ../lab-43-mock-loan-data/df_finance.csv .
```

487. Compute mean Annual_Income by Gender.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
print(d.groupby('Gender')['Annual_Income'].mean().round(2))"
```

488. Compute mean and median Annual_Income by Region and sort descending by the mean.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
print(d.groupby('Region')  
      ['Annual_Income'].agg(['mean','median']).round(2).sort_values('mean',  
                           ascending=False))"
```

489. Group by both keys to get a MultiIndex Series.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
print(d.groupby(['Gender', 'Region'])['Annual_Income'].mean().round(2))"
```

490. Prompt your AI assistant: "Using a pandas DataFrame df_finance, group Annual_Income by Gender and Region, take the mean, then unstack Region into columns so the result is a Gender-by-Region matrix rounded to 2 decimals. Add a final column showing each gender's overall mean and a final row showing each region's overall mean."

491. Save the generated code as groupby_income.py, read it, and confirm unstack() is applied to the correct level.

492. Run the script and inspect the matrix.

```
uv run python groupby_income.py
```

493. Interpret the output: which Gender x Region cell has the widest gap from the overall mean?

494. Discuss: groupby means say nothing about causation — what confounders (sector, tenure, education) would a fair-lending review demand before acting on this table?

Test it

The unstacked matrix has one row per Gender and one column per Region, with the row/column totals matching the ungrouped overall mean to 2 decimal places.

Note: The runnable source for this lab is labs/lab-45-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 46 — Loan Application Counts by Status and Type

Learning objective: LO6: Aggregate categorical loan data into counts and approval rates with groupby and crosstab.

Goal: The learner counts loan applications by Loan_Status and Loan_Type using groupby().size(), value_counts() and pd.crosstab(), then converts the raw counts into approval rates per loan product with normalize. The output answers a real credit-operations question: which product line has the weakest approval rate and the largest pending backlog?

What you'll build

A loan_counts.py script producing a Loan_Type x Loan_Status count table plus an approval-rate table. (Tools: uv, pandas, VS Code / Cursor AI assistant.)

Step-by-step

495. Create the project and add pandas.

```
uv init lab-46-loan-counts && cd lab-46-loan-counts && uv add pandas
```

496. Copy in df_finance.csv.

```
cp ../lab-43-mock-loan-data/df_finance.csv .
```

497. Count applications per Loan_Status.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
print(d['Loan_Status'].value_counts())"
```

498. Count applications per Loan_Type and Loan_Status with groupby().size().

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
print(d.groupby(['Loan_Type', 'Loan_Status']).size())"
```

499. Build the same table as a cross-tab with margins (row and column totals).

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
print(pd.crosstab(d['Loan_Type'], d['Loan_Status'], margins=True))"
```

500. Prompt your AI assistant: "Using pandas, build a crosstab of Loan_Type against Loan_Status from df_finance, then produce a second table normalised by row so each cell is the percentage of that loan type's applications in that status, rounded to 1 decimal. Print which Loan_Type has the lowest Approved percentage and which has the highest Pending percentage."

501. Save it as loan_counts.py, review the normalize='index' argument, and confirm each row sums to 100%.

502. Run the script.

```
uv run python loan_counts.py
```

503. Verify the totals: the crosstab grand total must equal len(df_finance).

504. Discuss: why is an approval rate a more actionable KPI for a credit committee than a raw approval count?

Test it

Every row of the normalised table sums to 100.0 (+/- 0.1) and the crosstab margin total equals the DataFrame row count.

Note: The runnable source for this lab is labs/lab-46-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 47 — Multi-Metric Aggregation by Employment Sector with .agg()

Learning objective: LO6: Perform multi-metric aggregation, flatten MultiIndex columns and produce a board-ready sector table.

Goal: This is the reference slide 137 activity. The learner groups `df_finance` by `Employment_Sector` and passes a dictionary to `.agg()` to compute the mean, median and standard deviation of `Annual_Income` plus the mean `Credit_Score` and mean `Years_at_Job` in one pass. The resulting MultiIndex columns are flattened and renamed to `Avg_Income`, `Median_Income`, `Income_Std_Dev`, `Avg_Credit_Score` and `Avg_Tenure_Years`.

What you'll build

A `sector_agg.py` script producing a flat, renamed one-row-per-sector summary table exported to `sector_summary.csv`. (Tools: `uv`, `pandas`, VS Code / Cursor AI assistant.)

Step-by-step

505. Create the project and add `pandas`.

```
uv init lab-47-sector-aggregation && cd lab-47-sector-aggregation && uv add pandas
```

506. Copy in `df_finance.csv`.

```
cp ../lab-43-mock-loan-data/df_finance.csv .
```

507. Run a single-metric baseline: mean `Annual_Income` by `Employment_Sector`.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv'); print(d.groupby('Employment_Sector')['Annual_Income'].mean().round(2))"
```

508. Prompt your AI assistant with the slide-137 brief: "Using `pandas` on a `DataFrame` named `df_finance`, group the data by `Employment_Sector` and perform a multi-metric aggregation with `.agg()`: calculate the mean, median and standard deviation for `Annual_Income`, and the mean for `Credit_Score` and `Years_at_Job`. Flatten the resulting MultiIndex columns and rename them to `Avg_Income`, `Median_Income`, `Income_Std_Dev`, `Avg_Credit_Score` and `Avg_Tenure_Years`. Round to 2 decimals, sort by `Avg_Income` descending and export to `sector_summary.csv`."

509. Save the answer as `sector_agg.py` and review the `.agg()` dictionary — confirm it maps each column to its list of functions.

510. Inspect how the MultiIndex is flattened: it should join the tuple levels, e.g. `['_'.join(c) for c in df.columns]`.

511. Run the script.

```
uv run python sector_agg.py
```

512. Verify the flattened column names are exactly the five required names.

```
uv run python -c "import pandas as pd; print(list(pd.read_csv('sector_summary.csv').columns))"
```

513. Cross-check one cell by hand: recompute `Income_Std_Dev` for the Banking sector independently.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
print(d[d.Employment_Sector=='Banking']['Annual_Income'].std().round(2))"
```

514. Discuss: why does a high Income_Std_Dev within a sector matter more to a lender than the sector's mean income?

Test it

sector_summary.csv has one row per Employment_Sector, exactly the five renamed columns plus the sector key, and no MultiIndex or tuple-shaped headers remain.

Note: The runnable source for this lab is labs/lab-47-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 48 — Pivot Table of Debt-to-Income Ratio and Matplotlib Visualisation

Learning objective: LO6: Cross-tabulate a risk metric with pivot_table and visualise aggregations as bar, line and distribution charts.

Goal: The learner builds the reference slide 139 pivot table — average Debt_to_Income_Ratio indexed by Education with Marital_Status as columns, rounded to 3 decimals — then turns the Topic 7 aggregations into three matplotlib charts: a bar chart of average income by sector, a line chart of approval rate across credit-score bands, and a histogram of the Debt_to_Income_Ratio distribution. All charts are saved as PNG for the dashboard.

What you'll build

A pivot_and_charts.py script writing dti_pivot.csv plus chart_income_by_sector.png, chart_approval_by_score.png and chart_dti_distribution.png. (Tools: uv, pandas, matplotlib, VS Code / Cursor AI assistant.)

Step-by-step

515. Create the project and add pandas and matplotlib.

```
uv init lab-48-pivot-and-charts && cd lab-48-pivot-and-charts && uv add  
pandas matplotlib
```

516. Copy in df_finance.csv.

```
cp ../lab-43-mock-loan-data/df_finance.csv .
```

517. Build the slide-139 pivot table.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_finance.csv');  
print(pd.pivot_table(d, values='Debt_to_Income_Ratio', index='Education',  
columns='Marital_Status', aggfunc='mean').round(3))"
```

518. Add `margins=True` to get the row and column averages, and confirm the grand mean matches `df['Debt_to_Income_Ratio'].mean()`.
519. Prompt your AI assistant: "Using pandas and matplotlib on `df_finance`, produce three saved PNG charts: (1) a horizontal bar chart of mean Annual_Income by Employment_Sector sorted descending, (2) a line chart of the Approved rate across Credit_Score bands created with `pd.cut` into 50-point bins, and (3) a histogram with 30 bins of Debt_to_Income_Ratio with a vertical dashed line at the mean. Give every chart a title, axis labels with units, and use `plt.tight_layout()` before `savefig` at `dpi=150`."
520. Save as `pivot_and_charts.py` and review: confirm matplotlib uses a non-interactive backend (`matplotlib.use('Agg')`) so it runs headless.
521. Run the script.

```
uv run python pivot_and_charts.py
```

522. Confirm all four output files exist.

```
ls -la dti_pivot.csv chart_*.png
```

523. Open each PNG and check it is readable: are the axis labels present and is the sector bar chart actually sorted?
524. Discuss: which of the three chart types best supports a credit-policy decision, and why is a distribution more honest than a single average?

Test it

`dti_pivot.csv` contains one row per Education level and one column per Marital_Status with all values rounded to 3 decimals, and the three PNG files open with titles and labelled axes.

Note: The runnable source for this lab is `labs/lab-48-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 49 — Streamlit Capstone Part 1 — Portfolio & Loan Analytics App Shell

Learning objective: LO6: Publish an aggregation as an interactive Streamlit application with sidebar filters and KPI metric cards.

Goal: The Streamlit capstone STARTS here. The learner creates the app project, writes `app.py`, and builds the shell of the 'Portfolio & Loan Analytics' dashboard: page config, a cached data loader for `df_finance.csv`, a sidebar with multiselect filters for Employment_Sector, Region and Loan_Status, and a row of four KPI metric cards (Total Applications, Approval Rate, Average Annual Income, Average Debt-to-Income Ratio) that all recompute live as the filters change. This same `app.py` is extended in labs 50, 61 and 62.

What you'll build

A Streamlit project with app.py serving the filtered KPI dashboard at localhost:8501.
(Tools: uv, Streamlit, pandas, VS Code / Cursor AI assistant.)

Step-by-step

525. Create the capstone project — keep this folder, labs 50, 61 and 62 all extend it.

```
uv init loan-analytics-app && cd loan-analytics-app
```

526. Add Streamlit and the data libraries.

```
uv add streamlit pandas matplotlib
```

527. Copy df_finance.csv from lab 43 into the app folder.

```
cp ../lab-43-mock-loan-data/df_finance.csv .
```

528. Prompt your AI assistant: "Write a Streamlit app.py titled 'Portfolio & Loan Analytics'. Use st.set_page_config with a wide layout. Add a load_data() function decorated with @st.cache_data that reads df_finance.csv. In the sidebar add three st.multiselect filters for Employment_Sector, Region and Loan_Status, each defaulting to all values. Filter the DataFrame by all three selections, then display four st.metric cards in st.columns(4): Total Applications, Approval Rate as a percentage, Average Annual Income formatted as SGD with thousands separators, and Average Debt-to-Income Ratio to 3 decimals. Show the filtered table below with st.dataframe."

529. Save the answer as app.py — this is THE file every remaining capstone lab edits. Read it and confirm @st.cache_data is on the loader and the filter is applied before the metrics are computed.

530. Run the app.

```
uv run streamlit run app.py
```

531. Open http://localhost:8501 and change one sidebar filter — verify all four metric cards update.

532. Set the Loan_Status filter to Approved only and confirm the Approval Rate card reads 100%.

533. Clear all values from one filter and confirm the app shows a friendly empty-state message rather than crashing; add an if filtered.empty: st.warning(...) guard if it does not.

534. Commit or note the file structure — labs 50, 61 and 62 add to this exact app.py.

```
ls -la app.py df_finance.csv pyproject.toml
```

Test it

The app loads at localhost:8501; selecting a single Employment_Sector changes Total Applications to that sector's row count, and clearing a filter shows a warning instead of a traceback.

Note: The runnable source for this lab is labs/lab-49-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 50 — Streamlit Capstone Part 2 — Charts and Pivot Tab in the Dashboard

Learning objective: LO6: Extend the Streamlit application with tabbed groupby charts and an interactive pivot table driven by user input.

Goal: The learner EXTENDS the same app.py from lab 49, adding st.tabs with three tabs: 'Sector Analysis' (bar chart of mean Annual_Income by Employment_Sector plus the .agg() sector summary table from lab 47), 'Loan Mix' (grouped bar chart of Loan_Type by Loan_Status counts), and 'Risk Pivot' (an interactive pivot table where the user chooses the index and column dimensions from selectboxes and sees average Debt_to_Income_Ratio, rendered with a background gradient). All charts respect the sidebar filters.

What you'll build

An extended app.py with a three-tab analytics section: sector charts, loan mix, and a user-configurable risk pivot. (Tools: uv, Streamlit, pandas, matplotlib, VS Code / Cursor AI assistant.)

Step-by-step

535. Return to the capstone app folder from lab 49 — do NOT create a new project.

```
cd loan-analytics-app
```

536. Confirm the lab 49 app still runs before changing anything.

```
uv run streamlit run app.py
```

537. Prompt your AI assistant: "Extend this existing Streamlit app.py. Below the KPI metric cards add st.tabs(['Sector Analysis','Loan Mix','Risk Pivot']). In Sector Analysis, show a matplotlib horizontal bar chart of mean Annual_Income by Employment_Sector from the filtered DataFrame via st.pyplot, plus the multi-metric .agg() table with Avg_Income, Median_Income, Income_Std_Dev, Avg_Credit_Score, Avg_Tenure_Years. In Loan Mix, show a grouped bar chart of application counts by Loan_Type and Loan_Status. In Risk Pivot, add two st.selectbox controls letting the user pick the pivot index and columns from Education, Marital_Status, Region and Gender, then display pivot_table of mean Debt_to_Income_Ratio styled with a background gradient. Keep the existing sidebar filters and metric cards unchanged."

538. Paste the generated tabs into app.py under the metric-card row. Verify the existing filter and KPI code is untouched.

539. Re-run the app.

```
uv run streamlit run app.py
```

540. Click through all three tabs and confirm each renders without error.
541. Change a sidebar filter and confirm the charts in every tab redraw with the filtered subset — this proves the filter is applied upstream of the tabs.
542. In the Risk Pivot tab, set index=Education and columns=Marital_Status and check the numbers match dti_pivot.csv from lab 48.
543. Guard against a pivot with an empty filtered frame — wrap the pivot in a try/except or an .empty check so the tab shows a message instead of an exception.
544. Discuss: what does moving from a static PNG chart to a Streamlit app change about who in the bank can use this analysis?

Test it

All three tabs render under any filter combination; the Risk Pivot values for Education x Marital_Status match the lab 48 pivot to 3 decimals when no filters are applied.

Note: The runnable source for this lab is labs/lab-50-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Topic 08 — Object Oriented Programming (8%)

Classes and objects · Methods and overloading · Initializer and destructor · Inheritance · Polymorphism

Key concepts

- A class is the blueprint of an object, bundling data (attributes) with behaviour (methods).
- An object is an instance of a class — one Stock object per holding in a portfolio.
- The initializer `__init__` sets up an instance's state; the destructor releases resources.
- Inheritance lets an Equity or Bond class extend a common Instrument class without duplicating code.
- Polymorphism allows one valuation call to behave correctly for every instrument type.

Lab 51 — Build a Stock Class — Attributes, Methods and `__init__`

Learning objective: LO7: Model a financial instrument as a class with attributes and behaviour, initialised through `__init__`.

Goal: The learner writes the first finance class: Stock. Instance attributes (ticker, name, sector, shares, buy_price, current_price) are set in the `__init__` initializer, and methods add behaviour — `market_value()`, `cost_basis()`, `unrealised_pnl()` and `pnl_percent()`. The lab contrasts a class attribute (a shared CURRENCY = 'SGD') with instance attributes, and shows why every holding needs its own state.

What you'll build

A stock.py module defining the Stock class with four financial methods, plus a demo creating three holdings. (Tools: uv, Python 3.12, VS Code / Cursor AI assistant.)

Step-by-step

545. Create the project.

```
uv init lab-51-stock-class && cd lab-51-stock-class
```

546. Write a minimal class by hand first — class Stock with a class attribute CURRENCY = 'SGD' and a method describe() — to see the blueprint-versus-instance distinction before adding __init__.

547. Prompt your AI assistant: "Write a Python class Stock for a portfolio holding. The __init__ initializer takes ticker, name, sector, shares, buy_price and current_price and stores them as instance attributes, with a class attribute CURRENCY = 'SGD'. Add methods market_value() returning shares * current_price, cost_basis() returning shares * buy_price, unrealised_pnl() returning market_value minus cost_basis, and pnl_percent() returning the P&L as a percentage of cost basis guarding against a zero cost basis. Add a describe() method printing a formatted one-line summary with the currency."

548. Save the answer as stock.py. Read it and confirm every method takes self as its first parameter and reads attributes via self.

549. Create three objects — instances of the same class with different state.

```
uv run python -c "from stock import Stock; s=Stock('D05','DBS Group','Banking',500,32.10,38.40); print(s.market_value(), s.unrealised_pnl(), round(s.pnl_percent(),2))"
```

550. Prove instance independence: create two Stock objects, change current_price on one, and confirm the other is unaffected.

551. Prove the class attribute is shared: print Stock.CURRENCY and s.CURRENCY and confirm they are the same object.

552. Test the zero-cost-basis guard by creating a Stock with buy_price=0 and calling pnl_percent().

```
uv run python -c "from stock import Stock; s=Stock('X','Test','Tech',10,0,5); print(s.pnl_percent())"
```

553. Write a demo block under if __name__ == '__main__': that builds three holdings and prints each describe() line.

```
uv run python stock.py
```

554. Discuss: what did wrapping price and shares in a class buy us over keeping parallel lists of tickers and prices?

Test it

Three Stock objects report independent market values; `pnl_percent()` returns 0.0 (not a `ZeroDivisionError`) when `buy_price` is 0; `Stock.CURRENCY` is visible from every instance.

Note: The runnable source for this lab is `labs/lab-51-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 52 — A Portfolio Class Holding Many Stock Objects

Learning objective: LO7: Compose objects — a Portfolio class that aggregates many Stock instances and reports book-level metrics.

Goal: The learner builds a Portfolio class that holds a list of Stock objects (composition). Methods include `add_holding()`, `total_value()`, `total_pnl()`, `weights()` returning each holding's share of the book, `sector_allocation()` aggregating by sector, and `top_holdings(n)`. The lab demonstrates why the Portfolio should never duplicate price data — it delegates to each Stock's own methods.

What you'll build

A `portfolio.py` module defining Portfolio, importing Stock from lab 51, with a demo book of six holdings. (Tools: uv, Python 3.12, pandas, VS Code / Cursor AI assistant.)

Step-by-step

555. Create the project and add pandas.

```
uv init lab-52-portfolio-class && cd lab-52-portfolio-class && uv add pandas
```

556. Copy `stock.py` from lab 51 so Portfolio can import it.

```
cp ../lab-51-stock-class/stock.py .
```

557. Prompt your AI assistant: "Write a Python class Portfolio that holds a list of Stock objects. `__init__` takes an owner name and an optional list of holdings. Add `add_holding(stock)`, `total_value()` summing each holding's `market_value()`, `total_cost()`, `total_pnl()`, `weights()` returning a dict of ticker to percentage of `total_value`, `sector_allocation()` returning a dict of sector to total market value, and `top_holdings(n=3)` returning the `n` largest holdings by market value. The Portfolio must call the Stock methods rather than recomputing prices itself."

558. Save as `portfolio.py` and review — confirm `total_value()` calls `h.market_value()` and does not re-implement `shares * price`.

559. Build a six-stock demo book across at least three sectors.

560. Print the book-level totals.

```
uv run python -c "from portfolio import build_demo; p=build_demo(); print(p.total_value(), p.total_pnl())"
```

561. Check the weights sum to 100%.

```
uv run python -c "from portfolio import build_demo; p=build_demo();  
print(round(sum(p.weights().values()),6))"
```

562. Confirm `sector_allocation()` totals equal `total_value()`.

```
uv run python portfolio.py
```

563. Update one Stock's `current_price` and re-run `total_value()` — the Portfolio total must change with no Portfolio code touched. This is the payoff of delegation.

564. Discuss: composition (a Portfolio HAS Stocks) versus inheritance (an Equity IS an Instrument) — why is Portfolio the wrong place to use inheritance?

Test it

`weights()` sums to 100.0; `sum(sector_allocation().values())` equals `total_value()`; changing one Stock's `current_price` changes the Portfolio total without editing `portfolio.py`.

Note: The runnable source for this lab is `labs/lab-52-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 53 — Dunder Methods — `__str__`, `__repr__`, `__len__` and `__add__` to Combine Portfolios

Learning objective: LO7: Apply method overloading through dunder methods so financial objects print, size and merge naturally.

Goal: The learner adds Python's special (dunder) methods to Stock and Portfolio: `__str__` for a human-readable holding line, `__repr__` for the debugger-friendly form, `__len__` so `len(portfolio)` returns the number of holdings, `__getitem__` so a portfolio can be indexed and iterated, `__eq__` to compare by ticker, and `__add__` so two portfolios can be merged with the `+` operator — combining shares where the same ticker appears in both. This is operator overloading applied to a real fund-merger scenario.

What you'll build

Enhanced `stock.py` and `portfolio.py` with six dunder methods, supporting `print()`, `len()`, iteration and `portfolio_a + portfolio_b`. (Tools: uv, Python 3.12, VS Code / Cursor AI assistant.)

Step-by-step

565. Create the project and copy the lab 52 modules.

```
uv init lab-53-dunder-methods && cd lab-53-dunder-methods && cp ../lab-52-  
portfolio-class/*.py .
```

566. Observe the default behaviour first: print a Stock object and note the unhelpful <stock.Stock object at 0x...> output.

```
uv run python -c "from stock import Stock;
print(Stock('D05', 'DBS', 'Banking', 500, 32.10, 38.40))"
```

567. Prompt your AI assistant: "Add dunder methods to these Stock and Portfolio classes. Stock gets `__str__` returning 'D05 | DBS Group | 500 sh @ SGD 38.40 | MV 19,200.00 | P&L +3,150.00', `__repr__` returning `Stock(ticker='D05', shares=500)`, and `__eq__` comparing by ticker. Portfolio gets `__len__` returning the number of holdings, `__getitem__` so it can be indexed and iterated, `__str__` returning a multi-line table of holdings with the total, and `__add__` that merges two Portfolios into a new one, summing shares and recomputing a weighted average buy_price when the same ticker appears in both."

568. Apply the generated dunder methods and read the `__add__` implementation carefully — confirm it returns a NEW Portfolio and does not mutate either operand.

569. Test `__str__` and `__repr__`.

```
uv run python -c "from stock import Stock; s=Stock('D05', 'DBS
Group', 'Banking', 500, 32.10, 38.40); print(str(s)); print(repr(s))"
```

570. Test `__len__` and iteration.

```
uv run python -c "from portfolio import build_demo; p=build_demo();
print(len(p)); [print(h.ticker) for h in p]"
```

571. Build two portfolios that share at least one ticker and merge them with the `+` operator.

```
uv run python merge_demo.py
```

572. Verify the merge conserves value: `(a + b).total_value()` must equal `a.total_value() + b.total_value()`.

573. Verify neither operand was mutated — `len(a)` and `len(b)` are unchanged after the merge.

574. Discuss: when is operator overloading good design, and when does a `+` on a domain object become misleading to the next developer?

Test it

`print(stock)` shows the formatted line; `len(portfolio)` returns the holding count; `(a + b).total_value()` equals `a.total_value() + b.total_value()` and both `a` and `b` are unchanged.

Note: The runnable source for this lab is `labs/lab-53-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 54 — Inheritance — an Instrument Base Class with Equity and Bond Subclasses

Learning objective: LO7: Use inheritance so Equity and Bond extend a shared Instrument class without duplicating code.

Goal: The learner refactors to a proper instrument hierarchy. A base Instrument class holds what every instrument shares (identifier, name, quantity, currency) and defines a value() method that raises NotImplementedError. Equity(Instrument) adds price and dividend_yield; Bond(Instrument) adds face_value, coupon_rate, years_to_maturity and market_yield, and values itself by discounting coupons and principal. Both use super().

What you'll build

An instruments.py module with Instrument, Equity and Bond classes, each subclass calling super().__init__(). (Tools: uv, Python 3.12, VS Code / Cursor AI assistant.)

Step-by-step

575. Create the project.

```
uv init lab-54-inheritance-instruments && cd lab-54-inheritance-instruments
```

576. Sketch the hierarchy on paper: what data does EVERY instrument have, and what belongs only to an Equity or only to a Bond? Only the shared fields go in the base class.

577. Prompt your AI assistant: "Write a Python class hierarchy for financial instruments. Base class Instrument has __init__(identifier, name, quantity, currency='SGD'), a value() method that raises NotImplementedError, and an income() method returning 0.0. Subclass Equity(Instrument) adds price and dividend_yield, overrides value() as quantity * price and income() as value * dividend_yield. Subclass Bond(Instrument) adds face_value, coupon_rate, years_to_maturity and market_yield, and overrides value() to return the present value of the coupon stream plus the discounted principal, multiplied by quantity. Both subclasses must call super().__init__() and each must have a __str__."

578. Save as instruments.py and confirm both subclasses call super().__init__() rather than re-assigning the shared attributes.

579. Verify the inheritance chain with isinstance and the MRO.

```
uv run python -c "from instruments import Instrument, Equity, Bond;
e=Equity('D05','DBS',100,38.40,0.055); print(isinstance(e, Instrument),
Equity.__mro__)"
```

580. Confirm the abstract base refuses to value itself.

```
uv run python -c "from instruments import Instrument;
import traceback"
```

```
try: Instrument('X','Generic',1).value()
except NotImplementedError: print('correctly abstract')"
```

581. Value an Equity and a Bond.

```
uv run python instruments.py
```

582. Sanity-check the bond maths: a bond priced at par ($\text{market_yield} == \text{coupon_rate}$) must value at $\text{face_value} * \text{quantity}$.

```
uv run python -c "from instruments import Bond; b=Bond('SGB10','SG Govt Bond',10,1000,0.03,5,0.03); print(round(b.value(),2))"
```

583. Check a discount bond: raise market_yield above coupon_rate and confirm the value falls below par.

584. Discuss: what code would you have duplicated if Equity and Bond were unrelated classes, and what breaks when a third instrument type is added later?

Test it

`isinstance(equity, Instrument)` is True; `Instrument.value()` raises `NotImplementedError`; a par bond values at exactly $\text{face_value} * \text{quantity}$ and a bond yielding above its coupon values below par.

Note: The runnable source for this lab is `labs/lab-54-*.py`, and it is also included as a section of the combined notebook `labs/Al Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 55 — Polymorphism — One `.value()` Call Across a Mixed Instrument Book

Learning objective: LO7: Apply polymorphism so a single valuation loop handles every instrument type correctly.

Goal: The learner adds a third subclass (a simple `CashDeposit` or `FXForward`), then builds a mixed book of Equity, Bond and Cash objects in one list and values the entire book with a single loop calling `item.value()` — Python dispatches to the correct override at runtime. The lab also adds a duck-typed `report_book()` function that works on any object exposing `.value()`, showing polymorphism does not require inheritance, and proves the open/closed principle by adding a fourth instrument type without editing the valuation loop.

What you'll build

A `book.py` script valuing a heterogeneous instrument book through one polymorphic loop, plus an asset-class breakdown. (Tools: `uv`, Python 3.12, `pandas`, VS Code / Cursor AI assistant.)

Step-by-step

585. Create the project, add pandas and copy the lab 54 hierarchy.

```
uv init lab-55-polymorphism-book && cd lab-55-polymorphism-book && uv add pandas && cp ../lab-54-inheritance-instruments/instruments.py .
```

586. Prompt your AI assistant: "Extend this instruments module with a CashDeposit(Instrument) subclass holding a principal and an annual interest rate, overriding value() as principal plus accrued interest and income() as principal * rate. Then write a function value_book(items) that takes a list of ANY Instrument subclasses and returns a pandas DataFrame with columns Identifier, Asset_Class (the class name), Value and Income, plus the book total — using a single loop that calls item.value() with no isinstance checks and no if/elif on type."

587. Save as book.py and audit the loop: if the generated code contains isinstance() or a type check inside value_book(), that is NOT polymorphism — rewrite it.

588. Build a mixed book of at least two Equities, two Bonds and one CashDeposit in a single Python list.

589. Run the polymorphic valuation.

```
uv run python book.py
```

590. Confirm each row's Asset_Class is derived at runtime via type(item).__name__ — the loop never knew the type in advance.

591. Group the book by Asset_Class to produce the allocation table.

```
uv run python -c "from book import value_book, build_book;
df=value_book(build_book()); print(df.groupby('Asset_Class')
['Value'].sum().round(2))"
```

592. Prove the open/closed principle: add a fourth subclass (e.g. FXForward) with its own value(), append one to the book, and re-run WITHOUT changing value_book().

593. Verify the book total rose by exactly the new instrument's value.

594. Discuss: how does polymorphism reduce risk in a valuation engine compared with a long if/elif chain on instrument type?

Test it

value_book() contains no isinstance or type comparisons; adding a brand-new Instrument subclass to the list changes the book total with zero edits to value_book(), and the Asset_Class column names it correctly.

Note: The runnable source for this lab is labs/lab-55-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 56 — StockTechnicalAnalyzer — an OOP Class Computing SMA, RSI and BUY/SELL Signals

Learning objective: LO7: Encapsulate a complete technical-analysis workflow in one class with modular methods and generated trading signals.

Goal: This is the reference slide 148 activity and the class the Topic 9 capstone consumes. The learner builds StockTechnicalAnalyzer following OOP principles: the ticker and analysis period are instance attributes (encapsulation) and the work is split into modular methods — `fetch_data()` returning a boolean status, `calculate_indicators()` computing a 20-day SMA and 14-day RSI, `generate_signals()` emitting BUY when RSI < 30 and price crosses above the SMA and SELL when RSI > 70 or price crosses below, and `plot_analysis()` drawing price/SMA and RSI matplotlib subplots. A `run_analysis()` method orchestrates the sequence and fails gracefully when the download returns nothing.

What you'll build

An analyzer.py module defining StockTechnicalAnalyzer, producing a signals table and a two-panel PNG chart. (Tools: uv, yfinance, pandas, matplotlib, VS Code / Cursor AI assistant.)

Step-by-step

595. Create the project and add the market-data stack.

```
uv init lab-56-stock-analyzer && cd lab-56-stock-analyzer && uv add yfinance pandas matplotlib
```

596. Prompt your AI assistant with the slide-148 brief: "Write a Python class StockTechnicalAnalyzer following OOP principles to perform stock analysis using yfinance. Encapsulation: store the ticker and analysis period as instance attributes set in `__init__`, with `self.data` starting as None. Modular methods: `fetch_data()` downloads historical data with yfinance and returns a boolean status, storing it on `self.data`; `calculate_indicators()` computes a 20-day Simple Moving Average and a 14-day Relative Strength Index as new columns; `generate_signals()` returns a DataFrame of BUY and SELL signals where BUY is RSI below 30 with the close crossing above the SMA and SELL is RSI above 70 or the close crossing below the SMA; `plot_analysis()` uses matplotlib subplots to visualise close price with the SMA on the top panel and RSI with 30/70 reference lines on the bottom panel, saving to PNG. Add `run_analysis()` that calls the methods in order and returns early with a message if `fetch_data()` fails."
597. Save as analyzer.py and review the RSI calculation — confirm it uses a 14-period average gain over average loss and not a simple percentage change.
598. Confirm the encapsulation: `fetch_data()` must store to `self.data`, and `calculate_indicators()` must read `self.data` rather than take the frame as an argument.
599. Instantiate the analyzer for one ticker and fetch.

```
uv run python -c "from analyzer import StockTechnicalAnalyzer;
a=StockTechnicalAnalyzer('AAPL','6mo'); print(a.fetch_data());
print(a.data.shape)"
```

600. Run the full analysis end to end.

```
uv run python analyzer.py
```

601. Check the indicators: the first 19 SMA values must be NaN and every non-null RSI must sit between 0 and 100.

```
uv run python -c "from analyzer import StockTechnicalAnalyzer;
a=StockTechnicalAnalyzer('AAPL','6mo'); a.run_analysis();
d=a.data.dropna(); print(d['RSI'].min(), d['RSI'].max())"
```

602. Inspect the generated signals table and verify each BUY row really has RSI below 30.

603. Test the failure path with a nonsense ticker — `fetch_data()` must return False and `run_analysis()` must exit cleanly with a message, not a traceback.

```
uv run python -c "from analyzer import StockTechnicalAnalyzer;
a=StockTechnicalAnalyzer('NOTATICKER','6mo'); print(a.run_analysis())"
```

604. Create three analyzer objects for three different tickers and confirm each keeps its own independent `self.data` — the payoff of instance state. Keep `analyzer.py`: lab 61 imports this class into the Streamlit app.

Test it

`run_analysis()` produces a PNG with two panels and a signals table where every BUY has $RSI < 30$ and every SELL has $RSI > 70$ or a downward SMA cross; an invalid ticker returns False from `fetch_data()` with no traceback.

Note: The runnable source for this lab is `labs/lab-56-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Topic 09 — Analyze Finance Data (6%)

Improve code with `apply` and `pipe` · Applications of statistics · Analyse finance data to track changes

Key concepts

- The `.apply()` method runs a function across rows or columns — used to classify credit risk row by row.
- The `.pipe()` method chains transformations into a readable, testable data pipeline.

- Descriptive statistics — mean, median, standard deviation and correlation — summarise risk and return.
- Tracking changes over time reveals trend, volatility and drawdown.
- The finished analysis is delivered as a Streamlit capstone dashboard for stakeholders.

Lab 57 — Credit Risk Classification with .apply()

Learning objective: LO8: Use .apply() with axis=1 to run a multi-column risk calculation across every row of a finance DataFrame.

Goal: This is the reference slide 152 activity. The learner defines determine_loan_risk(row) computing a risk score as $(\text{Debt_to_Income_Ratio} * 50) + (\text{Missed_Payments_Last_2Y} * 10) - (\text{Credit_Score} / 100)$, applies it with axis=1 to create Calculated_Risk_Score, then applies a lambda to that score to create a Risk_Profile column labelling scores above 60 as High, above 30 as Medium and the rest as Low. The lab contrasts .apply() with a manual for loop and shows why row-wise calculations that span several columns need axis=1.

What you'll build

A risk_apply.py script adding Calculated_Risk_Score and Risk_Profile columns to df_finance and exporting df_scored.csv. (Tools: uv, pandas, VS Code / Cursor AI assistant.)

Step-by-step

605. Create the project and add pandas.

```
uv init lab-57-credit-risk-apply && cd lab-57-credit-risk-apply && uv add pandas
```

606. Copy df_finance.csv from lab 43.

```
cp ../lab-43-mock-loan-data/df_finance.csv .
```

607. Prompt your AI assistant with the slide-152 brief: "Write a pandas script demonstrating .apply() on a financial DataFrame named df_finance containing Credit_Score, Debt_to_Income_Ratio and Missed_Payments_Last_2Y. Define a function determine_loan_risk(row) returning $(\text{Debt_to_Income_Ratio} * 50) + (\text{Missed_Payments_Last_2Y} * 10) - (\text{Credit_Score} / 100)$. Use .apply() with axis=1 to create a Calculated_Risk_Score column. Then use .apply() with a lambda on that new column to create a Risk_Profile column labelling scores above 60 as High, above 30 as Medium and the rest as Low. Display the first ten rows and the Risk_Profile value counts."

608. Save as risk_apply.py and confirm axis=1 is present on the first apply — without it pandas would pass columns, not rows.

609. Run the script.

```
uv run python risk_apply.py
```

610. Verify the score by hand for row 0 and compare against the column value.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_scored.csv');  
r=d.iloc[0]; print(round(r.Debt_to_Income_Ratio*50 +  
r.Missed_Payments_Last_2Y*10 - r.Credit_Score/100, 6),  
round(r.Calculated_Risk_Score, 6))"
```

611. Check the banding boundaries: no High row may have a score at or below 60, and no Low row may exceed 30.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_scored.csv');  
print(d.groupby('Risk_Profile')  
['Calculated_Risk_Score'].agg(['min', 'max', 'count']).round(2))"
```

612. Write the equivalent logic as a manual for loop with iterrows() and time both approaches to see the difference.

613. Cross-tabulate Risk_Profile against Loan_Status — does the bank's actual approval decision agree with the model's risk band?

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_scored.csv');  
print(pd.crosstab(d['Risk_Profile'], d['Loan_Status']))"
```

614. Discuss: this score is a linear rule with hard-coded weights. What must a credit-risk team document before such a model is used in a lending decision?

Test it

Every row's Calculated_Risk_Score matches the formula recomputed by hand; the Risk_Profile bands are mutually exclusive with min/max values that respect the 30 and 60 thresholds; the value counts sum to 500.

Note: The runnable source for this lab is labs/lab-57-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 58 — Lambda versus Named Functions in .apply()

Learning objective: LO8: Choose between lambda and named functions in apply, and assess code to identify readability and testability gaps.

Goal: The learner takes the same classification logic and writes it three ways: a one-line lambda, a named function passed by reference, and a vectorised alternative using np.select. Each is benchmarked and, critically, assessed for testability — a named function can be unit-tested in isolation, a lambda buried in an apply cannot. The learner writes pytest cases for the named function, including the boundary values that the lambda version silently gets wrong.

What you'll build

A `classify.py` module with three implementations plus `test_classify.py` containing passing boundary-value tests. (Tools: `uv`, `pandas`, `NumPy`, `pytest`, VS Code / Cursor AI assistant.)

Step-by-step

615. Create the project and add the libraries including `pytest`.

```
uv init lab-58-lambda-vs-function && cd lab-58-lambda-vs-function && uv add pandas numpy pytest
```

616. Copy `df_scored.csv` from lab 57.

```
cp ../lab-57-credit-risk-apply/df_scored.csv .
```

617. Write version A by hand: a `lambda` inside `.apply()` banding `Calculated_Risk_Score` into High/Medium/Low.

618. Prompt your AI assistant: "Refactor this `pandas` `lambda` apply into a named function `classify_risk(score)` with an explicit docstring stating the boundary behaviour at exactly 30 and exactly 60, and a third vectorised version using `numpy.select` with the same conditions. All three must produce identical output on the same column. Then write `pytest` tests for `classify_risk` covering scores of 0, 30, 30.01, 60, 60.01 and 100, plus a `None` input."

619. Save the three implementations in `classify.py` and the tests in `test_classify.py`.

620. Assess the code for gaps: does the `lambda` handle a `NaN` score? Does it define whether exactly 60 is High or Medium? Note every ambiguity you find.

621. Run the tests — expect the boundary cases to expose the ambiguity.

```
uv run pytest test_classify.py -v
```

622. Fix `classify_risk` so the documented boundary behaviour and the tests agree, then re-run until green.

```
uv run pytest test_classify.py -v
```

623. Prove all three versions agree on the real data.

```
uv run python -c "import pandas as pd; from classify import apply_lambda, apply_named, apply_vectorised; d=pd.read_csv('df_scored.csv'); a,b,c = apply_lambda(d), apply_named(d), apply_vectorised(d); print((a==b).all(), (b==c).all())"
```

624. Time the three approaches on the 500-row frame and again on a 500,000-row frame to see where vectorisation wins.

```
uv run python classify.py --benchmark
```

625. Discuss: when is a `lambda` the right choice, and why does a risk model that will face an auditor belong in a named, tested function?

Test it

All pytest boundary tests pass; the lambda, named and vectorised outputs are element-wise identical on `df_scored.csv`; the benchmark shows `np.select` fastest at scale.

Note: The runnable source for this lab is `labs/lab-58-*.py`, and it is also included as a section of the combined notebook `labs/Al Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 59 — Building a clean → enrich → score Pipeline with `.pipe()`

Learning objective: LO8: Chain transformations into a readable, testable data pipeline using `.pipe()`.

Goal: The reference slide 155 activity, extended into a full three-stage pipeline. The learner writes small single-purpose functions — `clean_data` (drop duplicates, coerce dtypes, handle missing income), `enrich_data` (add `Risk_Adjusted_Loan` as `Loan_Amount_Requested * Credit_Score / 850`, plus an income band and a tenure band), and `score_data` (attach the lab 58 risk classification) — then chains them with `.pipe()`. A `summarize_by_sector` step produces the final table of mean `Annual_Income`, total `Risk_Adjusted_Loan` and applicant count. The lab contrasts the pipe chain with the unreadable nested-call equivalent.

What you'll build

A `pipeline.py` module of four pipe-able functions and a single readable chained expression producing `sector_pipeline_summary.csv`. (Tools: `uv`, `pandas`, `pytest`, VS Code / Cursor AI assistant.)

Step-by-step

626. Create the project and add the libraries.

```
uv init lab-59-pipe-pipeline && cd lab-59-pipe-pipeline && uv add pandas
pytest
```

627. Copy `df_finance.csv` and the `classify` module from lab 58.

```
cp ../lab-43-mock-loan-data/df_finance.csv . && cp ../lab-58-lambda-vs-
function/classify.py .
```

628. Write the nested version first, deliberately:

`summarize_by_sector(score_data(enrich_data(clean_data(df))))`. Note how you must read it inside-out.

629. Prompt your AI assistant: "Using a pandas DataFrame named `df_finance`, build a `.pipe()` pipeline. Define `clean_data(df)` that drops duplicate `Applicant_IDs`, coerces numeric columns and fills missing `Annual_Income` with the sector median. Define `filter_by_region(df, region='Central')` that filters to a region. Define `enrich_data(df)` that adds `Risk_Adjusted_Loan` as `Loan_Amount_Requested` multiplied by

Credit_Score divided by 850, plus Income_Band via `pd.cut`. Define `score_data(df)` that adds `Calculated_Risk_Score` and `Risk_Profile`. Define `summarize_by_sector(df)` aggregating by `Employment_Sector` into mean `Annual_Income`, total `Risk_Adjusted_Loan` and applicant count. Chain them all with `.pipe()` in one readable expression and round the final summary to 2 decimals. Every function must take a `DataFrame` as its first argument, return a new `DataFrame`, and never mutate its input."

630. Save as `pipeline.py` and audit the no-mutation rule: each function must start with `df = df.copy()` or use non-mutating operations.

631. Prove immutability — run the pipeline and confirm the original `df_finance` is byte-identical afterwards.

```
uv run python -c "import pandas as pd; from pipeline import run_pipeline;
d=pd.read_csv('df_finance.csv'); before=d.copy(); run_pipeline(d);
print(d.equals(before))"
```

632. Run the full chain.

```
uv run python pipeline.py
```

633. Show the parameterised pipe: pass a `region` argument through the chain with `.pipe(filter_by_region, region='North')` and confirm the summary changes.

634. Test each stage in isolation — a pipeline's value is that every stage is independently testable.

```
uv run pytest test_pipeline.py -v
```

635. Verify the pipeline is order-safe: `score_data` before `enrich_data` must fail loudly rather than produce silent nulls. Add an assertion on the required columns at the top of each stage.

636. Discuss: compare the pipe chain against the nested version you wrote first — which would you rather hand to a colleague reviewing a credit model, and why?

Test it

`run_pipeline()` leaves the input `DataFrame` unmodified; the sector summary has one row per `Employment_Sector` with three rounded metric columns; every stage passes its isolated `pytest` and mis-ordering the stages raises an assertion instead of returning nulls.

Note: The runnable source for this lab is `labs/lab-59-*.py`, and it is also included as a section of the combined notebook `labs/Al Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 60 — Descriptive Statistics, Correlation and Tracking Change Over Time

Learning objective: LO8: Apply descriptive statistics to summarise risk and return, and track trend, volatility and drawdown over time.

Goal: Two halves. First, cross-sectional statistics on the loan book: `.describe()`, mean, median, standard deviation and a correlation matrix of `Credit_Score`, `Debt_to_Income_Ratio`, `Annual_Income` and `Calculated_Risk_Score`, with the median-versus-mean gap exposing income skew. Second, time-series tracking on a price history: daily returns, a rolling 20-day annualised volatility, the cumulative return trend and the maximum drawdown computed from the running peak — the three numbers an investment committee asks for.

What you'll build

A `stats_analysis.py` script producing the loan-book statistics and correlation matrix, plus a drawdown/volatility report and chart. (Tools: `uv`, `pandas`, `NumPy`, `matplotlib`, `yfinance`, VS Code / Cursor AI assistant.)

Step-by-step

637. Create the project and add the libraries.

```
uv init lab-60-statistics-and-trend && cd lab-60-statistics-and-trend && uv  
add pandas numpy matplotlib yfinance
```

638. Copy `df_scored.csv` from lab 57.

```
cp ../lab-57-credit-risk-apply/df_scored.csv .
```

639. Produce the baseline descriptive statistics for the numeric loan columns.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_scored.csv');  
print(d[['Annual_Income', 'Credit_Score', 'Debt_to_Income_Ratio', 'Calculated_  
Risk_Score']].describe().round(2))"
```

640. Compare mean against median for `Annual_Income` and explain the gap — which one belongs in a board report on a skewed distribution?

641. Compute the correlation matrix and identify the strongest driver of the risk score.

```
uv run python -c "import pandas as pd; d=pd.read_csv('df_scored.csv');  
print(d[['Annual_Income', 'Credit_Score', 'Debt_to_Income_Ratio', 'Missed_Paym  
ents_Last_2Y', 'Calculated_Risk_Score']].corr().round(3))"
```

642. Prompt your AI assistant: "Write a Python script that downloads 2 years of daily prices for a ticker with `yfinance` and computes: daily returns from the close, annualised return, annualised volatility as the standard deviation of daily returns times the square root of 252, a rolling 20-day annualised volatility series, the cumulative return series, and the maximum drawdown computed as the minimum of $(\text{cumulative} / \text{cumulative.cummax}() - 1)$ together with the peak and trough dates. Plot the cumulative return with the drawdown shaded underneath in a second `matplotlib` panel and save to PNG. Print all headline statistics."

643. Save as `stats_analysis.py` and check the volatility annualisation uses `sqrt(252)` on DAILY returns, a very common AI-generated error.

644. Run the analysis.

```
uv run python stats_analysis.py
```

645. Sanity-check the drawdown: it must be negative or zero, never positive, and the trough date must fall on or after the peak date.

646. Verify the correlation matrix is symmetric with a unit diagonal.

647. Discuss: a portfolio with the same annualised return but half the maximum drawdown — why does the risk committee prefer it, and which statistic alone would have hidden that difference?

Test it

The correlation matrix is symmetric with 1.0 on the diagonal; maximum drawdown is ≤ 0 with `trough_date >= peak_date`; the rolling volatility series has exactly 19 leading NaN values for a 20-day window.

Note: The runnable source for this lab is `labs/lab-60-*.py`, and it is also included as a section of the combined notebook `labs/Al Assisted Python Programming for Finance-All-Labs.ipynb`.

Lab 61 — Streamlit Capstone Part 3 — Technical Analysis and Risk Scoring Pages

Learning objective: LO8: Integrate the OOP analyzer and the apply/pipe pipeline into the Streamlit application as new interactive pages.

Goal: The capstone continues in the SAME `loan-analytics-app/app.py` from labs 49-50. The learner converts it to a multi-page app with an `st.sidebar.radio` navigation and adds two new pages. The Technical Analysis page imports `StockTechnicalAnalyzer` from lab 56, takes a ticker and period from user input, and renders the price/SMA and RSI charts plus the BUY/SELL signals table. The Risk Scoring page imports the lab 59 pipeline, runs `clean → enrich → score` on the filtered loan book, and shows the `Risk_Profile` distribution, the risk-versus-credit-score scatter and a downloadable scored CSV.

What you'll build

A multi-page `app.py` with Loan Analytics, Technical Analysis and Risk Scoring pages, importing `analyzer.py` and `pipeline.py`. (Tools: uv, Streamlit, pandas, matplotlib, yfinance, VS Code / Cursor AI assistant.)

Step-by-step

648. Return to the capstone app folder from labs 49-50 — do NOT create a new project.

```
cd loan-analytics-app
```

649. Add the market-data dependency and copy in the Topic 8 and Topic 9 modules the app will import.

```
uv add yfinance && cp ../lab-56-stock-analyzer/analyzer.py . && cp
../lab-59-pipe-pipeline/pipeline.py . && cp
../lab-58-lambda-vs-function/classify.py .
```

650. Prompt your AI assistant: "Convert this existing Streamlit app.py into a multi-page app using an st.sidebar.radio navigation with three pages: 'Loan Analytics', 'Technical Analysis' and 'Risk Scoring'. Move all the current KPI cards, filters and tabs under 'Loan Analytics' unchanged. On 'Technical Analysis', import StockTechnicalAnalyzer from analyzer.py, add a st.text_input for the ticker and a st.selectbox for the period, run the analysis on a button click inside a st.spinner, then render the price-with-SMA chart, the RSI chart with 30/70 reference lines, and the BUY/SELL signals table, showing st.error if fetch_data returns False. On 'Risk Scoring', import run_pipeline from pipeline.py, run it on the filtered loan book, and show the Risk_Profile distribution bar chart, a scatter of Credit_Score against Calculated_Risk_Score coloured by Risk_Profile, the sector summary table, and a st.download_button for the scored CSV. Cache the yfinance call with @st.cache_data."
651. Apply the changes to app.py and confirm the labs 49-50 Loan Analytics page still works exactly as before — a regression here means the refactor broke existing functionality.
652. Run the app.

```
uv run streamlit run app.py
```

653. Navigate to Technical Analysis, enter a valid ticker and confirm both charts and the signals table render.
654. Enter an invalid ticker and confirm the page shows an st.error message rather than a traceback — the Topic 5 error-handling discipline applied to a UI.
655. Navigate to Risk Scoring and confirm the Risk_Profile counts match the lab 57 value counts when no sidebar filters are applied.
656. Change a sidebar filter and confirm the Risk Scoring page recomputes on the filtered subset only.
657. Download the scored CSV from the app and verify it opens with the Calculated_Risk_Score and Risk_Profile columns present.

```
uv run python -c "import pandas as pd;
print(pd.read_csv('~Downloads/scored_loans.csv').columns.tolist())"
```

Test it

All three pages load from the sidebar radio; the Loan Analytics page is unchanged from lab 50; an invalid ticker shows st.error not a traceback; the unfiltered Risk_Profile counts match lab 57 exactly.

Note: The runnable source for this lab is labs/lab-61-*.py, and it is also included as a section of the combined notebook labs/AI Assisted Python Programming for Finance-All-Labs.ipynb.

Lab 62 — Streamlit Capstone Finale — Test, Polish and Deploy the Analytics App

Learning objective: LO8: Assess the completed application to identify gaps, test the results, and deploy the finished capstone.

Goal: The course finale. The learner completes the SAME app.py: adds an executive summary header with the headline KPIs, caching on every expensive computation, a data-refresh control, error guards on every empty-data path, and an About page documenting the data source, the risk-model formula and its limitations — the model-transparency the earlier labs argued for. The app is then assessed against a gap checklist, its outputs are regression-tested against the lab 47, 48 and 57 numbers, and it is deployed either locally on the network or to Streamlit Community Cloud.

What you'll build

The finished, tested and deployed Portfolio & Loan Analytics application with a README and a passing test suite. (Tools: uv, Streamlit, pandas, pytest, Git, Streamlit Community Cloud, VS Code / Cursor AI assistant.)

Step-by-step

658. Return to the capstone app folder — this is the final edit of the same app.py.

```
cd loan-analytics-app
```

659. Assess the app against a gap checklist and write down every failure: does every page handle an empty filtered frame? Is every expensive call cached? Does any number appear without units or a currency? Is the risk formula documented anywhere in the UI?

660. Prompt your AI assistant: "Review and complete this Streamlit app. Add an executive summary banner at the top of every page showing total applications, approval rate and total risk-adjusted exposure. Add `@st.cache_data` to every expensive computation with a Refresh Data button calling `st.cache_data.clear()`. Add an 'About & Methodology' page documenting the data source, the exact risk-score formula, the Risk_Profile band thresholds, the SMA and RSI parameters and a plain-English limitations section. Add empty-state guards so every chart shows an informative message instead of raising when the filtered DataFrame is empty. Finally write a requirements.txt and a README.md with run instructions."

661. Apply the changes, then write regression tests asserting the app's helper functions reproduce the known numbers from labs 47, 48 and 57.

662. Run the regression suite.

```
uv run pytest -v
```

663. Run the app and click through all four pages under three different filter combinations, including one that yields zero rows.

```
uv run streamlit run app.py
```

664. Confirm the zero-row case shows informative messages on every page with no traceback in the terminal.
665. Serve the app on the local network so a colleague can open it from their own machine.

```
uv run streamlit run app.py --server.address 0.0.0.0 --server.port 8501
```

666. Export the dependency list and initialise a Git repository for deployment.

```
uv export --no-hashes --format requirements-txt > requirements.txt && git init && git add . && git commit -m 'Portfolio & Loan Analytics capstone'
```

667. Deploy: push to GitHub and connect the repository at share.streamlit.io, selecting `app.py` as the entry point, then open the public URL and verify every page works in the cloud.
668. Present your app: walk through one business question — for example 'which employment sector carries our highest risk-adjusted exposure?' — and answer it live using only the dashboard, without touching Python.

Test it

pytest passes; all four pages render under every filter combination including the empty case; the deployed URL loads and reproduces the same KPI values as the local app; a non-programmer can answer a business question using only the UI.

Note: The runnable source for this lab is `labs/lab-62-*.py`, and it is also included as a section of the combined notebook `labs/AI Assisted Python Programming for Finance-All-Labs.ipynb`.

Financial Formula Reference

These are the formulas used across the labs and the assessment. Know what each measures and when the denominator can be zero — that is where both bugs and exception handling matter.

- **Return on Equity (ROE)** — $\text{Net Income} / \text{Shareholders' Equity} \times 100\%$. Measures profit generated per dollar of equity. Zero or negative equity makes this undefined or misleading.
- **Return on Assets (ROA)** — $\text{Net Income} / \text{Total Assets} \times 100\%$. Measures how efficiently assets generate profit.
- **Net Profit Margin** — $\text{Net Income} / \text{Revenue} \times 100\%$. The share of each revenue dollar that becomes profit. Guard against zero revenue.
- **Gross Profit Margin** — $(\text{Revenue} - \text{COGS}) / \text{Revenue} \times 100\%$. Profitability before operating expenses.
- **Price-to-Earnings (PE) Ratio** — $\text{Share Price} / \text{Earnings Per Share}$. A valuation multiple; negative earnings make it meaningless.

- **Debt-to-Income Ratio** — Total Debt Repayments / Gross Income. A core credit-risk measure in loan assessment.
- **Compound Interest** — $A = P \times (1 + r/n)^{(n \times t)}$, where P is principal, r the annual rate, n the compounding frequency and t the years.
- **Loan Monthly Instalment** — $P \times [r(1+r)^n] / [(1+r)^n - 1]$, where r is the monthly rate and n the number of payments.
- **Future Value of Regular Savings** — $P \times [(1 + r)^n - 1] / r$, used in the retirement calculator lab.
- **Year-on-Year Growth** — $(\text{Current} - \text{Previous}) / \text{Previous} \times 100\%$. Guard against a zero or missing previous value.
- **Simple Moving Average (SMA)** — The mean closing price over the last N periods; the basis of the crossover strategy in Lab 42.
- **Daily Return** — $(\text{Today's Close} - \text{Yesterday's Close}) / \text{Yesterday's Close}$. Chained returns give cumulative performance.

Assessment Preparation

The final assessment is conducted on Day 4 and is open book — you may use these slides, this Learner Guide and your lab files. The passing score is 80%.

- Written Assessment (WA) — Short-Answer Questions (SAQ), open-ended, 1 hour, open book.
- Practical Test (PP) — 7 open-ended practical tasks based on the Financial Data Solutions Inc. scenario, mapped to abilities A1.1–A1.9, 120 minutes, open book.
- The practical questions are based on a financial-services scenario with revenue, stock price, monthly sales and growth-rate datasets supplied in the paper.
- Expect to break a business requirement into functions, calculate growth rates, forecast revenue, classify price movements, debug and optimise a broken function, and document a function properly.
- Practise writing a complete function from a plain-English requirement without an AI assistant — you must be able to read and correct code unaided.
- Revisit any lab whose 'Test it' check you could not pass from memory.
- Sharpen your readiness with the Tertiary Infotech practice exams at <https://exams.tertiaryinfotech.com>

Glossary

- **uv** — A fast Python package and project manager used in every lab; replaces pip, virtualenv and pyenv.

- **pyproject.toml** — The project file uv creates to record your dependencies, making the environment reproducible.
- **Series / DataFrame** — The two core pandas structures: a single labelled column, and a labelled table of columns.
- **yfinance** — A Python package that downloads historical and current market data from Yahoo Finance.
- **Ticker** — The exchange symbol identifying a listed instrument, e.g. D05.SI for DBS Group on the SGX.
- **Market Capitalisation** — Share price multiplied by shares outstanding — the market value of a company's equity.
- **Comprehension** — Compact Python syntax that builds a list, set or dict in one expression, e.g. `[f(x) for x in xs]`.
- **Lambda** — A small anonymous function written inline, commonly passed to `map()` or `filter()`.
- **Exception** — An event during execution that disrupts normal flow, handled with `try/except` so a pipeline survives bad data.
- **Traceback** — The error report Python prints when an exception is unhandled; it names the file, line and error type.
- **groupby** — A pandas operation that splits data into groups, applies an aggregation, and combines the results.
- **Pivot table** — A cross-tabulation that summarises one metric across two categorical dimensions.
- **apply / pipe** — pandas methods that run a function across rows or columns (`apply`) or chain whole-DataFrame transformations (`pipe`).
- **Class / Object** — A class is the blueprint; an object is one instance of it — for example one Stock object per holding.
- **Inheritance** — Deriving a specialised class (Equity, Bond) from a general one (Instrument) without duplicating code.
- **Polymorphism** — One method call behaving correctly across different object types, e.g. `.value()` on any instrument.
- **Streamlit** — A Python framework that turns an analysis script into an interactive web application.
- **Backtest** — Running a trading strategy over historical data to estimate how it would have performed.
- **Look-ahead bias** — A backtesting error where the strategy uses information that was not available at the time of the trade.